普通高等教育“十一五”国家级规划教材

清华大学名优教材立项资助

清华大学 计算机系列教材

王生原 董渊 张素琴 吕映芝 蒋维杜 编著

编译原理（第3版）

清华大学出版社
北京

内 容 提 要

本书介绍程序设计语言编译程序构造的一般原理、基本设计方法和主要实现技术，主要内容包括文法、自动机和语言的基础知识，词法分析，语法分析，语法制导的语义计算，语义分析，中间代码生成，运行时存储组织，代码优化和目标代码生成。

除了基本设计原理外，书中还包含两个小型编译程序的设计实例，可选作课程设计的素材。一个是PL/0语言编译程序，其设计和实现框架贯穿于本书相关章节中；另一个是简单面向对象语言Decaf的编译程序。本书最后还介绍了业界广泛使用的开源编译器GCC及和它紧密相关的Binutils工具链，通过一系列程序实例说明这些工具的作用和基本用法。

本书可作为高等院校计算机科学与技术相关专业的本科生教材，也可作为相关教师、研究生或工程技术人员的参考书。

本书封面贴有清华大学出版社防伪标签，无标签者不得销售。
版权所有，侵权必究。侵权举报电话：010-62782989　13701121933

图书在版编目（CIP）数据

编译原理/王生原等编著. --3版. —北京：清华大学出版社，2015
清华大学计算机系列教材
ISBN 978-7-302-38141-9

Ⅰ. ①编…　Ⅱ. ①王…　Ⅲ. ①编译程序－程序设计－高等学校－教材　Ⅳ. ①TP314

中国版本图书馆CIP数据核字(2014)第227991号

责任编辑：白立军　战晓雷
封面设计：常雪影
责任校对：白　蕾
责任印制：何　芊

出版发行：清华大学出版社
网　　址：http://www.tup.com.cn，http://www.wqbook.com
地　　址：北京清华大学学研大厦A座　　**邮　　编**：100084
社 总 机：010-62770175　　**邮　　购**：010-62786544
投稿与读者服务：010-62776969，c-service@tup.tsinghua.edu.cn
质量反馈：010-62772015，zhiliang@tup.tsinghua.edu.cn
课件下载：http://www.tup.com.cn，010-62795954
印 装 者：北京鑫海金澳胶印有限公司
经　　销：全国新华书店
开　　本：185mm×260mm　　**印　　张**：25.75　　**字　　数**：626千字
版　　次：1998年1月第1版　2015年6月第3版　　**印　　次**：2015年6月第1次印刷
印　　数：1～3000
定　　价：49.00元

产品编号：026315-01

前　言

编译程序(或编译器、编译系统)在计算机科学与技术的发展历史中发挥了巨大作用,是计算机系统的核心支撑软件。“编译原理”一直以来是国内外大学计算机相关专业的重要课程,其知识结构贯穿程序设计语言、系统环境以及体系结构,能以相对独立的视角体现从软件到硬件以及软硬件协同的整机概念;同时,其理论基础又涉及形式语言与自动机、数据结构与算法等计算机学科的许多重要方面,不愧为联系计算机科学理论和计算机系统的典范。这一知识体系所涉及的原理和技术不仅用于编写编译程序,也适用于很多软件的设计。著名的计算机科学家 A. V. Aho 和 J. D. Ullman 在他们的著作中说:“在每一个计算机科学家的研究生涯中,这些原理和技术都会反复用到。”

本书介绍程序设计语言编译程序构造的一般原理、基本设计方法和主要实现技术,主要面向计算机科学与技术相关专业本科生的专业学习和素质培养,也可供从事系统软件和软件工具研究及开发的人员参考。

全书共 12 章。前面几章中有关词法分析和语法分析的部分,基本上延续了本书前两个版本的风格和内容,有利于之前阅读和使用过这套教材的教师和学生衔接。新版本重新组织了语法制导的方法、语义分析、中间代码生成、运行时存储组织、代码优化和目标代码生成等相关内容,进行了适当的充实与删减,力求在各主要知识点之间达到某种较合理的均衡,使学生在本科层次的学习中尽可能对编译程序的构造原理和实现技术从整体知识层面上有较好的掌握。

对于结合实例的讲解,本书沿用了前两个版本使用的 PL/0 编译程序。PL/0 编译程序比较简单,但不失代表性,在编译原理教学中具有广泛的使用基础。通常情况下,学生能够在很短的时间内掌握 PL/0 编译程序的实现脉络,对于快速了解一个具体编译程序的作用和设计思想有很好的帮助。和前面的版本不同,第 3 版中是将 PL/0 编译程序的介绍分散于不同章节中,不同学校或专业的课程可根据自身的情况选择集中学习和分阶段学习。

“编译原理”是一门对实践性要求较高的课程,通常应该设置专门的课程设计。书中涉及两个小型编译程序的设计实例,可选作课程设计的素材。一个是 PL/0 语言编译程序,其设计和实现框架贯穿于全书相关章节;另一个是简单面向对象语言 Decaf 的编译程序,参见第 11 章。不同学校或专业的课程可根据自身的情况制订适当的课程设计方案。

近年来,在许多专业应用场合,熟练使用与编译程序/系统相关的系统级软件工具已成为必须掌握的基本技能之一。为此,本书安排了有关开源的 GCC 编译器和相关工具链 Binutils 的章节(第 12 章),为学生将来有可能从事相关领域的工作进行基本和必要的准备。对于这部分内容,不同学校或专业的课程可根据自身情况引导或建议学生进行适当的训练。

本书的第 1 章、第 2 章和第 3 章由张素琴和王生原共同编写,第 4 章和第 6 章由吕映芝、张素琴和王生原共同编写,第 5 章由吕映芝编写,第 7 章和第 11 章由王生原编写,第 8 章和第 9 章由王生原和蒋维杜共同编写,第 10 章由董渊和王生原共同编写,第 12 章由董渊编写。

附录中包含 PL/0 源程序的 Pascal 版本和 C 版本的代码，Java 版本的代码可从清华大学出版社网站上获取。另外，若相关课程需要用到 Decaf 编译实验框架的代码，任课教师可与清华大学出版社或编者联系（仅限于用作教学资源的共享与交流）。

适合在“编译原理”课程中讲授的内容非常广泛，从国际上的著名教材来看，在侧重点、内容和风格上都有相当大的差异。由于编者水平所限，书中必然存在不当和疏漏之处，诚请广大读者批评指正。

编　者

2015 年 5 月

目　录

第 1 章　引论 …… 1
1.1　什么是编译程序 …… 1
1.2　编译过程和编译程序的结构 …… 2
1.2.1　编译过程概述 …… 2
1.2.2　编译程序的结构 …… 5
1.2.3　编译阶段的组合 …… 6
1.3　解释程序和一些软件工具 …… 7
1.3.1　解释程序 …… 7
1.3.2　处理源程序的软件工具 …… 8
1.4　PL/0 语言编译系统 …… 10
1.4.1　PL/0 语言编译系统构成 …… 11
1.4.2　PL/0 语言 …… 11
1.4.3　类 P-code 语言 …… 14
1.4.4　PL/0 编译程序 …… 15
1.4.5　PL/0 语言编译系统的驱动代码 …… 16
练习 …… 18
第 2 章　文法和语言 …… 19
2.1　文法的直观概念 …… 19
2.2　符号和符号串 …… 20
2.3　文法和语言的形式定义 …… 21
2.4　文法的类型 …… 25
2.5　上下文无关文法及其语法树 …… 26
2.6　句型的分析 …… 29
2.6.1　自上而下的分析方法 …… 30
2.6.2　自下而上的分析方法 …… 30
2.6.3　句型分析的有关问题 …… 31
2.7　有关文法实际应用的一些说明 …… 32
2.7.1　有关文法的实用限制 …… 32
2.7.2　上下文无关文法中的ε规则 …… 33
练习 …… 33
第 3 章　词法分析 …… 37
3.1　词法分析程序的设计 …… 37
3.1.1　词法分析程序和语法分析程序的接口方式 …… 37
3.1.2　词法分析程序的输出 …… 37

3.1.3 将词法分析工作分离的考虑 …… 38
3.1.4 词法分析程序中如何识别单词 …… 39
3.2 PL/0 编译程序中词法分析程序的设计和实现 …… 39
3.3 单词的形式化描述工具 …… 44
3.3.1 正规文法 …… 44
3.3.2 正规式 …… 45
3.3.3 正规文法和正规式的等价性 …… 46
3.4 有穷自动机 …… 47
3.4.1 确定的有穷自动机(DFA) …… 47
3.4.2 不确定的有穷自动机(NFA) …… 49
3.4.3 NFA 转换为等价的 DFA …… 50
3.4.4 确定有穷自动机的化简 …… 52
3.5 正规式和有穷自动机的等价性 …… 54
3.6 正规文法和有穷自动机的等价性 …… 57
3.7 词法分析程序的自动构造工具 …… 58
3.7.1 lex 描述文件中使用的正规表达式 …… 59
3.7.2 lex 描述文件的格式 …… 60
3.7.3 lex 的使用 …… 63
3.7.4 与 yacc 的接口约定 …… 63
练习 …… 64
第 4 章 自顶向下语法分析方法 …… 68
4.1 确定的自顶向下分析思想 …… 68
4.2 LL(1)文法的判别 …… 72
4.3 某些非 LL(1)文法到 LL(1)文法的等价变换 …… 77
4.3.1 提取左公共因子 …… 77
4.3.2 消除左递归 …… 80
4.4 不确定的自顶向下分析思想 …… 84
4.5 LL(1)分析的实现 …… 86
4.5.1 递归下降 LL(1)分析程序 …… 86
4.5.2 表驱动 LL(1)分析程序 …… 92
4.6 LL(1)分析中的出错处理 …… 95
4.6.1 应急恢复 …… 95
4.6.2 短语层恢复 …… 96
4.6.3 PL/0 语法分析程序的错误处理 …… 98
练习 …… 99
第 5 章 自底向上优先分析 …… 103
5.1 自底向上优先分析概述 …… 104
5.2 简单优先分析法 …… 104
5.2.1 优先关系定义 …… 105

5.2.2 简单优先文法的定义 …… 106
5.2.3 简单优先分析法的操作步骤 …… 106
5.3 算符优先分析法 …… 107
5.3.1 直观算符优先分析法 …… 107
5.3.2 算符优先文法的定义 …… 108
5.3.3 算符优先关系表的构造 …… 110
5.3.4 算符优先分析算法 …… 115
5.3.5 优先函数 …… 117
5.3.6 算符优先分析法的局限性 …… 121
练习 …… 121
第6章 LR分析 …… 123
6.1 LR分析概述 …… 123
6.2 LR(0)分析 …… 124
6.2.1 可归前缀和子前缀 …… 125
6.2.2 识别活前缀的有限自动机 …… 127
6.2.3 活前缀及可归前缀的一般计算方法 …… 128
6.2.4 LR(0)项目集规范族的构造 …… 130
6.3 SLR(1)分析 …… 137
6.4 LR(1)分析 …… 144
6.4.1 LR(1)项目集族的构造 …… 145
6.4.2 LR(1)分析表的构造 …… 146
6.5 LALR(1)分析 …… 148
6.6 二义性文法在LR分析中的应用 …… 153
练习 …… 156
第7章 语法制导的语义计算 …… 160
7.1 基于属性文法的语义计算 …… 160
7.1.1 属性文法 …… 160
7.1.2 遍历分析树进行语义计算 …… 164
7.1.3 *S*-属性文法和*L*-属性文法 …… 166
7.1.4 基于*S*-属性文法的语义计算 …… 166
7.1.5 基于*L*-属性文法的语义计算 …… 168
7.2 基于翻译模式的语义计算 …… 172
7.2.1 翻译模式 …… 172
7.2.2 基于*S*-翻译模式的语义计算 …… 173
7.2.3 基于*L*-翻译模式的自顶向下语义计算 …… 174
7.2.4 基于*L*-翻译模式的自底向上语义计算 …… 178
7.3 分析和翻译程序的自动生成工具yacc …… 183
7.3.1 yacc描述文件 …… 184
7.3.2 使用yacc的一个简单例子 …… 187

练习 …… 189
第 8 章　静态语义分析和中间代码生成 …… 195
8.1　符号表 …… 195
8.1.1　符号表的作用 …… 195
8.1.2　符号的常见属性 …… 196
8.1.3　符号表的实现 …… 197
8.1.4　符号表体现作用域与可见性 …… 197
8.1.5　实例：PL/0 编译程序中符号表的设计与实现 …… 199
8.2　静态语义分析 …… 203
8.2.1　静态语义分析的主要任务 …… 203
8.2.2　类型检查 …… 204
8.3　中间代码生成 …… 208
8.3.1　常见的中间表示形式 …… 208
8.3.2　生成抽象语法树 …… 210
8.3.3　生成三地址码 …… 211
8.4　多遍的方法 …… 220
练习 …… 223
第 9 章　运行时存储组织 …… 229
9.1　运行时存储组织概述 …… 229
9.1.1　运行时存储组织的作用与任务 …… 229
9.1.2　程序运行时存储空间的布局 …… 230
9.1.3　存储分配策略 …… 231
9.2　活动记录 …… 234
9.2.1　过程活动记录 …… 234
9.2.2　嵌套过程定义中非局部量的访问 …… 236
9.2.3　嵌套程序块的非局部量访问 …… 239
9.2.4　动态作用域规则和静态作用域规则 …… 240
9.3　过程调用 …… 241
9.4　PL/0 编译程序的运行时存储组织 …… 243
9.4.1　PL/0 程序运行栈中的过程活动记录 …… 244
9.4.2　实现过程调用和返回的类 P-code 指令 …… 245
9.5　面向对象语言存储分配策略 …… 247
9.5.1　类和对象的角色 …… 247
9.5.2　面向对象程序运行时的特征 …… 247
9.5.3　对象的存储组织 …… 248
9.5.4　例程的动态绑定 …… 249
9.5.5　其他话题 …… 251

练习 …… 251
第 10 章　代码优化和目标代码生成 …… 255
10.1　基本块、流图和循环 …… 255
10.1.1　基本块 …… 255
10.1.2　流图 …… 256
10.1.3　循环 …… 257
10.2　数据流分析基础 …… 258
10.2.1　数据流方程的概念 …… 259
10.2.2　到达-定值数据流分析 …… 259
10.2.3　活跃变量数据流分析 …… 262
10.2.4　几种重要的变量使用数据流信息 …… 263
10.3　代码优化技术 …… 268
10.3.1　窥孔优化 …… 270
10.3.2　局部优化 …… 271
10.3.3　循环优化 …… 275
10.3.4　全局优化 …… 278
10.4　目标代码生成技术 …… 279
10.4.1　目标代码生成的主要环节 …… 280
10.4.2　一个简单的代码生成过程 …… 282
10.4.3　高效使用寄存器 …… 285
10.4.4　图着色寄存器分配 …… 288
10.4.5　PL/0 编译器的目标代码生成程序 …… 289
练习 …… 292
第 11 章　课程设计 …… 296
11.1　基于 PL/0 编译器的课程设计 …… 296
11.2　基于 Decaf 编译器的课程设计 …… 297
11.2.1　Decaf 编译器实验的总体结构 …… 298
11.2.2　词法和语法分析(阶段一) …… 300
11.2.3　语义分析(阶段二) …… 303
11.2.4　中间代码生成(阶段三) …… 309
11.2.5　代码优化(阶段四) …… 317
11.2.6　目标代码生成(阶段五) …… 320
11.2.7　基于 Decaf 编译器的课程设计 …… 333
11.3　软件包相关信息说明 …… 335
第 12 章　编译器和相关工具实例——GCC/Binutils …… 336
12.1　开源编译器 GCC …… 336
12.1.1　GCC 介绍 …… 337
12.1.2　GCC 总体结构 …… 338
12.1.3　GCC 编译流程 …… 339

12.1.4 GCC 代码组织 …… 341
12.1.5 小结 …… 341
12.2 开源工具 Binutils …… 341
12.2.1 目标文件 …… 341
12.2.2 汇编器和链接器 …… 342
12.2.3 其他工具 …… 343
12.2.4 小结 …… 343
12.3 编译器和工具使用实例 …… 343
12.3.1 编译特定版本的编译器 …… 343
12.3.2 查看目标文件 …… 347
12.3.3 程序代码优化 …… 349
12.3.4 小结 …… 353
练习 …… 353
附录 A PL/0 编译程序文本 …… 354
参考文献 …… 398

第1章 引 论

1.1 什么是编译程序

编译程序是现代计算机系统的基本组成部分之一，而且多数计算机系统都配有不止一种高级语言的编译程序，对有些高级语言甚至配置了几个不同性能的编译程序。从功能上看，一个编译程序就是一个语言翻译程序。语言翻译程序把一种语言（称作源语言）书写的程序翻译成另一种语言（称作目标语言）的等价程序。比如，汇编程序是一个翻译程序，它把汇编语言程序翻译成机器语言程序。如果源语言是像 FORTRAN、Pascal 或 C 那样的高级语言，目标语言是像汇编语言或机器语言那样的低级语言，则这种翻译程序称作编译程序。把编译程序看成一个“黑盒子”，它所执行的转换工作可用图 1.1 来说明。

一个编译程序的重要性体现在它使得多数计算机用户不必考虑与机器有关的烦琐细节，使程序员和程序设计专家独立于机器，这对于当今机器的数量和种类持续不断地增长的年代尤为重要。

使用过计算机的人都知道，除了编译程序外，还需要一些其他程序才能生成一个可在计算机上执行的目标程序。下面分析一个程序设计语言程序的典型的处理过程（见图 1.2），可以从中进一步了解编译程序的作用。

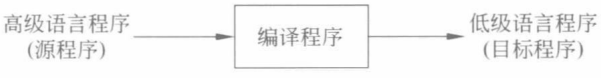


图 1.1 编译程序的功能

图 1.2 高级语言程序的处理过程

一个源程序有时可能分成几个模块存放在不同的文件里，将这些源程序汇集在一起的任务，由一个叫做预处理程序的程序来完成，有些预处理程序也负责宏展开，像 C 语言的预处理程序要完成文件合并、宏展开等任务。图 1.2 中的编译程序生成的目标程序是汇编代

码形式，需要经由汇编程序翻译成可再装配(或可重定位)的机器代码，再经由装配/连接编辑程序与某些库程序连接成真正能在机器上运行的代码。也就是说，一个编译程序的输入可能要由一个或多个预处理程序来产生；另外，为得到能运行的机器代码，编译程序的输出可能仍需要进一步地处理。

编译程序的基本任务是将源语言程序翻译成等价的目标语言程序。源语言的种类成千上万，从常用的诸如 FORTRAN、Pascal、C、Java 和 C++ 等语言，到各种各样的计算机应用领域的专用语言；而目标语言也是种类繁多的，加上编译程序由于构造不同，所执行的具体功能有差异，又分成了各种类型，如一趟编译、多趟编译、具有调试或优化功能的编译等。尽管存在这些明显的复杂因素，但是任何编译程序所必须执行的主要任务基本是一样的，通过理解这些任务，使用同样的基本技术，可以为各种各样的源语言和目标语言设计和构造编译程序。

据说第一个编译程序出现在 20 世纪 50 年代早期，很难讲出确切的时间，因为当初大量的实验和实现工作是由不同的小组独立完成的，多数早期的编译工作是将算术公式翻译成机器代码。用现在的标准来衡量，当时的编译程序能完成的工作十分初步，如只允许简单的单目运算，数据元素的命名方式有很多限制，然而它们奠定了对高级语言编译程序的研究和开发的基础。20 世纪 50 年代中期出现了 FORTRAN 等一批高级语言，相应的一批编译系统开发成功。随着编译技术的发展和社会对编译程序需求的不断增长，20 世纪 50 年代末有人开始研究编译程序的自动生成工具，提出并研制编译程序的编译程序。它的功能是以任一语言的词法规则、语法规则和语义解释出发，自动产生该语言的编译程序。目前很多自动生成工具已广泛使用，如词法分析程序的生成系统 LEX，语法分析程序的生成系统 YACC 等。

1.2 编译过程和编译程序的结构

1.2.1 编译过程概述

编译程序完成从源程序到目标程序的翻译工作，是一个复杂的整体的过程。从概念上来讲，一个编译程序的整个工作过程是划分成阶段进行的，每个阶段将源程序的一种表示形式转换成另一种表示形式，各个阶段进行的操作在逻辑上是紧密连接在一起的，图 1.3 给出了一个编译过程的各个阶段，这是一种典型的划分方法，将编译过程划分成词法分析、语法分析、语义分析、中间代码生成、代码优化和目标代码生成 6 个阶段。

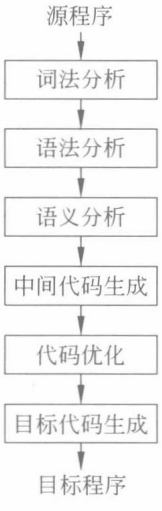


图 1.3 编译的各个阶段

下面通过源程序在不同阶段所被转换成的表示形式来介绍各个阶段的任务。

1. 词法分析

词法分析是编译过程的第一个阶段。这个阶段的任务是从左到右一个字符一个字符地读入源程序，对构成源程序的字符流进行扫描和分解，从而识别出一个个单词(一些场合下也称单词符号或符号)。这里所谓的单词是指逻辑上紧密相连的一组

字符，这些字符具有集体含义。例如，标识符是由字母字符开头，后跟字母、数字字符的字符序列组成的一种单词。保留字（关键字或基本字）是一种单词，此外还有算符、界符等。例如，某源程序片段如下：

```
begin var sum,first,count:real;sum :=first+count * 10 end.
```

词法分析阶段将构成这段程序的字符组成了如下单词序列：

(1) 保留字 begin　(2) 保留字 var　(3) 标识符 sum
(4) 逗号，　(5) 标识符 first　(6) 逗号，
(7) 标识符 count　(8) 冒号：　(9) 保留字 real
(10) 分号；　(11) 标识符 sum　(12) 赋值号 :=
(13) 标识符 first　(14) 加号+　(15) 标识符 count
(16) 乘号 *　(17) 整数 10　(18) 保留字 end
(19) 界符 .

可以看出，5 个字符 b、e、g、i 和 n 构成了一个称为保留字的单词 begin，两个字符：和＝构成了表示赋值运算的符号:=。这些单词间的空格在词法分析阶段都被滤掉了。

用 id1、id2 和 id3 分别表示 sum、first 和 count 这 3 个标识符的内部形式，那么经过词法分析后上述程序片段中的赋值语句 sum :=first+count * 10 则表示为 id1 :=id2+id3 * 10。

2. 语法分析

语法分析是编译过程的第二个阶段。语法分析的任务是在词法分析的基础上将单词序列分解成各类语法短语，如“程序”、“语句”、“表达式”等。这种语法短语也称为语法单位，可表示成语法树，比如上述程序段中的单词序列

```
id1 :=id2+id3 * 10
```

经语法分析得知其是 Pascal 语言的赋值语句，表示成如图 1.4 所示的语法树或者如图 1.5 所示的简捷形式的语法树。

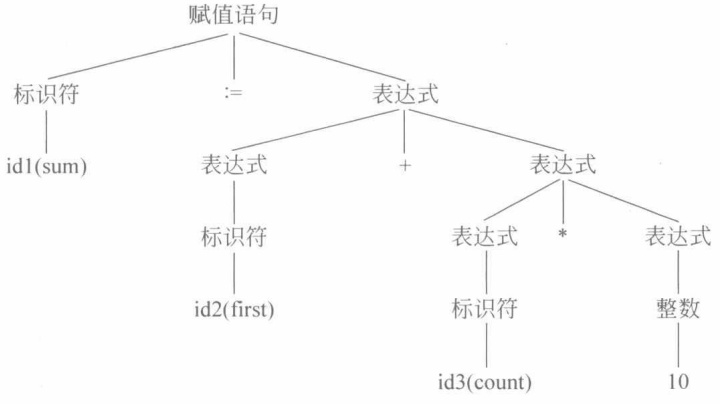


图 1.4　语句 id1 :=id2+id3 * 10 的语法树

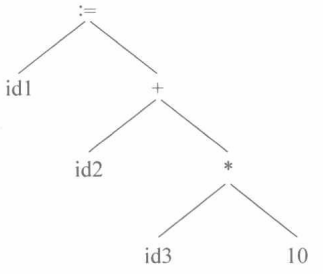


图 1.5　语句 id1 :=id2+id3 * 10 的语法树的简捷形式

语法分析所依据的是语言的语法规则，即描述程序结构的规则。通过语法分析确定整个输入串是否构成一个语法上正确的程序。

程序的结构通常是由递归规则表示的，例如，可以用下面的规则来定义表达式：

(1) 任何标识符是表达式。

(2) 任何常数(整常数、实常数)是表达式。

(3) 若表达式 1 和表达式 2 都是表达式，那么：

表达式 1＋表达式 2

表达式 1 * 表达式 2

(表达式 1)

都是表达式。

类似地，语句也可以递归地定义，如

(1) 标识符 :＝表达式 是语句。

(2) while(表达式) do 语句

和

If (表达式) then 语句 else 语句

都是语句。

上述赋值语句 id1 :＝id2＋id3 * 10 之所以能表示成图 1.4 所示的语法树，依据的是赋值语句和表达式的定义规则。

词法分析和语法分析本质上都是对源程序的结构进行分析。但词法分析的任务仅对源程序进行线性扫描即可完成，比如识别标识符，因为标识符的结构是字母打头的字母和数字串，这只要顺序扫描输入流，遇到既不是字母又不是数字的字符时，将前面所发现的所有字母和数字组合在一起构成标识符单词即可。但这种线性扫描不能用于识别递归定义的语法成分，比如不能用此办法去匹配表达式中的括号。

3. 语义分析

语义分析是审查源程序有无语义错误，为代码生成阶段收集类型信息。例如，语义分析的一个工作是进行类型审查，审查每个算符是否具有语言规范允许的运算对象，当不符合语言规范时，编译程序应报告错误。有的编译程序要对实数用作数组下标的情况报告错误。某些语言规定运算对象可被强制转换数据类型，那么当二目运算施于一个整型对象和一个实型对象时，编译程序应将整型对象转换成实型对象进行处理而不能认为是源程序的错误，假如在语句 sum :＝first＋count * 10 中，count 是实型，10 是整型，则语义分析阶段进行类型审查之后，在语法分析所得到的分析树上增加一个语义处理结点 inttoreal，表示整型 10 变成实型 10.0 的一目算符，则图 1.5 的树变成图 1.6 所示的树。

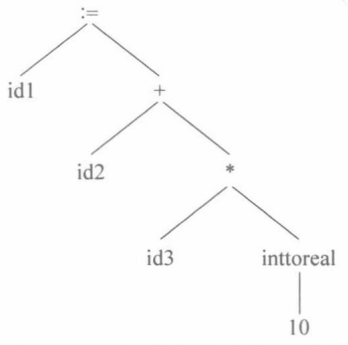


图 1.6　插入语义处理结点的树

4. 中间代码生成

在进行了上述的语法分析和语义分析阶段的工作之后，有的编译程序将源程序变成一种内部表示形式，这种内部表示形式叫做中间语言或中间代码。所谓“中间代码”是一种结构简单、含义明确的记号系统，这种记号系统可以设计为多种多样的形式，重要的设计原则为两点：一是容易生成；二是容易将它翻译成目标代码。很多编译程序采用了

一种近似“三地址指令”的“四元式”中间代码，这种四元式的形式为

(运算符，运算对象 1，运算对象 2，结果)

例如，源程序 sum :=first+count * 10 可生成如图 1.7 所示的四元式序列，其中 t_i(i=1，2，3)是编译程序生成的临时名字，用于存放运算的中间结果。

5. 代码优化

这一阶段的任务是对前一阶段产生的中间代码进行变换或进行改造，目的是使生成的目标代码更为高效，即省时间和省空间。例如，图 1.7 的代码可变换为图 1.8 的代码，仅剩两个四元式而执行同样的计算。也就是说，在编译程序的这个阶段已经把将 10 转换成实型数的代码化简了，同时因为 t_3 仅仅用来将其值传递给 id1，也可以被化简，这只是优化工作的两个方面；此外，诸如公共子表达式的删除、强度削弱、循环优化等优化工作将在第 10 章详细介绍。

(1)	(inttoreal	10	—	t_1)
(2)	(*	id3	t_1	t_2)
(3)	(+	id2	t_2	t_3)
(4)	(:=	t_3	—	id1)

图 1.7 中间代码

(*)	id3	10.0	t_1)	
(+	id2	t_1	id1)	

图 1.8 优化后的中间代码

(1)	MOV	id3,	R2
(2)	MUL	#10.0,	R2
(3)	MOV	id2	R1
(4)	ADD	R2,	R1
(5)	MOV	R1,	id1

图 1.9 目标代码

6. 目标代码生成

这一阶段的任务是把中间代码变换成特定机器上的绝对指令代码或可重定位的指令代码或汇编指令代码。这是编译的最后阶段，它的工作与硬件系统结构和指令含义有关，这个阶段的工作很复杂，涉及硬件系统功能部件的运用、机器指令的选择、各种数据类型变量的存储空间分配以及寄存器和后缓寄存器的调度等。

例如，使用两个寄存器(R1 和 R2)，可能将图 1.8 所示的中间代码生成如图 1.9 所示的某种汇编代码。第 1 条指令将 id3 的内容送至寄存器 R2，第 2 条指令将其与实常数 10.0 相乘，这里用 # 表明 10.0 处理为常数，第 3 条指令将 id2 移至寄存器 R1，第 4 条指令将 R1 和 R2 中的值相加，第 5 条指令将寄存器 R1 的值移到 id1 的地址中。

上述编译过程的阶段划分是一个典型处理模式，事实上并非所有的编译程序都分成这样几个阶段，有些编译程序并不需要生成中间代码，有些编译程序不进行优化，即优化阶段可省去，有些最简单的编译程序在语法分析的同时产生目标指令代码，如 1.4 节介绍的 PL/0 语言编译程序。不过多数实用的编译程序都包含上述几个阶段的工作过程。

1.2.2 编译程序的结构

上述编译过程的 6 个阶段的任务可以分别由 6 个模块完成，分别称作词法分析程序、语法分析程序、语义分析程序、中间代码生成程序、代码优化程序和目标代码生成程序。此外，一个完整的编译程序还必须包括表格管理程序和出错处理程序。图 1.10 给出了一个典型的编译程序结构框图。

表格管理和出错处理与上述 6 个阶段都有联系。编译过程中源程序的各种信息被保留在种种不同的表格里，编译各阶段的工作都涉及构造、查找或更新有关的表格，因此需要有

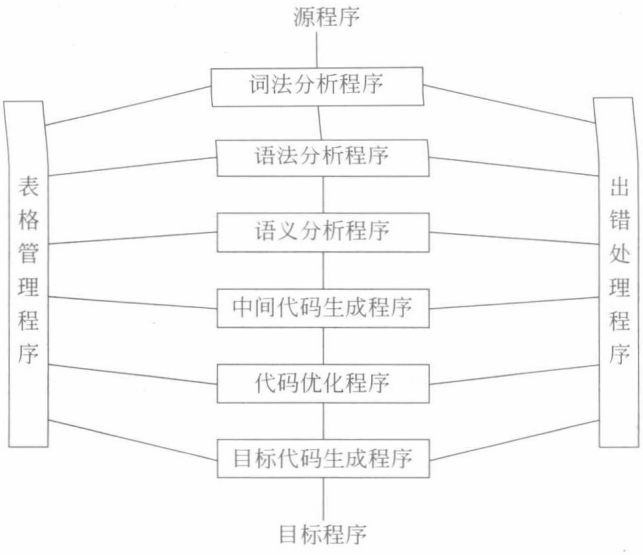


图 1.10　编译程序结构框图

表格管理的工作。如果编译过程中发现源程序有错误，编译程序应报告错误的性质和错误发生的地点，并且将错误所造成的影响限制在尽可能小的范围内，使得源程序的其余部分能继续被编译下去，有些编译程序还能自动校正错误，这些工作由出错处理程序完成。

1.2.3　编译阶段的组合

1.2.1 节所讨论的编译阶段的划分是编译程序的逻辑组织。有时把编译的过程分为前端(front end)和后端(back end)，前端的工作主要依赖于源语言而与目标机无关。前端通常包括词法分析、语法分析、语义分析和中间代码生成这些阶段，某些优化工作也可在前端做，还包括与前端每个阶段相关的出错处理工作和符号表管理工作。后端指的是那些依赖于目标机而一般不依赖于源语言，只与中间代码有关的那些阶段的工作，即目标代码生成，以及相关出错处理和符号表操作。

若按照这种组合方式实现编译程序，可以设想，某一编译程序的前端加上相应的后端则可以为不同的机器构成同一个源语言的编译程序。也可以设想，不同语言编译的前端生成同一种中间语言，再使用一个共同的后端，则可为同一机器生成几个语言的编译程序。

一个编译过程可由一遍、两遍或多遍完成。所谓“遍”，也称作“趟”，是对源程序或其等价的中间语言程序从头到尾扫描并完成规定任务的过程。每一遍扫描可完成上述一个阶段或多个阶段的工作。例如，一遍可以只完成词法分析工作，一遍完成词法分析和语法分析工作，甚至一遍完成整个编译工作。对于多遍的编译程序，第一遍的输入是用户书写的源程序，最后一遍的输出是目标程序，其余是上一遍的输出为下一遍的输入。在实际的编译系统的设计中，编译的几个阶段的工作究竟应该怎样组合，即编译程序究竟分成几遍，参考的因素主要是源语言和机器(目标机)的特征。例如，源语言的结构直接影响编译的遍的划分；像 PL/1 或 ALGOL 68 那样的语言，允许名字的说明出现在名字的使用之后，那么在看到名字之前是不便为包含该名字的表达式生成代码的，这种语言的编译程序至少分成两遍才容易生成代码。另外，机器的情况，即编译程序工作的环境也影响编译程序的遍数的划分。一个

多遍的编译程序可以比一遍的编译程序少占内存，遍数多一点，整个编译程序的逻辑结构可能更清晰，但遍数多也意味着增加读写中间文件的次数，势必消耗较多时间，显然会比一遍的编译程序要慢。

1.3 解释程序和一些软件工具

1.3.1 解释程序

编译程序是一个语言处理程序，它把一个高级语言程序翻译成某个机器的汇编语言程序或二进制代码程序，这个二进制代码程序在机器上运行以生成结果。因此通过编译程序使得程序员可以先准备好一个在该机器上运行的程序，然后这个程序便会以机器的速度运行。但是在不把整个程序全部翻译完成之后，这个程序是不能开始运行，也不能产生任何结果的。编译和运行是两个独立分开的阶段。但在一个交互环境中，并不需要将这两个阶段分隔开。这里介绍另一种语言处理程序，叫解释程序，它不需要在运行前先把源程序翻译成目标代码，也可以实现在某台机器上运行程序并生成结果。

解释程序接受某个语言的程序并立即运行这个源程序。它的工作模式是一个个的获取、分析并执行源程序语句，一旦第一个语句分析结束，源程序便开始运行并且生成结果，它特别适合程序员以交互方式工作的情况，即希望在获取下一个语句之前了解每个语句的执行结果，允许执行时修改程序。

著名的解释程序有 BASIC 语言解释程序、LISP 语言解释程序、UNIX 命令语言(shell)解释程序、数据库查询语言 SQL 解释程序以及 Java 语言环境中(见图 1.15)的 BYTECODE 解释程序。

图 1.11 表示了解释程序的功能，图 1.12 表示了编译程序和解释程序的不同工作模式。

图 1.11 解释程序的功能

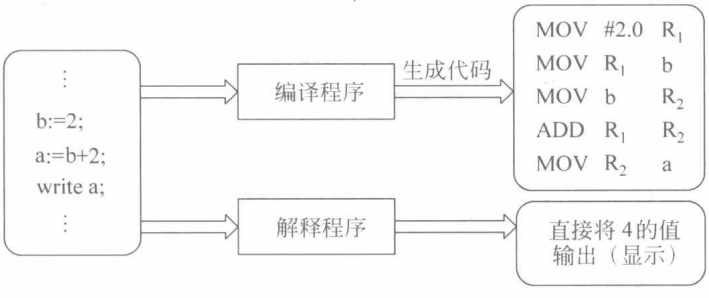


图 1.12 编译程序和解释程序的不同工作模式

解释程序的输入包括源程序和源程序的初始数据(输入数据)，它不生成目标代码，直接

输出结果。编译程序和解释程序的存储组织也有很大不同。经由编译程序处理时，在源程序被编译的阶段，存储区中要为源程序（中间形式）和目标代码开辟空间，要存放编译用的各种表格，如符号表。在目标代码运行阶段，存储区中主要是目标代码和数据，编译所用的任何信息都不再需要了。

解释程序一般是把源程序一个语句一个语句地进行语法分析，转换为一种内部表示形式，存放在源程序区。例如，BASIC 解释程序将 LET 和 GOTO 这样的关键字表示为一个字节的操作码，标识符用其在符号表的入口位置表示。因为解释程序允许在执行用户程序时修改用户程序，这就要求在解释程序工作的整个过程中，源程序、符号表等内容始终存放在存储区中，并且存放格式要设计得易于使用和修改。

图 1.13 和图 1.14 分别表示了编译程序的编译阶段、运行阶段以及解释程序的存储区内容。

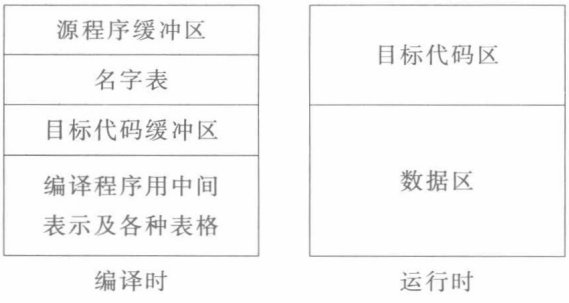


图 1.13　编译程序的编译阶段和运行阶段的存储区内容

图 1.14　解释程序的存储区内容

程序的解释是非常慢的，有时一个高级语言源程序的解释会比运行等价的机器代码程序慢 100 倍。因此当程序的运行速度非常重要时，是不能采用解释方式的。另外，解释程序的空间开销也是比较大的。

编译程序和解释程序是两类重要的高级语言处理程序。有些语言，如 BASIC、LISP 和 Pascal 等，既有编译程序，也有解释程序。Java 语言的处理环境既有编译程序，也有解释程序。图 1.15 展示了 Java 语言处理环境。

1.3.2　处理源程序的软件工具

人们已经认识到，为了提高软件的开发效率和质量，需要有一套软件开发过程所遵循的规范或标准，应使用先进的软件开发方法，并有相应的软件工具的支持。而这些软件工具的开发，其中很多要用到编译的原理和技术。实际上，编译程序本身也是一种软件开发工具，有了它们才能使用编程效率高的高级语言来编写程序。为了进一步提高编程效率，缩短调试时间，软件工作人员研制了许多针对源程序处理的软件工具，这些工具首先要像编译程序那样对源程序进行分析。下面是这些工具的一些例子。

1. 语言的结构化编辑器

用户可使用这种编辑器在语言的语法制导下编制出所需的源程序。结构化编辑器不仅具有通常的正文编辑器的正文编辑和修改功能，而且还能像编译程序那样对源程序正文进行分析。因此，结构化编辑器能够执行一些对正确编制程序有帮助的附加的任务。例如，它

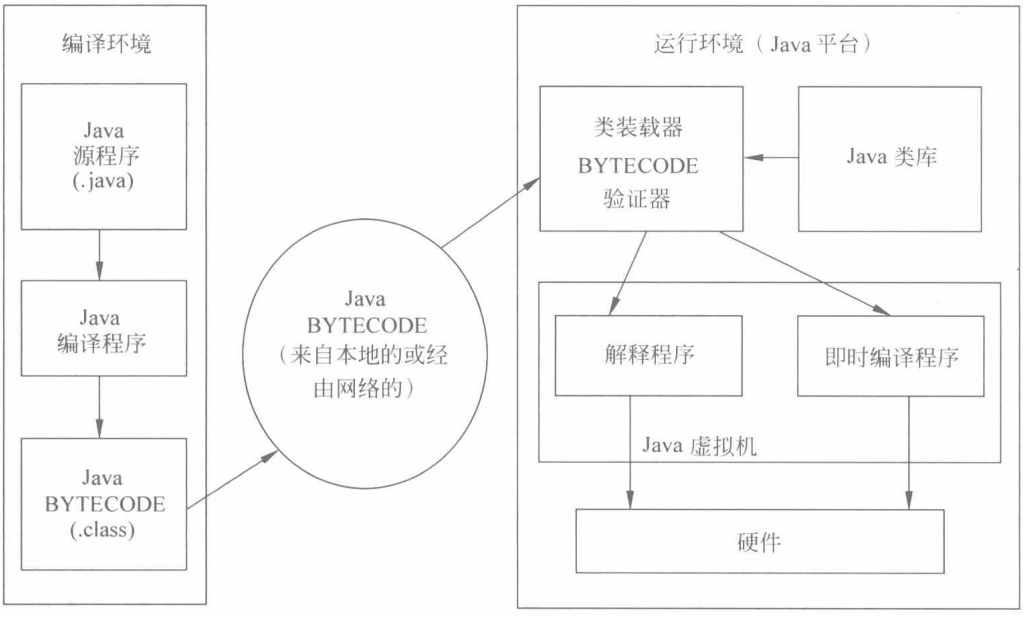


图 1.15　Java 语言环境

能够检查用户的输入是否正确，能够自动地提供关键字，当用户输入 if 后，编辑器立即显示 then 并将这两个关键字之间必须出现的条件留给用户输入，还能够检查 begin 或左括号与 end 或右括号是否相匹配等。由于结构化编辑器具有上述功能，既可保证编写出的源程序无语法错误，并有统一的、可读性好的程序格式，这无疑将会提高程序的开发效率和质量。这类商用产品很多，如 Turbo-Edit、Editplus 和 Ultraedit 等。很多集成开发环境中里也都包含这种类似的工具，如 JBuilder 中就有 Java 程序的结构化编辑器。

2. 语言程序的调试工具

调试是软件开发过程中的一个重要环节，结构化编辑器只能解决语法错误的问题，而对一个已通过编译的程序来说，需进一步了解的是程序执行的结果与编程人员的意图是否一致，程序的执行是否实现预计的算法和功能。这种算法的错误或者程序没能反映算法的功能等问题就要用调试器来协助解决。有一种调试器允许用户使用源程序正文和它的符号来调试，即一行一行地跟踪程序，查看变量和数据结构的变化以进行调试工作。当然，这些符号的信息必须由编译程序提供。调试器的实现可以有很多途径。其中一种是写一个解释器，以交互的方式翻译和执行每一行，它必须维护其所有的运行时的资源以保证在程序执行期间可以很容易地查询不同变量的当前值。如果不通过解释手段调试，而是在编译之后的代码上进行调试，编译程序必须在目标代码（汇编）生成时同时生成特定的调试信息，例如，关联标识符和它表示的地址的信息，用于无歧义地引用一个声明了多次的标识符的信息等。调试功能越强，实现越复杂，它涉及源程序的语法分析和语义处理技术。

3. 程序格式化工具

程序格式化工具分析源程序并以使程序结构变得清晰可读的形式打印出来。例如，注释可以以一种专门的字形出现，且语句的嵌套层次结构可以用缩排方式（齿形结构）表示

出来。

4. 语言程序测试工具

语言程序的测试工具有两种：静态分析器和动态测试器。

静态分析器是在不运行程序的情况下对源程序进行静态分析，以发现程序中潜在的错误或异常。它对源程序进行语法分析并制定相应表格，检查变量定值与引用的关系，如某变量未被赋值就被引用，或定值后未被引用，或多余的源代码等一些编译程序的语法分析发现不了的错误。

动态测试工具也是首先对源程序进行分析，在分析基础上将用于记录和显示程序执行轨迹的语句或函数插入到源程序的适当位置，并用测试用例记录和显示程序运行时的实际路径，将运行结果与期望的结果进行比较分析，帮助编程人员查找问题。

5. 程序理解工具

程序理解工具对程序进行分析，确定模块间的调用关系，记录程序数据的静态属性和结构属性，并画出控制流程图，帮助用户理解程序。

6. 高级语言之间的转换工具

由于计算机硬件的不断更新换代，更新更好的程序设计语言的推出为提高计算机的使用效率提供了良好条件，然而一些已有的非常成熟的软件如何在新机器新语言情况下使用呢？为了减少重新编制程序所耗费的人力和时间，就要解决如何把一种高级语言转换成另一种高级语言，乃至汇编语言转换成高级语言的问题。这种转换工作要对被转换的语言进行词法和语法分析，只不过生成的目标语言是另一种高级语言而已。这与实现一个完整的编译程序相比工作量要少些。

1.4 PL/0 语言编译系统

PL/0 语言编译系统是世界著名计算机科学家 N. Wirth 编写的，它由编译程序和解释程序两部分组成。对 PL/0 编译程序进行实例分析，有助于对一般编译过程和编译程序结构的理解。PL/0 编译程序的源语言为 PL/0，目标语言是一个类 P-code 的代码。本书在介绍有关编译技术和方法时，会以 PL/0 编译程序的相关内容为例来说明这些方法是如何在实践中得以应用的。

N. Wirth 原本使用的编写语言是 Pascal(参见附录 A.1)，考虑到普遍性和通用性，本书给出了 PL/0 编译系统的 C 语言版本(参见附录 A.2)。C 语言版本完整保持与 Pascal 版本的结构一致，函数(过程)、变量等也使用同样的名字。附录 A 中代码的电子版可从清华大学出版社网站中的本书相关电子资源中下载。此外，本书相关电子资源中还包括 Java 版本的 PL/0 编译系统，也是原先 Pascal 版本的翻版。如不特别指明，本书后续部分在涉及 PL/0 编译程序的例子中均指 C 语言的版本。

本节首先介绍 PL/0 语言编译系统的构成，其次给出 PL/0 编译程序的源语言和目标语言，最后简单介绍 PL/0 编译程序的基本组成。PL/0 编译程序的实现细节将在后续章节中穿插讨论。

1.4.1 PL/0 语言编译系统构成

PL/0 语言编译系统由编译程序和解释程序两部分组成，分别称为 PL/0 编译程序和类 P-code 解释程序。PL/0 语言程序被 PL/0 编译程序转换为等价的类 P-code 程序。当编译程序正常结束时，PL/0 语言编译系统会调用解释程序（也称类 P-code 虚拟机），解释执行所生成的目标程序，如图 1.16 所示。

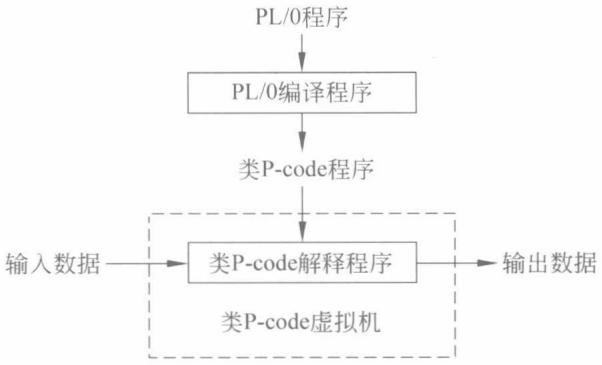


图 1.16　PL/0 语言编译系统

为了描述方便，通常用 T 形图来表示一个编译程序涉及的三个方面的语言，即源语言、目标语言和编译程序的书写语言（实现语言）。T 形图的左上角表示源语言，右上角表示目标语言，底部表示书写语言，PL/0 编译程序的 T 形图如图 1.17 所示。

图 1.17　PL/0 编译程序 T 形图

PL/0 编译程序将 PL/0 源程序翻译成类 P-code 目标程序，源语言为 PL/0，目标语言为类 P-code。PL/0 编译程序可用 C、Pascal 或 Java 等各种语言书写。

在本书的 PL/0 语言编译系统所有版本中，类 P-code 虚拟机都是采用与编写 PL/0 编译程序相同的语言（即 C、Pascal 或 Java）来仿真或解释实现的。读者可通过阅读附录 A 源代码中名为 interpret 的函数（或过程）理解类 P-code 虚拟机的解释实现过程。

对 PL/0 编译程序和类 P-code 虚拟机进行联编，就可以生成目标平台上可执行的 PL/0 语言编译系统。用高级语言实现的类 P-code 虚拟机是平台无关的，因此 PL/0 语言编译系统可方便地移植到任何目标平台。

1.4.2 PL/0 语言

PL/0 语言的程序结构很简单，是 Pascal 的一个子集。图 1.18 和图 1.19 分别给出了两个小的 PL/0 程序例子。

```
const          a=10;          /*常量说明部分*/
var            b,c;           /*变量说明部分*/
procedure      p;             /*过程说明部分*/
    begin
      c:=b+a
    end;
begin                         /*主程序*/
    read(b);
    while b#0 do
      begin
        call p; write(2*c); read(b)
      end
end.
```

图 1.18　PL/0 源程序例 1

```
var m,n,r,q;
{计算 m 和 n 的最大公约数 }
procedure gcd;
    begin
        while r#0 do
            begin
                q :=m / n;
                r :=m - q * n;
                m :=n;
                n :=r;
            end
    end;
begin
    read(m);
    read(n);
    {为了方便,规定 m>=n }
    if m<n then
        begin
            r :=m;
            m :=n;
            n :=r;
        end;
    begin
        r:=1;
        call gcd;
        write(m);
    end;
end.
```

图 1.19　PL/0 源程序例 2

在实践中,程序语言的语法描述常采用一种称为扩展巴克斯范式(EBNF)的形式来描述。采用这种形式的好处是简洁和易读,比较适合于编译程序的手工构造。在后面的章节

里介绍的 PL/0 编译程序手工构造过程都将基于这种 EBNF 形式的描述。

表 1.1 是 PL/0 语言语法的一个 EBNF 描述。

表 1.1 PL/0 语言语法的 EBNF 描述

PL/0 语法单位	EBNF 描述
<程序>	::=<分程序>.
<分程序>	::=[<常量说明部分>][<变量说明部分>][<过程说明部分>]<语句>
<常量说明部分>	::=**const**<常量定义>{,<常量定义>};
<常量定义>	::=<id>=<integer>
<变量说明部分>	::=**var**<id>{,<id>};
<过程说明部分>	::=<过程首部><分程序>{;<过程说明部分>};
<过程首部>	::=**procedure** <id>;
<语句>	::=<赋值语句>|<条件语句>|<当型循环语句>|<过程调用语句>|<读语句>|<写语句>|<复合语句>|<空语句>
<赋值语句>	::=<id>:=<表达式>
<复合语句>	::=**begin**<语句>{;<语句>} **end**
<空语句>	::=ε
<条件>	::=<表达式><关系运算符><表达式>|**odd** <表达式>
<表达式>	::=[+|-]<项>{<加减运算符><项>}
<项>	::=<因子>{<乘除运算符><因子>}
<因子>	::=<id>|<integer>|'('<表达式>')'
<加减运算符>	::=+|-
<乘除运算符>	::=*|/
<关系运算符>	::==|#|<|<=|>|>=
<条件语句>	::=**if**<条件>**then** <语句>
<过程调用语句>	::=**call**<id>
<当型循环语句>	::=**while** <条件>**do** <语句>
<读语句>	::=**read** '('<id>{,<id>} ')'
<写语句>	::=**write** '('<表达式>{,<表达式>}')'

其中用到的 EBNF 元符号含义如下：

- < >：用尖括号括起来的中文字表示语法构造成分，或称语法单元；而用尖括号括起来的英文字表示一类词法单元。
- ::=：表示左部的语法单位由右部定义，可读作“定义为”。
- |：表示“或”，即多选项。
- {}：用花括号括起来的成分可以重复 0 次到任意多次。
- []：用方括号括起来的成分为任选项，即出现一次或不出现。

由表 1.1 的 EBNF 描述可以观察到：PL/0 程序由分程序组成，分程序由说明部分和语句组成。说明部分包括常量说明、变量说明和过程说明，并分别由关键字 const、var、procedure 开头。过程说明中可以嵌套过程说明。PL/0 语法整体上与 Pascal 的语法类似。

PL/0 的语句有赋值语句、条件语句、循环语句、过程调用语句、读语句、写语句和复合语句。有两种控制转移语句：if 语句和 while 语句，if 语句无 else 部分。复合语句是由 begin 和 end 括起来的语句序列。I/O 语句由保留字 read 和 write 开始，read 语句一次可为多个变量读入数据，write 语句一次可输出多个算术表达式的值。

PL/0 的表达式有关系表达式和算术表达式两类：控制转移语句中的测试条件是关系表达式，其中使用了关系运算符；算术表达式使用整型运算符。关系表达式由两个算术表达式的比较组成，通过 6 个关系运算进行比较。算术表达式由整型常数、整型变量以及常见的算术运算符组成。算术表达式还可以出现在赋值、I/O 语句中。

PL/0 中的变量是简单整型变量，需在变量说明部分进行说明，但无须给出数据类型，因为只有整型变量。

最后，PL/0 的语句序列中由分号来分隔语句，最后一个语句之后可以没有分号。可以有空语句序列。

在表 1.1 中，仅出现在::=右边的符号均为词法单元，即单词或单词符号。PL/0 的单词可分为 5 个大类：保留字、运算符、标识符、无符号整数、界符。

保留字有 13 个：

const,var,procedur,begin,end,odd,if,then,call,while,do,read,write

运算符有 11 个，分别为：4 个整型算术运算符号+、-、* 和/，6 个比较运算符号<、<=、>、>=、# 和=，以及赋值运算符 :=。

界符包括(、)、,、;和.。

无符号整数<integer>是由一个或多个数字组成的序列。数字为 0,1,2,…,9。

标识符<id>是字母开头的字母数字序列。字母包括大小写英文字母：a,b,…,z,A,B,…,Z。

1.4.3 类 P-code 语言

类 P-code 语言是 PL/0 编译程序的目标语言，可以看作类 P-code 虚拟机的汇编语言。类 P-code 虚拟机是一种简单的纯栈式结构的机器，它有一个栈式存储器，有 4 个控制寄存器：指令寄存器 i、指令地址寄存器 p、栈顶寄存器 t 和基址寄存器 b。类 P-code 程序运行期间的数据存储和算术及逻辑运算都在栈顶进行。

类 P-code 虚拟机的指令格式形如

F L A

它由 3 个部分构成，其含义如下：

F： 指令的操作码。

L： 若起作用，则表示引用层与声明层之间的层次差；若不起作用，则置为 0。

A： 不同的指令含义不同。

类 P-code 虚拟机完整的指令集合见表 1.2。对过程调用相关指令更详细的解释参见 9.4.2 节。

表 1.2　类 P-code 虚拟机指令系统

指令分类	指令格式	指令功能
过程调用相关指令	INT 0 A	在栈顶开辟 A 个存储单元
	OPR 0 0	结束被调用过程，返回调用点并退栈
	CAL L A	调用地址为 A 的过程，调用过程与被调用过程的层差为 L
存取指令	INT 0 A	立即数 A 存入 t 所指单元，t 加 1
	LOD L A	将层差为 L、偏移量为 A 的存储单元的值取到栈顶，t 加 1
	STO L A	将栈顶的值存入层差为 L、偏移量为 A 的单元，t 减 1
一元运算和比较指令	OPR 0 1	求栈顶元素的相反数，结果值留在栈顶
	OPR 0 6	栈顶内容若为奇数则变为 1，若为偶数则变为 0
二元运算指令	OPR 0 2	次栈顶与栈顶的值相加，结果存入次栈顶，t 减 1
	OPR 0 3	次栈顶的值减去栈顶的值，结果存放次栈顶，t 减 1
	OPR 0 4	次栈顶的值乘以栈顶的值，结果存放次栈顶，t 减 1
	OPR 0 5	次栈顶的值乘以栈顶的值，结果存放次栈顶，t 减 1
二元比较指令	OPR 0 8	次栈顶与栈顶内容若相等，则将 0 存于次栈顶，t 减 1
	OPR 0 9	次栈顶与栈顶内容若不相等，则将 0 存于次栈顶，t 减 1
	OPR 0 10	次栈顶内容若小于栈顶，则将 0 存于次栈顶，t 减 1
	OPR 0 11	次栈顶内容若大小于等于栈顶，则将 0 存于次栈顶，t 减 1
	OPR 0 12	次栈顶内容若大于栈顶，则将 0 存于次栈顶，t 减 1
	OPR 0 13	次栈顶内容若小于等于栈顶，则将 0 存于次栈顶，t 减 1
转移指令	JMP 0 A	无条件转移至地址 A
	JPC 0 A	若栈顶为 0，则转移至地址 A，t 减 1
输入输出指令	OPR 0 14	栈顶的值输出至控制台屏幕，t 减 1
	OPR 0 15	控制台屏幕输出一个换行
	OPR 0 16	从控制台读入一行输入，置入栈顶，t 加 1

1.4.4　PL/0 编译程序

PL/0 编译程序采用单遍扫描方式的编译过程，由词法分析程序、语法语义分析程序以及代码生成程序 3 个独立的过程组成。PL/0 编译程序以语法语义分析程序为核心，当语法分析需要读单词时就调用词法分析程序，而当语法语义分析正确需生成相应语言成分的目标代码时，就调用代码生成程序。PL/0 编译程序的组织结构如图 1.20 所示。

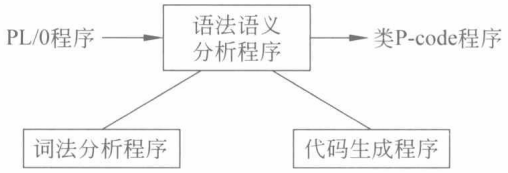


图 1.20　PL/0 编译程序组织结构

当源程序编译有错时，PL/0 编译程序用出错处理程序对词法和语法语义分析遇到的错误

给出在源程序中出错的位置和错误性质。当源程序编译正确时，编译程序正常结束，可输出相应的类 P-code 目标程序。

图 1.21 是 PL/0 编译程序对图 1.18 的源程序编译后的类 P-code 目标程序。

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口，为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点的下一条指令(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24)(条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

图 1.21　类 P-code 目标代码

1.4.5　PL/0 语言编译系统的驱动代码

如图 1.16 所示，PL/0 语言编译系统包括 PL/0 编译程序和类 P-code 解释程序，其驱动代码为附录 A 中代码的主函数 main()。main()函数的主体框架如下所示：

```
int main()
{
    ......
    /* 打开源程序文件 */
    if 打开源程序文件成功{
        ......                          /* 初始化输出文件信息 */
        init();                         /* 初始化各类名字和符号信息 */
        err=0;                          /* 初始化错误数 */
        ......                          /* 初始化其他信息 */
        if(-1 != getsym())              /* 成功读取第一个单词 */
        {
            ......
            if(-1==block(...))          /* 编译整个程序体对应的<分程序> */
            {
                ......                  /* 编译过程未成功结束，关闭所有已打开的文件，返回 */
            }
```

```
            ......                          /* 编译过程结束,关闭已打开的输出文件 */
            if (sym ! =period)              /* 当前符号不是程序结束符'.'*/
            {
                error(9);                   /* 提示 9 号出错信息:缺少程序结束符'.'*/
            }
            if (err==0)                     /* 未发现程序中的错误 */
            {
                ......
                interpret();                /* 调用解释程序,执行所产生的类 P-code 代码 */
                ......
            }
            else
            {
                printf("Errors in PL/0 program");
            }
        }
        ......
    }
    else
    {
        printf("Can't open file! \n");      /* 打开源程序文件不成功 */
    }
    ......                                  /* 关闭已打开的文件,成功返回 */
}
```

PL/0 编译程序以语法语义分析程序为核心,在分析过程中,它不断地调用词法分析程序 getsym()获取下一个单词,调用代码生成程序生成类 P-code 代码,在需要报错时调用 error()函数。语法语义分析程序执行一种自上而下的分析过程,采用递归下降分析/翻译子程序的构造方法,每个语法单元对应一个子程序。语法单元<分程序>对应的子程序为 block()。上面代码中的 block()是最顶层的<分程序>处理函数,用于处理整个程序体所对应的<分程序>。关于 block()函数以及其他递归子程序的构造,将在第 4 章进一步讨论。

getsym()实现基本的词法分析功能,用于获取下一个单词。第 3 章将会涉及 getsym()实现的一些细节。

interpret()用于解释执行类 P-code 程序,相当于实现了类 P-code 虚拟机。它采用了如下数据结构,从中可以了解类 P-code 虚拟机的基本结构:

```
运行栈          int s[stacksize]        /* stacksize:对应数据区存储单元数的上界 */
指令寄存器      struct instruction {
                    enum fct f;         /* 操作码 */
                    int l;              /* 引用层与声明层的层差 */
                    int a;              /* 因不同的 f 各异 */
                } i;
指令地址寄存器  int p;
```

```
基址寄存器          int b;
栈顶寄存器          int t;
虚拟机代码段        struct instruction code[cxmax];        /* cxmax: 代码区指令数的上界 */
```

interpret()顺序读取 code 中的类 P-code 指令逐条解释执行。不同指令的执行结果会以不同方式修改运行栈 s、指令地址寄存器 p、基址寄存器 b 以及栈顶寄存器 t 的状态。对照图 1.21 中执行类 P-code 目标代码的例子，有助于对 interpret()代码的理解。

对于一些与运行时存储组织相关的类 P-code 指令(表 1.2 中的存取指令和过程调用相关指令)，其执行语义略微复杂一些，将在第 9 章作进一步的解释。然而，对照图 1.21 中的例子，读者也不难理解它们的基本含义。

练　习

1. 解释下列术语：

编译程序，源程序，目标程序，编译程序的前端、后端和遍

2. 编译程序有哪些主要构成成分？各自的主要功能是什么？

3. 什么是解释程序？它与编译程序的主要不同是什么？

4. 对下列错误信息，请指出可能是编译的哪个阶段(词法分析、语法分析、语义分析、代码生成)报告的。

(1) else 没有匹配的 if。

(2) 数组下标越界。

(3) 使用的函数没有定义。

(4) 在数中出现非数字字符。

5. 通过 1.4 节的介绍以及对附录 A 中源码的初步阅读，要求读者：

(1) 熟悉 PL/0 编译程序的源语言和目标语言；

(2) 了解 PL/0 编译程序的基本结构；

(3) 了解 PL/0 语言编译系统驱动程序的基本结构。

第 2 章　文法和语言

一个程序设计语言是一个记号系统，如同自然语言一样，它的完整定义应包括语法和语义两个方面。所谓一个语言的语法是指一组规则，用它可以形成和产生一个合适的程序。目前广泛使用的手段是上下文无关文法，即用上下文无关文法作为程序设计语言语法的描述工具。语法只是定义什么样的符号序列是合法的，与这些符号的含义毫无关系，比如对于一个 Pascal 程序来说，一个上下文无关文法可以定义符号串 A：=B+C 是一个合乎语法的赋值语句，而 A：=B+就不是。但是，如果 B 是实型的，而 C 是布尔型的，或者 B、C 中任何一个变量没有事先说明，则 A：=B+C 仍不是正确的程序，也就是说程序结构上的这种特点——类型匹配、变量作用域等是无法用上下文无关手段检查的，这些工作属于语义分析工作。程序设计语言的语义常常分为两类：静态语义和动态语义。静态语义是一系列限定规则，并确定哪些合乎语法的程序是合适的；动态语义也称作运行语义或执行语义，表明程序要做些什么，要计算什么。

阐明语法的一个工具是文法，这是形式语言理论的基本概念之一。本章将介绍文法和语言的概念，重点讨论上下文无关文法及其句型分析中的有关问题。

阐明语义要比阐明语法困难得多，尽管形式语义学的研究已取得重大进展，但是仍没有哪一种公认的形式系统可用来自动构造出正确的编译系统。本书不对形式语义学进行介绍。

2.1　文法的直观概念

在给出文法和语言的形式定义之前，先直观地认识一下文法的概念。

当我们表述一种语言时，无非是说明这种语言的句子，如果语言只含有有穷多个句子，则只需列出句子的有穷集就行了，但对于含有无穷多个句子的语言来讲，存在着如何给出它的有穷表示的问题。

以自然语言为例，人们无法列出全部句子，但是人们可以给出一些规则，用这些规则来说明（或者定义）句子的组成结构，如“我是大学生”是汉语的一个句子。汉语句子可以由主语后随谓语而成，构成谓语的是动词和直接宾语，采用第 1 章使用过的 EBNF 来表示这种句子的构成规则：

〈句子〉::=〈主语〉〈谓语〉
〈主语〉::=〈代词〉|〈名词〉
〈代词〉::=我|你|他
〈名词〉::=王明|大学生|工人|英语
〈谓语〉::=〈动词〉〈直接宾语〉
〈动词〉::=是|学习
〈直接宾语〉::=〈代词〉|〈名词〉

“我是大学生”的构成符合上述规则，而“我大学生是”不符合上述规则，即它不是句子。这些规则成为判别句子结构合法与否的依据，换句话说，将这些规则看成是一种元语言，用它描述汉语。这里仅仅涉及汉语句子的结构描述。这样的语言描述称为文法。

一旦有了一组规则以后，可以按照如下方式用它们去推导或产生句子。开始去找::=左端的带有〈句子〉的规则并把它表示成::=右端的符号串，这个动作表示成：〈句子〉⇒〈主语〉〈谓语〉，然后在得到的串〈主语〉〈谓语〉中，选取〈主语〉或〈谓语〉，再用相应的规则::=右端代替之。例如，选取了〈主语〉，并采用规则〈主语〉::=〈代词〉，那么得到：〈主语〉〈谓语〉⇒〈代词〉〈谓语〉，重复做下去，得到句子“我是大学生”的全部动作过程如下：

〈句子〉⇒〈主语〉〈谓语〉
⇒〈代词〉〈谓语〉
⇒我〈谓语〉
⇒我〈动词〉〈直接宾语〉
⇒我是〈直接宾语〉
⇒我是〈名词〉
⇒我是大学生

符号⇒的含义是，使用一条规则，代替其左端的某个符号，产生其右端的符号串。

显然，按照上述办法，不仅生成“我是大学生”这样的句子，还可以生成“王明是大学生”，“王明学习英语”，“我学习英语”，“他学习英语”，“你是工人”，“你学习王明”等许多其他句子。事实上，使用文法作为工具，不仅是为了严格地定义句子的结构，也是为了用适当条数的规则把语言的全部句子描述出来，可以说文法是以有穷的集合刻画无穷的集合的一个工具。

2.2 符号和符号串

正如英语是由句子组成的集合，而句子又是由单词和标点符号组成的序列那样，程序设计语言 Pascal 或 C 语言是由一切 Pascal 程序或 C 程序所组成的集合，而程序是由类似 if、begin、end 的符号以及字母和数字这样一些基本符号所组成，从字面上看，每个程序都是一个“基本符号”串，假设有一个基本符号集，那么 Pascal 或 C 语言可看成是在这个基本符号集上定义的、按一定规则构成的一切基本符号串组成的集合。为了给出语言的形式定义，首先讨论符号和符号串的有关概念。

1. 字母表

字母表是元素的非空有穷集合，字母表中的元素称为符号，因此字母表也称为符号集。不同的语言可以有不同的字母表，例如汉语的字母表中包括汉字、数字及标点符号等。C 语言的字母表由字母、数字、若干专用符号及 char、struct、if、do 之类的保留字组成。

2. 符号串

由字母表中的符号组成的任何有穷序列称为符号串，例如 001110 是字母表 $\Sigma=\{0,1\}$ 上的符号串，又如字母表 $A=\{a,b,c\}$ 上的一些符号串有 a，b，c，ab，aaca。在符号串中，符号的顺序是很重要的，例如，符号串 ab 就不同于 ba，abca 和 aabc 也不同。可以使用字母表示符号串，如 x=STR 表示“x 是由符号 S、T 和 R，并按此顺序组成的符号串”。

如果某符号串 x 中有 m 个符号，则称其长度为 m，表示为 $|x|=m$，如 001110 的长度

是 6。

允许空符号串，即不包含任何符号的符号串，用 ε 表示，其长度为 0，即 $|\varepsilon|=0$。

下面介绍有关符号串的一些运算。

1）符号串的头尾，固有头和固有尾

如果 $z=xy$ 是一符号串，那么 x 是 z 的头，y 是 z 的尾，如果 x 是非空的，那么 y 是固有尾；同样，如果 y 非空，那么 x 是固有头。设 $z=$abc，那么 z 的头是 ε，a，ab 和 abc；除 abc 外，其他都是固有头；z 的尾是 ε，c，bc 和 abc；z 的固有尾是 ε，c 和 bc。

当我们对符号 $z=xy$ 的头感兴趣而对其余部分不感兴趣时，可以采用省略写法：$z=x\cdots$；如果只是为了强调 x 在符号串 z 中的某处出现，则可表示为：$z=\cdots x\cdots$；符号 t 是符号串 z 的第一个符号，则表示为 $z=t\cdots$。

2）符号串的连接

设 x 和 y 是符号串，它们的连接 xy 是把 y 的符号写在 x 的符号之后得到的符号串。例如，设 $x=$ST，$y=$abu，则它们的连接 $xy=$STabu，看出 $|x|=2$，$|y|=3$，$|xy|=5$。由于 ε 的含义，显然有 $\varepsilon x=x\varepsilon=x$。

3）符号串的方幂

设 x 是符号串，把 x 自身连接 n 次得到符号串 z，即 $z=xx\cdots xx$，称为符号串 x 的方幂，写作 $z=x^n$，即把符号串 x 相继地重复写 n 次。$x^0=\varepsilon$，$x^1=x$，$x^2=xx$，$x^3=xxx$ 分别对应于 $n=0,1,2$ 和 3，例如，设 $x=$AB，则 $x^0=\varepsilon$，$x^1=$AB，$x^2=$ABAB，$x^3=$ABABAB。对于 $n>0$，有 $x^n=xx^{n-1}=x^{n-1}x$。

4）符号串集合

若集合 A 中的一切元素都是某字母表上的符号串，则称 A 为该字母表上的符号串集合。两个符号串集合 A 和 B 的乘积定义如下：$AB=\{xy\mid x\in A$ 且 $y\in B\}$，即 AB 是满足 x 属于 A，y 属于 B 的所有符号串 xy 所组成的集合。例如，若 $A=\{$a,b$\}$，$B=\{$c,d$\}$，则集合 $AB=\{$ac,ad,bc,bd$\}$。因为对任意符号串 x 有 $\varepsilon x=x\varepsilon=x$，所以有 $\{\varepsilon\}A=A\{\varepsilon\}=A$。

指定字母表 Σ 之后，可用 Σ^* 表示 Σ 上的所有有穷长的串的集合。例如，如 $\Sigma=\{0,1\}$，则 $\Sigma^*=\{\varepsilon,0,1,00,01,10,11,000,001,010,\cdots\}$，也可表示为字母表的方幂形式：

$$\Sigma^*=\Sigma^0\cup\Sigma^1\cup\Sigma^2\cdots\cup\Sigma^n\cdots$$

Σ^* 称为集合 Σ 的**闭包**。而 $\Sigma^+=\Sigma^1\cup\Sigma^2\cdots\Sigma^n\cdots$ 称为 Σ 的**正闭包**。显然有

$$\Sigma^*=\Sigma^0\cup\Sigma^+$$

$$\Sigma^+=\Sigma\Sigma^*=\Sigma^*\Sigma$$

Σ^* 具有可数的无穷数量的元素。使用一般集合论的表示符号：若 x 是 Σ^* 中的元素，则表示为 $x\in\Sigma^*$，否则 $x\notin\Sigma^*$。对于所有的 Σ，有 $\varepsilon\in\Sigma^*$。

2.3 文法和语言的形式定义

这里使用 2.2 节所引用的术语，将 2.1 节中描述汉语句子的规则加以形式化，然后给出文法和语言的形式定义。

规则，也称**重写规则**、**产生式**或**生成式**，是形如 $\alpha\to\beta$ 或 $\alpha::=\beta$ 的 (α,β) 有序对，其中 α 称为规则的左部，β 称作规则的右部。这里使用的符号 $\to$（$::=$）读作“定义为”。例如 $A\to$a 读

作“A 定义为 a”。也把它说成是一条关于 A 的规则(产生式)。

定义 2.1 文法 G 定义为四元组(V_N, V_T, P, S)。

其中 V_N 为非终结符(或语法实体,或变量)集;V_T 为终结符集;P 为规则$(\alpha \to \beta)$的集合,$\alpha \in (V_N \cup V_T)^*$ 且至少包含一个非终结符,$\beta \in (V_N \cup V_T)^*$;$V_N$,$V_T$ 和 P 是非空有穷集。S 称作识别符或开始符,它是一个非终结符,至少要在一条规则中作为左部出现。

V_N 和 V_T 不含公共的元素,即 $V_N \cap V_T = \varnothing$。

通常用 V 表示 $V_N \cup V_T$,V 称为文法 G 的字母表或字汇表。

例 2.1 有文法 $G=(V_N, V_T, P, S)$,其中,$V_N=\{S\}$,$V_T=\{0,1\}$,$P=\{S \to 0S1, S \to 01\}$。这里,非终结符集中只含一个元素 S;终结符集由两个元素 0 和 1 组成;有两条产生式;开始符是 S。

例 2.2 有文法 $G=(V_N, V_T, P, S)$。

其中,$V_N=\{$标识符,字母,数字$\}$,$V_T=\{$a,b,c,…,x,y,z,0,1,…,9$\}$

$P=\{$ 〈标识符〉→〈字母〉

〈标识符〉→〈标识符〉〈字母〉

〈标识符〉→〈标识符〉〈数字〉

〈字母〉→a

〈字母〉→b

⋮

〈字母〉→z

〈数字〉→0

〈数字〉→1

⋮

〈数字〉→9$\}$

$S=$〈标识符〉

这里,使用尖括号括起非终结符。

很多时候,不用将文法 G 的四元组显式地表示出来,而只将产生式写出。一般约定,第一条产生式的左部是识别符;用尖括号括起来的是非终结符,不用尖括号括起来的是终结符,或者用大写字母表示非终结符,小写字母表示终结符。另外,也有一种习惯写法,将 G 写成 $G[S]$,其中 S 是识别符,例 2.1 还可以写成

G: $S \to 0S1$

$S \to 01$

或

$G[S]$: $S \to 0S1$

$S \to 01$

为定义文法所产生的语言,还需要引入**推导**的概念,即定义 V^* 中的符号之间的关系:直接推导$\Rightarrow$、长度为 $n(n \geqslant 1)$的推导$\overset{+}{\Rightarrow}$和长度为 $n(n \geqslant 0)$的推导$\overset{*}{\Rightarrow}$。

定义 2.2 设 $\alpha \to \beta$ 是文法 $G=(V_N, V_T, P, S)$的规则(或说是 P 中的一个产生式),γ 和 δ 是 V^* 中的任意符号,若有符号串 v、w 满足

$$v = \gamma\alpha\delta, \quad w = \gamma\beta\delta$$

则说 v(应用规则 $\alpha\to\beta$)直接产生 w,或说 w 是 v 的**直接推导**,或说 w 直接**归约**到 v,记作 $v\Rightarrow w$。

例如,对于例 2.1 的文法 G,可以给出直接推导的一些例子。

(1) $v=0S1$,$w=0011$,直接推导:$0S1\Rightarrow 0011$,使用的规则:$S\to 01$,这里 $\gamma=0$,$\delta=1$。

(2) $v=S$,$w=0S1$,直接推导:$S\Rightarrow 0S1$,使用的规则:$S\to 0S1$,这里 $\gamma=\varepsilon$,$\delta=\varepsilon$。

(3) $v=0S1$,$w=00S11$,直接推导:$0S1\Rightarrow 00S11$,使用的规则:$S\to 0S1$,这里 $\gamma=0$,$\delta=1$。

对于例 2.2 的文法 G,直接推导的例子如下:

(1) $v=$〈标识符〉,$w=$〈标识符〉〈字母〉,直接推导:〈标识符〉⇒〈标识符〉〈字母〉,使用的规则:〈标识符〉→〈标识符〉〈字母〉,这里 $\gamma=\delta=\varepsilon$。

(2) $v=$〈标识符〉〈字母〉〈数字〉,$w=$〈字母〉〈字母〉〈数字〉,直接推导:〈标识符〉〈字母〉〈数字〉⇒〈字母〉〈字母〉〈数字〉,使用的规则:〈标识符〉→〈字母〉,这里 $\gamma=\varepsilon$,$\delta=$〈字母〉〈数字〉。

(3) $v=$abc〈数字〉,$w=$abc5,直接推导:abc〈数字〉⇒abc5,使用的规则:〈数字〉→5,这里 $\gamma=$abc,$\delta=\varepsilon$。

定义 2.3 如果存在直接推导的序列:

$$v=w_0\Rightarrow w_1\Rightarrow w_2\Rightarrow\cdots\Rightarrow w_n=w(n>0)$$

则称 v 推导出(产生)w(推导长度为 n),或称 w 归约到 v,记作 $v\overset{+}{\Rightarrow}w$。

定义 2.4 若有 $v\overset{+}{\Rightarrow}w$,或 $v=w$,则记作 $v\overset{*}{\Rightarrow}w$。

例如,对例 2.1 的文法,存在直接推导序列 $v=0S1\Rightarrow 00S11\Rightarrow 000S111\Rightarrow 00001111=w$,即 $0S1\overset{+}{\Rightarrow}00001111$,也可记作 $0S1\overset{*}{\Rightarrow}00001111$。

对例 2.2 的文法,存在直接推导序列 $v=$〈标识符〉⇒〈标识符〉〈数字〉⇒〈字母〉〈数字〉⇒x〈数字〉⇒$x1=w$,即〈标识符〉$\overset{+}{\Rightarrow}x1$,也可记作〈标识符〉$\overset{*}{\Rightarrow}x1$。

定义 2.5 设 $G[S]$是一个文法,如果符号串 x 是从识别符号推导出来的,即有 $S\overset{*}{\Rightarrow}x$,则称 x 是文法 $G[S]$的**句型**。若 x 仅由终结符号组成,即 $S\overset{*}{\Rightarrow}x$,$x\in V_T^*$,则称 x 为 $G[S]$的**句子**。

例如,S、$0S1$、000111 都是例 2.1 的文法 G 的句型,其中 000111 是 G 的句子。〈标识符〉〈字母〉、〈字母〉〈数字〉、$a1$ 等都是例 2.2 文法 G 的句型,其中 $a1$ 是 G 的句子。

定义 2.6 文法 G 所产生的语言定义为集合$\{x\mid S\overset{*}{\Rightarrow}x$,其中 S 为文法识别符号,且 $x\in V_T^*\}$。可用 $L(G)$表示该集合。

从定义 2.6 看出两点:第一,符号串 x 可从识别符号推出,即 x 是句型。第二,x 仅由终结符号组成,即 x 是文法 G 的句子。也就是说,**文法描述的语言是该文法一切句子的集合**。考虑例 2.1 的文法 G,有两条产生式(规则):(1)$S\to 0S1$ 和(2)$S\to 01$,通过对第一个产生式使用 $n-1$ 次,然后使用第二个产生式一次,得到

$$S\Rightarrow 0S1\Rightarrow 00S11\Rightarrow\cdots\Rightarrow 0^{n-1}S\,1^{n-1}\Rightarrow 0^n1^n$$

是不是 $L(G)$中的元素仅是这样的串(0^n1^n)?是的,这可以由下面的讨论证得。在使用了第二个产生式后,发现句型中 S 的个数减少了一个。每次使用第一个产生式之后,S 的左端多一个 0,右端多一个 1,S 的个数不变。因此,使用了 $S\to 01$ 之后,就再也没有 S 留在结

果串中了。由于两个产生式都是以 S 为左端，所以为了生成句子，仅能按下列次序使用产生式：使用第一个产生式若干次，然后使用第二个产生式。因此 $L(G)=\{0^n1^n \mid n\geqslant 1\}$。

例 2.2 的文法 G 的句子是字母字符打头的、字母字符和数字字符构成的串。这就是程序设计语言中用于表示名字的标识符。

例 2.3 设 $G=(V_N,V_T,P,S)$，$V_N=\{S,B,E\}$，$V_T=\{a,b,e\}$，P 由下列产生式组成：

(1) $S\rightarrow aSBE$

(2) $S\rightarrow aBE$

(3) $EB\rightarrow BE$

(4) $aB\rightarrow ab$

(5) $bB\rightarrow bb$

(6) $bE\rightarrow be$

(7) $eE\rightarrow ee$

例 2.1 和例 2.2 都是较简单的文法的例子，比较容易确定哪些符号串可以推导出来，哪些不能。一般说来，确定文法将产生什么集合可能是很困难的，不过描述程序设计语言的文法还是较容易确定其句子的形式的。现在分析例 2.3 产生的句子。

首先证明，对每一个 $n\geqslant 1$，$L(G)$ 含有句子 $a^nb^ne^n$，因为能使用产生式(1) $n-1$ 次，得到推导序列：$S\overset{*}{\Rightarrow}a^{n-1}S(BE)^{n-1}$，然后使用产生式(2)一次，得到：$S\overset{*}{\Rightarrow}a^n(BE)^n$。然后从 $a^n(BE)^n$ 继续推导，总是对 EB 使用产生式(3)的右部进行替换，而最终在得到的串中，所有的 B 都先于所有的 E。例如，若 $n=3$，$aaaBEBEBE\Rightarrow aaaBBEEBE\Rightarrow aaaBBEBEE\Rightarrow aaaBBBEEE$。即有：$S\overset{*}{\Rightarrow}a^nB^nE^n$。

接着，使用产生式(4)一次，得到 $S\overset{*}{\Rightarrow}a^nbB^{n-1}E^n$，然后使用产生式(5) $n-1$ 次得到：$S\overset{*}{\Rightarrow}a^nb^nE^n$，最后使用产生式(6)一次，使用产生式(7) $n-1$ 次，得到：$S\overset{*}{\Rightarrow}a^nb^ne^n$。

也能证明，对于 $n\geqslant 1$，串 $a^nb^ne^n$ 是唯一形式的终结符号串。在从 S 开始的任何推导中，在未使用产生式(2)之前，是不能使用产生式(4)、(5)、(6)或(7)的，因为从(4)到(7)的每一个产生式的左端都要求 B 或者 E 的左边直接有一个终结符。使用产生式(2)之前，所有的句型都是由多个 a 跟以一个 S，然后跟以与 a 同样多个 B 和 E 组成。使用产生式(2)之后，对于 $n\geqslant 1$，句型就由 n 个 a，后面跟某种次序的 n 个 B 和 n 个 E 组成，因为没有 S 出现在句型中了，所以以后的推导中不可再使用产生式(1)和(2)了。从 S 推导出的串的特点是，前面部分全部是终结符而后面部分全部是非终结符。在使用产生式(3)到(7)中任何一个之后，推导出的串仍具有这种特点。而产生式(4)到(7)只能在终结符和非终结符的边界上使用，它们的作用是把一个 B 变为 b 或把一个 E 变为 e。产生式(3)的使用可把 B 移到左边，把 E 移到右边。为得到终结符号串，在任何 E 被转换成 e 之前，所有的 B 都必须在终结符和非终结符接口处被转换为 b。否则假设在所有的 B 转换到 b 之前，把一个 E 转换到 e。这时推导出的串可表示为 $a^nb^ie\alpha$，其中 $i<n$，α 是由 B 和 E 组成的串，接着进行推导时，只有产生式(3)和(7)可以使用。(3)用在非终结符中，(3)只能重新安排 α 中的 B 和 E，但不能删除任何 B。产生式(7)用于终结符和非终结符间的交换处，能把 E 转换为 e，但最后一个 B 是最左非终结符。没有产生式能改变这个 B。即永远无法推导出终结符号串，因此所做的假设是不成立的，结论只能是：在任何 E 被转换为 e 之前，所有的 B 都必须在终结符和非终结

符之间的接口处转换成 b，后面的推导都是在形为 $a^n \underbrace{B\cdots BE}_{n}\underbrace{\cdots E}_{n}$ 的串上继续，即 $a^nb^ne^n$ 是仅可能推导出的串。

因此 $L(G)=\{a^nb^ne^n \mid n\geqslant 1\}$。

定义 2.7 若 $L(G1)=L(G2)$，则称**文法 G1 和 G2 是等价的**。

例如文法 $G[A]$：

$A\rightarrow 0R$

$A\rightarrow 01$

$R\rightarrow A1$

和例 2.1 的文法等价。

2.4 文法的类型

自从乔姆斯基(Chomsky)于 1956 年建立形式语言的描述以来，形式语言的理论发展很快。这种理论对计算机科学有着深刻的影响，特别是对程序设计语言的设计、编译方法和计算复杂性等方面更有重大的作用。

乔姆斯基把文法分成 4 种类型，即 0 型、1 型、2 型和 3 型。这几类文法的差别在于对产生式施加不同的限制。

设 $G=(V_N,V_T,P,S)$，如果它的每个产生式 $\alpha\rightarrow\beta$ 是这样一种结构：$\alpha\in(V_N\cup V_T)^*$ 且至少含有一个非终结符，而 $\beta\in(V_N\cup V_T)^*$，则 G 是一个 **0 型文法**。

0 型文法也称短语文法。一个非常重要的理论结果是，0 型文法的能力相当于图灵机(Turing machine)。或者说，任何 0 型语言都是递归可枚举的；反之，递归可枚举集必定是一个 0 型语言。

对 0 型文法产生式的形式作某些限制，以给出 1 型、2 型和 3 型文法的定义。

设 $G=(V_N,V_T,P,S)$ 为一个文法，若 P 中的每一个产生式 $\alpha\rightarrow\beta$ 均满足 $|\beta|\geqslant|\alpha|$，仅仅 $S\rightarrow\varepsilon$ 除外，则文法 G 是 **1 型或上下文有关的**(context-sensitive)。

例 2.3 的文法是上下文有关的。同样，例 2.1 和例 2.2 的文法也是上下文有关的。

在有些定义中，将上下文有关文法的产生式的形式描述为 $\alpha_1A\alpha_2\rightarrow\alpha_1\beta\alpha_2$，其中 α_1、α_2 和 β 都在 $(V_N\cup V_T)^*$ 中(即在 V^* 中)，$\beta\neq\varepsilon$，A 在 V_N 中。这种定义与前面的定义等价。但它更能体现“上下文有关”这一术语，因为只有 A 出现在 α_1 和 α_2 的上下文中，才允许用 β 取代 A。

设 $G=(V_N,V_T,P,S)$，若 P 中的每一个产生式 $\alpha\rightarrow\beta$ 满足：α 是一个非终结符，$\beta\in(V_N\cup V_T)^*$，则此文法称为 **2 型的或上下文无关的**(context-free)。有时将 2 型文法的产生式表示为 $A\rightarrow\beta$ 的形式，其中 $A\in V_N$，也就是说用 β 取代非终结符 A 时，与 A 所在的上下文无关，因此取名为上下文无关。

例 2.1 和例 2.2 中的文法都是上下文无关的。下面再给出一个例子(例 2.4)，例子中的文法 G 是上下文无关文法，G 的语言是由相同个数的 a 和 b 所组成的 $\{a,b\}^*$ 上的串。

例 2.4 $G=(\{S,A,B\},\{a,b\},P,S)$，其中 P 由下列产生式组成：

$$S\rightarrow aB$$

$$A\rightarrow bAA$$

$$S\rightarrow bA$$

$$B \to b$$
$$A \to a$$
$$B \to bS$$
$$A \to aS$$
$$B \to aBB$$

有时，为书写简洁，常把相同左部的产生式，形如

$$A \to \alpha_1$$
$$A \to \alpha_2$$
$$\vdots$$
$$A \to \alpha_n$$

缩写为

$$A \to \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$$

这里的元符号|读做“或”。

例 2.4 的 P 可写为

$$S \to aB \mid bA$$
$$A \to a \mid aS \mid bAA$$
$$B \to b \mid bS \mid aBB$$

设 $G=(V_N, V_T, P, S)$，若 P 中的每一个产生式的形式都是 $A \to aB$ 或 $A \to a$，其中 A 和 B 都是非终结符，$a \in V_T^*$，则 G 是**3 型文法**或**正规文法**。

例 2.5 文法 $G=(\{S,A,B\},\{0,1\},P,S)$，其中 P 由下列产生式组成：

$$S \to 0A$$
$$S \to 1B$$
$$S \to 0$$
$$A \to 0A$$
$$A \to 0S$$
$$A \to 1B$$
$$B \to 1B$$
$$B \to 1$$
$$B \to 0$$

显然 G 是正规文法。

4 种文法类型的定义是逐渐增加限制的，因此每一种正规文法都是上下文无关的，每一种上下文无关文法都是上下文有关的，而每一种上下文有关文法都是 0 型文法。称 0 型文法产生的语言为**0 型语言**。上下文有关文法、上下文无关文法和正规文法产生的语言分别称为**上下文有关语言**、**上下文无关语言**和**正规语言**。

2.5 上下文无关文法及其语法树

上下文无关文法有足够的能力描述现今程序设计语言的语法结构，如描述算术表达式、描述各种语句等。

例 2.6 文法 $G=(\{E\},\{+,*,i,(,)\},P,E)$，其中 P 为：

$E\to i$

$E\to E+E$

$E\to E*E$

$E\to (E)$

这里的非终结符 E 表示一类算术表达式，i 表示程序设计语言中的变量。该文法定义了(描述了)由变量、+、*、(和)组成的算术表达式的语法结构，即：

(1) 变量是算术表达式；

(2) 若 E_1 和 E_2 是算术表达式，则 E_1+E_2、E_1*E_2 和 (E_1) 也是算术表达式。

描述一种简单赋值语句的产生式为

〈赋值语句〉→i==E

描述条件语句的文法片段为

〈条件语句〉→if〈条件〉then〈语句〉|
if〈条件〉then〈语句〉else〈语句〉

因此我们关心上下文无关文法的句子分析和分析方法的研究。本书的后面章节中，对“文法”一词若无特别说明，则均指上下文无关文法。

在前面提到了句型、推导等概念，现在介绍一种描述上下文无关文法的句型推导的直观工具，即**语法树**，也称**推导树**。

给定文法 $G=(V_N,V_T,P,S)$，对于 G 的任何句型都能构造与之关联的语法树(推导树)。这棵树满足下列 4 个条件：

(1) 每个结点都有一个标记，此标记是 V 的一个符号。

(2) 根的标记是 S。

(3) 若一个结点 n 至少有一个它自己除外的子孙，并且有标记 A，则 A 肯定在 V_N 中。

(4) 如果结点 n 的直接子孙从左到右的次序是结点 $n_1,n_2,\cdots,n_k$，其标记分别为 $A_1,A_2,\cdots,A_k$，那么 $A\to A_1A_2\cdots A_k$ 一定是 P 中的一个产生式。

下面用一个例子来说明语法树(推导树)的构造。

例 2.7 $G=(\{S,A\},\{a,b\},P,S)$，其中 P 为

(1) $S\to aAS$

(2) $A\to SbA$

(3) $A\to SS$

(4) $S\to a$

(5) $A\to ba$

图 2.1 是 G 的一棵推导树。

标记 S 的顶端结点是树根，它的直接子孙为 a、A 和 S 三个结点，a 在 A 和 S 的左边，A 在 S 的左边，$S\to aAS$ 是一个产生式。同样，A 结点至少有一个除它自己以外的子孙(A 的直接子孙为 S，b 和 A)，A 肯定是非终结符。

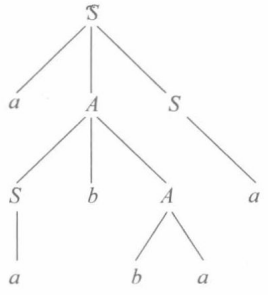


图 2.1 推导树

图 2.1 的推导树是例 2.7 的文法 G 的句型 $aabbaa$ 的推导过程

非常直观自然的描述，从左至右读出图 2.1 的推导树的叶子标记，得到的就是句型 $aabbaa$。常常把 $aabbaa$ 叫做推导树的结果，也把推导树叫做句型 $aabbaa$ 的语法树，而且在以后的章节中，只使用语法树这个术语。

语法树表示了在推导过程中施用了哪个产生式和施用在哪个非终结符上，它并没有表明施用产生式的顺序。比如例 2.7 文法 G 的句型 $aabbaa$ 的推导过程可以列举以下 3 个：

推导过程 1：$S \Rightarrow aAS \Rightarrow aAa \Rightarrow aSbAa \Rightarrow aSbbaa \Rightarrow aabbaa$

推导过程 2：$S \Rightarrow aAS \Rightarrow aSbAS \Rightarrow aabAS \Rightarrow aabbaS \Rightarrow aabbaa$

推导过程 3：$S \Rightarrow aAS \Rightarrow aSbAS \Rightarrow aSbAa \Rightarrow aabAa \Rightarrow aabbaa$

其中第 1 个推导过程的特点是在推导中总是对当前串中的最右非终结符施用产生式进行替换，施用产生式的顺序为(1)、(4)、(2)、(5)和(4)。第 2 个推导过程恰恰相反，在推导中总是对当前串中的最左非终结符施用产生式进行替换，施用产生式的顺序为(1)、(2)、(4)、(5)和(4)。除上述 3 个推导过程外，显然还可以给出一些不同的推导过程，这里不再列举。

如果在推导的任何一步 $\alpha \Rightarrow \beta$，其中 α、β 是句型，都是对 α 中的最左(最右)非终结符进行替换，则称这种推导为**最左(最右)推导**。上述第 1 个推导是最右推导，第 2 个是最左推导。在形式语言中，最右推导常被称为**规范推导**。由规范推导所得的句型称为右句型或**规范句型**。

不管是上述第 1 个还是第 2 个、第 3 个推导过程，它们相联的语法树都是图 2.1 的语法树。这就是说，一棵语法树表示了一个句型的种种可能的(但未必是所有的)不同推导过程，包括最左(最右)推导。但是，一个句型是否只对应唯一的一棵语法树呢？一个句型是否只有唯一的一个最左(最右)推导呢？不是的。

例如，对于例 2.6 的文法 G，句型 $i*i+i$ 就有两个不同的最左推导 1 和 2，它们所对应的语法树分别如图 2.2 和图 2.3 所示。

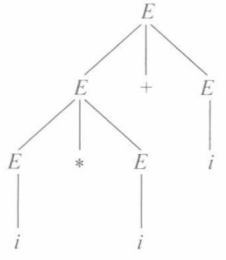


图 2.2　推导 1 的语法树

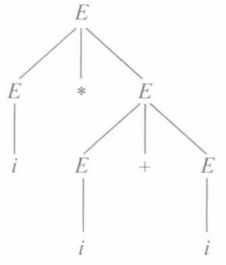


图 2.3　推导 2 的语法树

推导 1：$E \Rightarrow E+E \Rightarrow E*E+E \Rightarrow i*E+E \Rightarrow i*i+E \Rightarrow i*i+i$

推导 2：$E \Rightarrow E*E \Rightarrow i*E \Rightarrow i*E+E \Rightarrow i*i+E \Rightarrow i*i+i$

$i*i+i$ 是例 2.6 文法 G 的一个句子，这个句子可以用完全不同的两种办法生成，在生成过程的第 1 步，一种办法使用产生式 $E \rightarrow E+E$ 进行推导，另一种办法是使用产生式 $E \rightarrow E*E$。因而 $i*i+i$ 对应了两棵不同的语法树(见图 2.2 和图 2.3)。

如果一个文法存在某个句子对应两棵不同的语法树，则说这个文法是**二义**的。或者说，若一个文法中存在某个句子，它有两个不同的最左(最右)推导，则这个文法是二义的。例 2.6 的文法 G 是二义的。

注意，文法的二义性和语言的二义性是两个不同的概念。因为可能有两个不同的文法

G 和 G'，其中可能 G 是二义的而 G' 不是二义的，但是却有 $L(G)=L(G')$，也就是说，这两个文法所产生的语言是相同的。如果产生上下文无关语言的每一个文法都是二义的，则说此语言是先天二义的。对于一个程序设计语言来说，常常希望它的文法是无二义的，因为希望对它的每个语句的分析是唯一的。

形式语言理论已经证明，要判定任给的一个上下文无关文法是否为二义的，或它是否产生一个先天二义的上下文无关语言，这两个问题是递归不可解的[20]。即，不存在一个算法，它能在有限步骤内确切判定任给的一个文法是否为二义的。我们所能做的事是为无二义性寻找一组充分条件（当然它们未必都是必要的）。例如，在例 2.6 的文法中，假若规定了运算符＋与＊的优先顺序和结合规则，即按惯例，让＊的优先性高于＋，且它们都服从左结合，那么就可以构造出一个无二义文法，如例 2.8 的文法。

例 2.8 定义表达式的无二义文法 $G[E]$：

$E\rightarrow T\mid E+T$

$T\rightarrow F\mid T*F$

$F\rightarrow(E)\mid i$

它和例 2.6 的文法产生的语言是相同的，即它们是等价的。

2.6 句型的分析

对于上下文无关文法，语法树是句型推导过程的几何表示。从 2.5 节所给的例子看出，语法树确实将所给句型的结构很直观地显示出来了。语法树是句型结构分析的极好工具。而这里所说的句型分析问题，是说如何知道所给定的符号串是文法的句型。句型的分析就是识别一个符号串是否为某文法的句型，是某个推导的构造过程。进一步说，当给定一个符号串时，试图按照某文法的规则为该符号串构造推导或语法树，以此识别出它是该文法的一个句型；当符号串全部由终结符号组成时，就是识别它是不是某文法的句子。因此也有人把语法树称为**语法分析树**或**分析树**。对于程序设计语言来说，要识别的是程序设计语言的程序，程序是定义程序设计语言的文法的句子。句型分析是一个识别输入符号串是否为语法上正确的程序的过程。在语言的编译实现中，把完成句型分析的程序称为**分析程序**或**识别程序**，分析算法又称识别算法。

本书介绍的分析算法都称为从左到右的分析算法，即总是从左到右地识别输入符号串，首先识别符号串中的最左符号，进而识别右边的一个符号。当然，也可以定义从右向左的分析算法，但从左到右的分析更为自然，因为程序是从左到右地书写与阅读的。

这种分析算法又可分成两大类，即自上而下的和自下而上的。所谓自上而下分析法，是从文法的开始符号出发，反复使用各种产生式，寻找“匹配”于输入符号串的推导。自下而上的方法则是从输入符号串开始，逐步进行“归约”，直至归约到文法的开始符号。从语法树建立的方式可以很好理解这两类方法的区别。自上而下方法是从文法符号开始，将它作为语法树的根，向下逐步建立语法树，使语法树的末端结点符号串正好是输入符号串；自下而上方法则是从输入符号串开始，以它作为语法树的末端结点符号串，自下而上地构造语法树。

2.6.1 自上而下的分析方法

下面以一个简单的例子说明自上而下分析方法的基本思想。

例 2.9 考虑文法 $G[S]$：

(1) $S \rightarrow cAd$

(2) $A \rightarrow ab$

(3) $A \rightarrow a$

识别输入串 $w=cabd$ 是否为该文法的句子。即从根符号 S 开始，如图 2.4(a)所示，试着为 $cabd$ 构造一棵语法树。在构造的第 1 步，唯一的一个产生式可施用，则构造了直接推导 $S \Rightarrow cAd$，从 S 向下画语法树，如图 2.4(b)所示。这棵树的最左叶子标记为 c，已和 w 的第 1 个符号匹配。考虑下一个叶子标记 A，可用 A 的第 1 个候选(产生式(2))去扩展 A，则会得到如图 2.4(c)所示的语法树，构造的直接推导为 $cAd \Rightarrow cabd$。这时输入符号串 w 的第 2 个符号 a 得到了匹配。第 3 个输入符号为 b，将它与下一个叶子标记 b 相比较，得以匹配，叶子 d 匹配了第 4 个输入符号，这时可以宣布识别过程胜利结束。所构造的推导过程为 $S \Rightarrow cAd \Rightarrow cabd$。

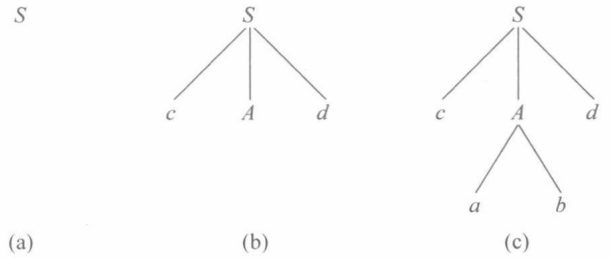


图 2.4 自上而下的分析步骤

2.6.2 自下而上的分析方法

仍使用例 2.9 中的文法来为输入符号串 $cabd$ 构造语法树，所采用的是自下而上的方法。

首先从输入符号串开始。扫描 $cabd$，从中寻找一个子串，该子串与某一产生式的右端相匹配。子串 a 和子串 ab 都是合格的，假若选用了 ab，用产生式(2)的左端 A 去替代它，即把 ab 归约到 A，得到了串 cAd。构造了一个直接推导 $cAd \Rightarrow cabd$，即从 $cabd$ 叶子开始向上构造语法树，如图 2.5(b)所示。接下去，在得到的串 cAd 中又找到了子串 cAd 与产生式(1)的右端相匹配，则用 S 替代 cAd，或称将 cAd 归约到 S，得到了又一直接推导 $S \Rightarrow cAd$，形成了图 2.5(c)所示的语法树，符号串 $cabd$ 的推导序列为 $S \Rightarrow cAd \Rightarrow cabd$。

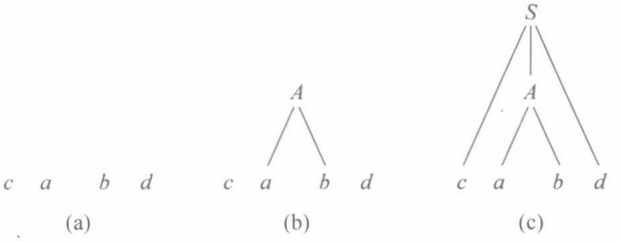


图 2.5 自下而上的分析步骤

2.6.3 句型分析的有关问题

在自上而下的分析方法中，对于例 2.9 的文法，构造了第一个直接推导 $S \Rightarrow cAd$ 后，接下来要扩展非终结符 A 了，在 A 的两个选择中采用了产生式(2)，很顺利地完成了识别过程。假若当时是另一种选择，采用产生式(3)的右部扩展 A，那将会出现什么情况呢？即构造的推导序列是 $S \Rightarrow cAd \Rightarrow cad$，语法树如图 2.6 所示。看到输入符号串 $w=cabd$ 的第 2 个符号与叶子结点 a 得以匹配，但第 3 个符号不能与下一叶子结点 d 匹配，这时如果宣告分析失败，则意味着识别程序不能为串 $cabd$ 构造语法树，即 $cabd$ 不是句子，这显然是错误的结论，导致失败的原因是在分析中对 A 的选择不是正确的。因此在自上而下分析方法中的主要问题是：假定要被代换的最左非终结符是 V，且有 n 条规则：$V \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$，那么如何确定用哪个右部去替代 V 呢？有一种办法是从各种可能的选择中随机挑选一种，并希望它是正确的。如果以后发现它是错误的，必须退回去，再试另外的选择，这种方式称为**回溯**。显然这样做代价极高，效率很低。在第 4 章将专门介绍自上而下分析方法中回溯问题的解决。

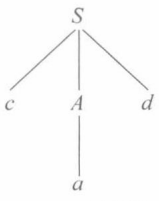


图 2.6　cad 的语法树

在自下而上的分析方法中，在分析程序工作的每一步，都是从当前串中选择一个子串，将它归约到某个非终结符，暂且把这个子串称为“可归约串”。问题是，每一步如何确定这个“可归约串”。在例 2.9 的文法对串 $cabd$ 的分析中，如果不是选择子串 ab 用产生式(2)，而是选择子串 a 用产生式(3)将 a 归约到 A，那么最终就达不到归约到 S 的结果，因而也无从知道 $cabd$ 是一个句子。为什么在 $cabd$ 中，ab 是“可归约串”，而 a 不是“可归约串”？如何知道这一点，这是自下而上分析的关键问题。因此需要精确定义“可归约串”。事实上，存在种种不同的方法刻画“可归约串”。对这个概念的不同定义形成了不同的自下而上分析方法。在一种称作“规范归约”的分析中，这种“可归约串”称作句柄。现在给出句柄的定义。

定义 2.8　令 G 是一个文法，S 是文法的开始符号，$\alpha\beta\delta$ 是文法 G 的一个句型。如果有 $S \overset{*}{\Rightarrow} \alpha A\delta$ 且 $A \overset{+}{\Rightarrow} \beta$，则称 β 是句型 $\alpha\beta\delta$ 相对于非终结符 A 的**短语**。特别地，如果有 $A \Rightarrow \beta$，则称 β 是句型 $\alpha\beta\delta$ 相对于规则 $A \rightarrow \beta$ 的**直接短语**(也称**简单短语**)。一个右句型的直接短语称为该句型的**句柄**。句柄的概念只适合于右句型。

如果所考虑的文法是无二义的，那么每个右句型有唯一的最右推导，因而其句柄是唯一的；对于二义文法，右句型就可能有多个句柄。对于无二义文法，一个右句型的唯一句柄是其所有直接短语中最左边的那一个，对于这种情形，该句型的最左直接短语即是它的句柄。

下面举一些例子来理解短语、直接短语和句柄的概念。

考虑例 2.8 中的无二义文法 $G[E]$的一个句型 $i*i+i$。为了叙述方便，将句型写作 $i_1*i_2+i_3$。因为有 $E \overset{*}{\Rightarrow} F*i_2+i_3$，且 $F \Rightarrow i_1$，则称 i_1 是句型 $i_1*i_2+i_3$ 的相对于非终结符 F 的短语，也是相对于规则 $F \rightarrow i$ 的直接短语。

又有 $E \overset{*}{\Rightarrow} i_1*F+i_3$，且 $F \Rightarrow i_2$，则 i_2 是句型 $i_1*i_2+i_3$ 的相对于 F 的短语，也是相对于规则 $F \rightarrow i$ 的直接短语。

还有 $E \overset{*}{\Rightarrow} i_1*i_2+F$，且 $F \Rightarrow i_3$，则 i_3 也是句型 $i_1*i_2+i_3$ 的相对于 F 的短语，也是相

对于规则 $F \rightarrow i$ 的直接短语。

还有 $E \overset{*}{\Rightarrow} T * i_2 + i_3$，且 $T \overset{+}{\Rightarrow} i_1$，则 i_1 是句型 $i_1 * i_2 + i_3$ 的相对于 T 的短语。

还有 $E \overset{*}{\Rightarrow} i_1 * i_2 + T$，且 $T \overset{+}{\Rightarrow} i_3$，则 i_3 是句型 $i_1 * i_2 + i_3$ 的相对于 T 的短语。

还有 $E \overset{*}{\Rightarrow} T + i_3$，且 $T \overset{+}{\Rightarrow} i_1 * i_2$，则 $i_1 * i_2$ 是句型 $i_1 * i_2 + i_3$ 的相对于 T 的短语。

还有 $E \overset{*}{\Rightarrow} E + i_3$，且 $E \overset{+}{\Rightarrow} i_1 * i_2$，则 $i_1 * i_2$ 是句型 $i_1 * i_2 + i_3$ 的相对于 E 的短语。

还有 $E \overset{*}{\Rightarrow} E$，且 $E \Rightarrow i_1 * i_2 + i_3$，则 $i_1 * i_2 + i_3$ 是句型 $i_1 * i_2 + i_3$ 相对于 E 的短语。

即 i_1、i_2、i_3、$i_1 * i_2$ 和 $i_1 * i_2 + i_3$ 都是句型 $i_1 * i_2 + i_3$ 的短语，而且 i_1、i_2、i_3 均为直接短语，其中 i_1 是最左直接短语，即句柄。注意，$i_1 * i_2 + i_3$ 是一个右句型。

虽然 $i_2 + i_3$ 是句型 $i_1 * i_2 + i_3$ 的一部分，但并不是它的短语，因为尽管有 $E \overset{+}{\Rightarrow} i_2 + i_3$，但不存在从文法开始符号 E 到 $i_1 * E$ 的推导。

再讨论例 2.7 的文法的句型 $aabbaa$，为区别其中的 a 和 b，把它写作 $a_1a_2b_1b_2a_3a_4$。它的直接短语有 a_2、b_2a_3 和 a_4，a_2 是句柄；它的短语有 a_2、b_2a_3、a_4、$a_2b_1b_2a_3$ 和 $a_1a_2b_1b_2a_3a_4$。

从句型的推导树上很容易找出句型的短语和直接短语。设 A 是句型 $\alpha\beta\delta$ 的某一子树的根，其中 β 是形成此子树的末端结点的符号串，则 β 是句型 $\alpha\beta\delta$ 的相对于 A 的短语。若这个子树只有一层分支，则 β 是句型 $\alpha\beta\delta$ 的直接短语。

2.7 有关文法实际应用的一些说明

引进文法的目的在于描述程序设计语言。在实际应用中，一方面，需要对文法提出一些限制条件，但这些限制并不真正限制由文法所能描述的语言；另一方面，有时还需要对文法进行扩充，例如，在上下文无关语言的许多说明和分析中允许有称作 ε 规则的产生式（即，对于任何非终结符 A，允许 $A \rightarrow \varepsilon$ 的产生式）。本节就这两个方面的问题进行一些讨论。

2.7.1 有关文法的实用限制

在实际使用中，应限制文法中不得含有**有害规则**和**多余规则**。所谓有害规则，是指形为 $U \rightarrow U$ 的产生式。它对描述语言显然是没有必要的。说它有害，是说它只会引起文法的二义性。所谓多余规则，是指文法中那些连一个句子的推导都用不到的规则，这类规则在文法中以两种形式出现。一种是文法中某些非终结符不在任何规则的右部出现，所以任何句子的推导中都不可能用到它。如例 2.10 的文法 $G[S]$ 中，非终结符 D 不在任何规则的右部出现，那么规则(7)是多余规则，这种非终结符也称为**不可到达的**。另一种情况则是在文法中有这样的非终结符：不能够从它推导出终结符号串来。这种非终结符也称为**不可终止的**。例 2.10 中的文法 $G[S]$ 的非终结符 C 属这种情况，那么规则(6)也是在任何句子的推导中都不能使用的，是多余的。因而规则(2)也是多余的。

例 2.10 有文法 $G[S]$：

(1) $S \rightarrow Be$

(2) $B \rightarrow Ce$

(3) $B \rightarrow Af$

(4) $A \rightarrow Ae$

(5) $A \rightarrow e$

(6) $C \rightarrow Cf$

(7) $D \rightarrow f$

对文法 $G=(V_N, V_T, S, P)$来说，为了保证其非终结符 A 在句子推导中出现，必须满足如下两个条件：

(1) A 必须在某句型中出现。即有 $S \overset{*}{\Rightarrow} \alpha A \beta$，其中 α、β 属于$(V_N \cup V_T)^*$。

(2) 必须能够从 A 推出终结符号串 t 来。即 $A \overset{+}{\Rightarrow} t$，其中 $t \in V_T^*$。

若程序设计语言的文法包含多余规则时，其中必定有错误存在，因此检查文法是否包含多余规则是很有必要的。

2.7.2 上下文无关文法中的ε规则

在本书介绍的定义里，上下文无关文法中某些规则可具有形式 $A \rightarrow \varepsilon$，其中 $A \in V_N$，这种规则称为 ε 规则。但在很多著作和讲义中限制这种规则的出现，是因为 ε 规则会使得有关文法的一些讨论和证明变得复杂。

其实，两种定义的唯一差别是 ε 句子在不在语言中。这不是什么本质问题，前面说过，使用文法作为工具，不仅是为了严格地定义句子的结构，也是为了用适当条数的规则把语言的全部句子描述出来，即找出语言的有穷描述，而如果语言 L 有一个有穷的描述，则 $L1=L \cup \{\varepsilon\}$ 也同样有一个有穷的描述，并且可以证明，若 L 是上下文有关语言、上下文无关语言或正规语言，则 $L \cup \{\varepsilon\}$ 和 $L-\{\varepsilon\}$ 也分别是上下文有关语言、上下文无关语言和正规语言。

下面的几个定理有助于进一步理解上下文无关文法的两种定义的关系。本书不给予任何证明，有兴趣的读者可参考有关书中内容。

定理 2.1 若 L 是由文法 $G=(V_N, V_T, P, S)$产生的语言，P 中的每一个产生式的形式均为 $A \rightarrow \alpha$，其中 $A \in V_N$，$\alpha \in (V_N \cup V_T)^*$（即 α 可能为 ε），则 L 能由这样的一种文法产生，即每一个产生式或者为 $A \rightarrow \beta$ 形式，其中 A 为一个非终结符，即 $A \in V_N$，$\beta \in (V_N \cup V_T)^+$；或者为 $S \rightarrow \varepsilon$ 形式，且 S 不出现在任何产生式的右边。

定理 2.2 如果 $G=(V_N, V_T, P, S)$是一个上下文有关文法，则存在另一个上下文有关文法 $G1$，它所产生的语言与 G 产生的相同，其中 $G1$ 的开始符号不出现在 $G1$ 的任何产生式的右边。又如果 G 是一个上下文无关文法，也能找到这样一个上下文无关文法 $G1$；如果 G 是一个正规文法，则也能找到这样一个正规文法 $G1$。

练　习

1. 文法 $G=(\{A, B, S\}, \{a, b, c\} P, S)$

其中 P 为

$S \rightarrow Ac \mid aB$

$A \rightarrow ab$

$B \rightarrow bc$

写出 $L(G[S])$的全部元素。

2. 文法 $G[N]$为

$N \rightarrow D \mid ND$

$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \mid$

$G[N]$的语言是什么？

3. 为只包含数字、加号和减号的表达式，例如 9－2＋5、3－1、7 等构造一个文法。

4. 证明文法 $G=(\{E,O\},\{(,),+,*,v,d\},P,E)$是二义的，其中 P 为

$E \rightarrow EOE \mid (E) \mid v \mid d$

$O \rightarrow + \mid *$

5. 已知文法 $G[Z]$：

$Z ::= a\ Z\ b$

$Z ::= a\ b$

写出 $L(G[Z])$的全部元素。

6. 已知文法 G：

〈表达式〉::=〈项〉|〈表达式〉＋〈项〉

〈项〉::=〈因子〉|〈项〉*〈因子〉

〈因子〉::=(〈表达式〉)|i

试给出下述表达式的推导及语法树。

(1) i　　(2) (i)　　(3) $i*i$

(4) $i*i+i$　　(5) $i+(i+i)$　　(6) $i+i*i$

7. 习题 1 中的文法 $G[S]$是二义的吗？为什么？

8. 考虑下面的上下文无关文法：

$$S \rightarrow SS* \mid SS+ \mid a$$

(1) 表明通过此文法如何生成串 $aa+a*$，并为该串构造语法树。

(2) 该文法生成的语言是什么？

9. 已知文法 $S \rightarrow S(S)S \mid \varepsilon$。

(1) 该文法生成的语言是什么？

(2) 该文法是二义的吗？说明理由。

10. 令文法 $G[E]$为

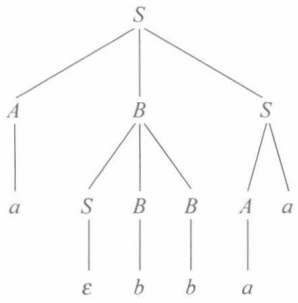
$E \rightarrow T \mid E+T \mid E-T$

$T \rightarrow F \mid T*F \mid T/F$

$F \rightarrow (E) \mid i$

证明 $E+T*F$ 是它的一个右句型，指出这个句型的所有短语、直接短语和句柄。

11. 一个上下文无关文法生成句子 $abbaa$ 的唯一语法树如下：



(1) 给出该句子相应的最左推导和最右推导。

(2) 该文法的产生式集合 P 可能有哪些元素?

(3) 找出该句子的所有短语、简单短语、句柄。

12. 构造产生如下语言的上下文无关文法各一个:

(1) $\{a^n b^n \mid n \geqslant 0\}$

(2) $\{a^m b^n \mid m \geqslant n \geqslant 0\}$

(3) $\{uawb \mid u,w \in \{a,b\}^* \wedge |u| = |w|\}$

(4) $\{a^n b^m \mid n \geqslant 2m \geqslant 0\}$

(5) $\{a^n b^m \mid n \geqslant 0, m \geqslant 0, \text{且 } 3n \geqslant m \geqslant 2n\}$

(6) $\{ww^R \mid w \in \{a,b\}^*\}$,其中,$w^R$ 表示 w 的**反向串**,其含义是将 w 中的字母依次反转,首尾字母交换位置,下同。

(7) $\{uvwv^R \mid u,v,w \in \{a,b\}^+ \wedge |u| = |w| = 1\}$

(8) $\{w \mid w \in \{a,b\} \wedge w = w^R\}$

13. 构造产生如下语言的上下文无关文法各一个:

(1) $\{a^n b^m c^{2m} \mid n,m \geqslant 0\}$

(2) $\{wcw^R \mid w \in \{a,b\}^*\}$

(3) $\{a^m b^n c^k \mid m = n \text{ 或 } n = k\}$

(4) $\{a^m b^n c^k \mid m = k \text{ 或 } n = k\}$

(5) $\{a^n b^m c^k \mid n \leqslant m + k, k \leqslant n, \text{且 } m,n,k \geqslant 1\}$

(6) $\{a^n b^k c^m \mid n,k,m \geqslant 0, \text{且 } n \geqslant m\}$

(7) $\{a^n b^n c^m d^m \mid n \geqslant 1, m \geqslant 1\} \cup \{a^n b^m c^m d^n \mid n \geqslant 1, m \geqslant 1\}$

(8) $\{w_1 c\, w_2 c \cdots c\, w_k c c w_j{}^R \mid k \geqslant 1 \wedge 1 \leqslant j \leqslant k \text{ 且对任何 } 1 \leqslant i \leqslant k, \text{有 } w_i \in \{a,b\}^+\}$

14. 考虑 C 语言中的表达式,圆括号表示函数的参数,方括号表示数组的下标。如果把 C 语言中的表达式里除了括号以外的字符都去掉,例如,$f(a[i] * (b[i][j], c[g(x)]), d[i])$ 去掉除括号以外的字符后就变成了一个括号匹配的串$([](][][()])[])$。试设计一个文法来定义所有的圆括号和方括号都匹配的串。

15. 分以下两种情形,各写一个文法,使其语言是十进制非负偶数的集合:

(1) 允许 0 打头。

(2) 不允许 0 打头。

16. 构造产生语言$\{w \mid w \in \{a,b\}^*$,其中 a、b 的数目不相同$\}$的一个上下文无关文法。

17. 语言$\{a^n b^m c^n d^m \mid a^n b^m c^n d^m \mid n,m \geqslant 0\}$是上下文无关的吗? 能看出它们反映程序设计语言的什么特性吗?

18. 给出生成下述语言的一个 3 型文法：

(1) $\{a^n \mid n \geqslant 0\}$

(2) $\{a^n b^m \mid n, m \geqslant 1\}$

(3) $\{a^n b^m c^k \mid n, m, k \geqslant 0\}$

19. 以下是一个条件表达式文法：

<stmt>→<if-stmt>|other

<if-stmt>→if (<exp>)<stmt>|if (<exp>)<stmt>else <stmt>

<exp>→false |true

其中，<stmt>、<if-stmt>和<exp>为非终结符，<stmt>为开始符号，带下划线的单词为终结符。

(1) 说明该文法是二义的。

(2) 试给出一个无二义的条件表达式文法，使其等价于该文法。

20. 试构造一个程序，使其根据 C 语言语法定义是合法的，但不能被 C 语言的编译程序所接受。

21. 参考 1.4.2 节表 1.1 中 PL/0 语言语法的 EBNF 描述，试给出 PL/0 语言语法的一种上下文无关文法描述。

第3章 词法分析

词法分析是编译的第一个阶段，它的主要任务是从左至右逐个字符地对源程序进行扫描，产生一个个单词序列，用于语法分析。执行词法分析的程序称为词法分析程序或扫描程序。本章讨论词法分析程序的设计原则、单词的描述技术、识别机制及词法分析程序的自动构造原理。

3.1 词法分析程序的设计

3.1.1 词法分析程序和语法分析程序的接口方式

词法分析程序完成的是编译第一阶段的工作。词法分析工作可以是独立的一遍，把字符流的源程序变为单词序列，输出到一个中间文件，这个文件作为语法分析程序的输入而继续编译过程。然而，更一般的情况是将词法分析程序设计成一个子程序，每当语法分析程序需要一个单词时，则调用该子程序。词法分析程序每得到一次调用，便从源程序文件中读入一些字符，直到识别出一个单词，或说直到下一个单词的第一个字符为止。这种设计方案中，词法分析程序和语法分析程序放在同一遍里，而省掉了中间文件或存储区，本书介绍的PL/0编译程序就是这样一种方案。如不特别指明，后续章节中的词法分析程序均指可供语法分析程序调用的子程序，如图3.1所示。

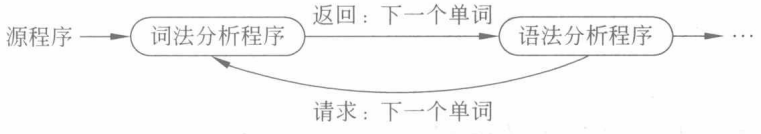


图3.1 语法分析程序调用词法分析程序

3.1.2 词法分析程序的输出

当从语法分析程序接到下一个单词的请求时，词法分析程序从左到右读入源程序的字符流，以识别下一个单词。在识别出下一个单词同时验证其词法正确性之后，词法分析程序将结果以**单词符号**的形式发送至语法分析程序以回应其请求。若在单词识别过程中发现词法错误，则返回出错信息。

单词符号一般可分成下列5类：

(1) 关键字，也称保留字，如Pascal语言中的begin、end、if、while和var等。

(2) 标识符，用来表示各种名字，如常量名、变量名和过程名等。

(3) 常数，各种类型的常数，如25、3.1415、TRUE和"ABC"等。

(4) 运算符，如+、*、<=等。

(5) 界符，如逗号、分号、括号等。

词法分析程序所输出的单词符号可以采用以下二元式表示：

(单词种别,单词自身的值)

单词的种别是语法分析需要的信息,而单词自身的值则是编译其他阶段需要的信息。比如在 PL/0 的语句"const i=25,yes=1;"中的单词 25 和 1 的种别都是常数,常数的值 25 和 1 对于代码生成来说是必不可少的。有时,对某些单词来说,不仅仅需要它的值,还需要其他一些信息以便编译的进行。比如,对于标识符来说,还需要记载它的类别、层次以及其他属性,如果这些属性全部收集在符号表中,那么可以将单词的二元式表示设计成如下形式：

(标识符,指向该标识符所在符号表中位置的指针)

如上述语句中的单词 i 和 yes 的表示为

(标识符,指向 i 的表项的指针)

(标识符,指向 yes 的表项的指针)

单词的种别可以用整数编码表示,假如标识符编码为 1,常数为 2,关键字为 3,运算符为 4,界符为 5,程序段"if i=5 then x=y;"在经词法分析器扫描后输出的单词符号和它们的表示如下：

关键字 if	(3,'if')
标识符 I	(1,指向 i 的符号表入口)
等号=	(4,'=')
常数 5	(2,'5')
关键字 then	(3,'then')
标识符 x	(1,指向 x 的符号表入口)
赋值号=	(4,'=')
标识符 y	(1,指向 y 的符号表入口)
分号;	(5,';')

3.1.3 将词法分析工作分离的考虑

词法也是语法的一部分,词法描述完全可以归并到语法描述中去,只不过词法规则更简单些,这在后面的章节中可以看到。既然这样,为什么将词法分析作为一个独立的阶段？为什么把编译过程的分析工作划分成词法分析和语法分析两个阶段？主要的考虑因素有以下几点：

(1) 使整个编译程序的结构更简洁、清晰和条理化。词法分析比语法分析简单得多,但是由于源程序结构上的一些细节,常使得识别单词的工作极为曲折和费时。例如,空白和注释的处理;再比如对于 FORTRAN 那种受书写格式限制的语言,需在识别单词时进行特殊处理等。如果统统合在语法分析时一并考虑,显然会使得分析程序的结构复杂得多。

(2) 编译程序的效率会改进。大部分编译时间花费在扫描字符以把单词符号分离出来。把词法分析独立出来,采用专门的读字符和分离单词的技术可大大加快编译速度。另外,单词的结构可用有效的方法和工具进行描述和识别,进而可建立词法分析程序的自动构造工具。

(3) 增强编译程序的可移植性。在同一个语言的不同实现中,或多或少地会涉及与设

备有关的特征，比如采用 ASCII 还是 EBCDIC 字符编码。另外，语言的字符集的特殊性的处理，一些专用符号，如 Pascal 中的 ↑ 的表示等，都可置于词法分析程序中解决而不影响编译程序其他成分的设计。

词法分析程序的主要功能是从字符流的源程序中识别单词，它要从左至右逐个字符地扫描源程序，因此它还可完成其他一些任务。比如，滤掉源程序中的注释和空白（由空格、制表符或回车换行字符引起的空白）；又如，为了使编译程序能将发现的错误信息与源程序的出错位置联系起来，词法分析程序负责记录新读入的字符行的行号，以便行号与出错信息相关联；再如，在支持宏处理功能的源语言中，可以由词法分析程序完成其预处理等。很多工作与源语言的具体要求以及编译程序的整个设计有关，在此不一一列举。

3.1.4 词法分析程序中如何识别单词

词法分析中识别下一个单词的过程，简单来看就是逐个读取字符，然后将它们拼在一起的过程。词法分析程序的作用就是在这个拼单词的过程中如何获得下一个有意义的单词符号，即识别出单词种别以及单词自身的值。

要识别出有意义的单词符号，主要是依据程序设计语言的词法规则描述。描述一个语言的词法规则，通常需要借助形式化或半形式化的描述工具，以保证没有歧义性。常见的可用于词法规则描述的工具有状态转换图、扩展巴克斯范式（EBNF）、有限状态自动机、正规表达式以及正规文法等。在词法规则的基础上，进一步设计单词符号的结构。

在 3.2 节将会给出用状态转换图和 EBNF 描述的 PL/0 语言词法规则以及 PL/0 编译器中单词符号的设计。

在识别出有一个意义的单词后，词法分析程序将单词种别连同单词自身的值一起构成一个单词符号，返回给调用它的语法分析程序。

有限状态自动机、正规表达式以及正规文法是适合于正规语言的描述及处理的形式模型。在现实程序设计语言中，几乎任何一种有意义的单词种别对应的单词集合都是正规语言，这些模型的表达能力足以描述任何程序设计语言的词法规则。另外，基于这些语言模型处理正规语言的方法已经非常成熟。因此，以这些模型为基础来设计词法分析程序的自动构造工具是人们目前普遍采取的途径。

本章从 3.3 节至 3.6 节将介绍有关这些形式模型的基础理论和方法，在此基础上读者可以理解词法分析程序自动构造工具的一般原理和方法。

另外，在 3.7 节将结合实例对流行的自动构造工具 lex 的使用进行简介。

3.2 PL/0 编译程序中词法分析程序的设计和实现

本节以 PL/0 编译程序为背景，介绍一个词法分析程序的设计实例，借此使读者进一步了解词法分析程序构造的一些细节。

由 1.4.2 节可知，可将 PL/0 语言的单词分为保留字、运算符、标识符、无符号整数和界符 5 个大类，以下是针对这 5 类单词的一种 EBNF 描述：

```
<无符号整数> ::= <数字>{<数字>}
<标识符>     ::= <字母>{<字母>|<数字>}
```

```
<字母>      ::= a|b|…|X|Y|Z
<数字>      ::= 0|1|2|…|8|9
<保留字>    ::= const|var|procedure|begin|end|odd|if|then|call|while|do|read|write
<运算符>    ::= +|-|*|/|=|#|<|<=|>|>=|:=
<界符>      ::= (|)|,|;|.
```

保留字、运算符和界符这几类各自仅包含有限个单词符号，在实践中更方便将每个单词符号设计为独立的词法单元，即每个单词符号拥有独立的种别。这样，PL/0 编译程序所设计的单词符号对应有 31 个单词种别：标识符 1 个，无符号整数 1 个，保留字 13 个，运算符 11 个，以及界符 5 个。在 PL/0 词法分析程序中，这 31 个单词种别采用下列枚举类型表示：

```
enum symbol {
    nul,        ident,      number,     plus,       minus,
    times,      slash,      oddsym,     eql,        neq,
    lss,        leq,        gtr,        geq,        lparen,
    rparen,     comma,      semicolon,  period,     becomes,
    beginsym,   endsym,     ifsym,      thensym,    whilesym,
    writesym,   readsym,    dosym,      callsym,    constsym,
    varsym,     procsym,
};
```

其中，nul 不对应单词符号，只是出于实现技术的考虑，代表“不能识别的符号”。

例如，在 PL/0 词法分析程序扫描下列语句

```
position :=initial+rate*60;
```

之后，所生成单词符号序列的单词种别对应为

ident becomes ident plus ident times number semicolon

PL/0 词法分析程序定义为：

```
int getsym()
```

PL/0 语法分析程序在需要读取下一个单词时，就调用 getsym()，getsym() 返回下一个单词符号。除标识符和无符号整数外，其他单词符号只包含单词种别的信息。标识符和无符号整数的单词符号包含单词种别和单词自身的值两个部分。由于标识符是在语法分析阶段登录在符号表里的，所以对于标识符来说，PL/0 词法分析程序所返回的单词自身的值不是符号表位置的指针，而是标识符的名字串。

PL/0 编译程序定义 3 个全程变量来传递单词种别和单词自身的值。

(1) 通过全局变量 sym 传递单词种别：

```
enum symbol sym;
```

(2) 通过全局变量 id 传递标识符单词自身的值，即标识符的名字：

```
char id [al+1];                /*al 为预设的标识符最大长度*/
```

(3) 通过全局变量 num 传递无符号整数单词自身的值，即它的整数数值：

```
int num;
```

比如，在识别出标识符 position 之后，全局变量 sym 的值被置为 ident，id 的值被置为“position”；在识别出无符号整数 60 之后，全局变量 sym 的值被置为 number，num 的值被置为整数值 60。

getsym()逐个读取下面的字符，然后将它们拼成下一个有意义的单词，返回相应的单词符号。图 3.2 描述了 PL/0 语言的词法规则，可用于指导单词识别的过程。

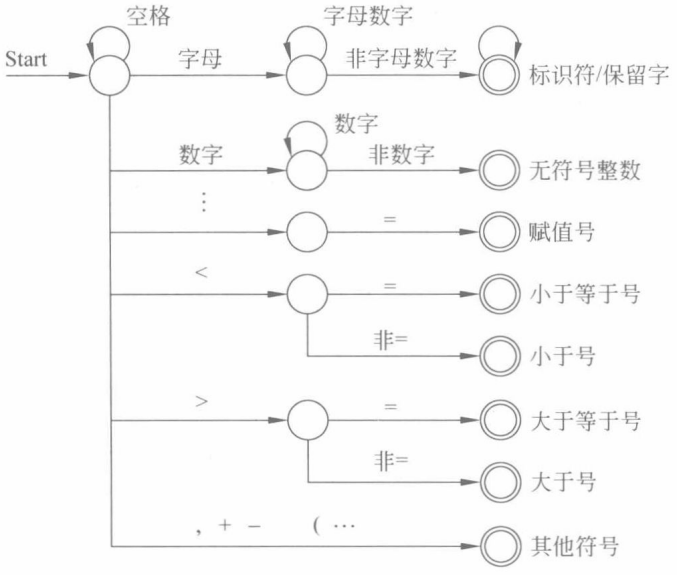


图 3.2　PL/0 词法规则状态转换图

值得注意的是，在识别字母数字串的单词后如何区分是标识符还是保留字。常采取的方法是预设一个保留字表，通过查表来确定是否保留字。比如，在 PL/0 编译程序中定义如下保留字表：

```
/* 设置保留字名字，按照字母顺序排列，便于折半查找 */
strcpy(&(word[0][0]),"begin");
strcpy(&(word[1][0]),"call");
⋮
strcpy(&(word[12][0]),"write");
/* 设置保留字的单词种别 */
wsym[0]=beginsym;
wsym[1]=callsym;
⋮
wsym[12]=writesym;
```

另外，在识别双符号运算符之类的单词时，要注意到可能需要进行字符退还。例如，在读取字符<后，如果下一字符是=，则所识别的单词是小于等于号<=；否则，识别的单词是小于号<，但此时要注意退还已经读到的一个非<字符，即需要保证下一次读到的字符仍然是那个非<字符。

下面是词法分析函数 getsym( )的一个代码片段：

```
int getsym()                              /* 词法分析,获取一个符号 */
{
    ⋮
    while (ch==' '||ch==10||ch==13||ch==9)          /* 忽略空格、换行、回车和 Tab */
    {
        getchdo;                          /* 取下一字符到 ch */
    }
    if (ch>='a' && ch<='z')               /* 标识符或保留字以 a~z 开头 */
    {
        ⋮                                 /* 标识符或保留字的字母数字串置于字符数组 a */
        strcpy(id,a);                     /* 设置标识符或保留字名字串 id */
        ⋮                                 /* 在保留字表 wsym 中搜索当前符号是否为保留字 */
        if (...)                          /* 是保留字 */
        {
            sym=wsym[k];                  /* 置保留字的单词种别至 sym */
        }
        else                              /* 搜索失败,不是保留字 */
        {
            sym=ident;                    /* 置单词种别全局量 sym 为 ident,即标识符 */
        }
    }
    else
    {
        if (ch>='0' && ch<='9')           /* 检测是否为数字:以 0~9 开头 */
        {
            ⋮
            sym=number;                   /* 置单词种别全局量 sym 为 number,即无符号整数 */
            ⋮                             /* 获取数字并转换为十进制整数值,置于 num */
            if (...)                      /* 数字的位数超出允许的范围,报错 */
            {
                error(30);
            }
        }
        else                              /* 不是数字 */
        {
            if (ch==':')                  /* 检测赋值符号 */
            {
                getchdo;
                if (ch=='=')
                {
                    sym=becomes;          /* 置单词种别为 number,即赋值符号 */
                    getchdo;
                }
                else
                {
```

```
                sym=nul;        /* 不能识别的符号 */
            }
        }
        else
        {
            if (ch=='<')        /* 检测小于或小于等于符号 */
            {
                getchdo;
                if (ch=='=')
                {
                    sym=leq;    /* 置单词种别为 leq,即小于等于符号 */
                    getchdo;
                }
                else
                {
                    sym=lss;    /* 置单词种别为 lss,即小于符号 */
                }
            }
            else
            {
                if (ch=='>')    /* 检测大于或大于等于符号 */
                {
                    :           /* 类似于检测小于等于符号的情形 */
                                /* 置单词种别为 geq 或 gtr */
                }
                else
                {               /* 当符号不满足上述条件时,全部按照单字符符号处理 */
                    sym=ssym[ch];
                    :
                }
            }
        }
    }
    return 0;
}
```

这一代码片段可以体现函数 getsym()的基本流程,其控制过程的几个重要方面如下:

(1) 识别空格。空格在词法分析时是一种不可缺少的界符,而在语法分析时则是无用的,所以需要滤掉。

(2) 识别保留字和标识符。如前所述,PL/0 编译程序中定义了一个保留字表 word。对每个字母开头的字母数字字符串要查这个表。保留字表按字母顺序存放,词法分析程序使用折半查找。若可以查到,则识别为保留字,将对应的单词种别(存于 wsym 表中)放在 sym 中(如 if 的对应值为 ifsym,then 的对应值为 thensym);若查不到,则认为是用户定义的标识符,将 sym 置为 ident,而将代表标识符名字的串存放于 id 中。

(3) 拼数。当扫描到数字串时，将字符串形式的十进制数转换成机内表示的二进制数，然后把单词种别 number 放在 sym 中，数值本身的值存放在 num 中。

(4) 其他单字符或双字符的界符、运算符识别后将相应的单词种别送至 sym 中。

函数 getsym( )在需要取下一字符时调用 getchdo，它是对函数 getch( )的包装。getch( )的基本功能是：略过空格，读取一个字符；每次读入源文件的一行，存入 line 缓冲区，line 被 getsym 取空后再读一行。这里不必对 getch( )的源码作进一步解释。

3.3 单词的形式化描述工具

如 3.1.4 节所述，描述一个程序设计语言的词法规则，通常需要借助形式化或半形式化的描述工具。本节主要介绍有限状态自动机、正规表达式以及正规文法等形式化描述工具的基础理论和方法，在此基础上读者可以理解词法分析程序自动构造工具的一般原理和方法。

3.3.1 正规文法

正规文法也称为 3 型文法 $G=(V_N,V_T,S,P)$，其 P 中的每一条规则都有下述形式：$A\to aB$ 或 $A\to a$，其中 $A,B\in V_N$，$a\in V_T^*$。正规文法所描述的是 V_T 上的正规集。

程序设计语言中的几类单词可用下述规则描述：

〈标识符〉→l|l〈字母数字〉

〈字母数字〉→l|d|l〈字母数字〉|d〈字母数字〉

〈无符号整数〉→d|d〈无符号整数〉

〈运算符〉→+|−|*|/|=|=〈等号〉…

〈等号〉→=

〈界符〉→,|;|(|)|…

其中 l 表示 a～z 中的任一英文字母，d 表示 0～9 中的任一数字。

关键字也是一种单词，一般关键字都是由字母构成的，它的描述也极容易，实际上，关键字集合是标识符集合的子集。

比较复杂的单词，如无符号实数 25.55e+5 和 2.1 等，它们可以由例 3.1 的规则描述。

例 3.1

〈无符号数〉→d〈余留无符号数〉|.〈十进小数〉|e〈指数部分〉

〈余留无符号数〉→d〈余留无符号数〉|.〈十进小数〉|e〈指数部分〉|ε

〈十进小数〉→d〈余留十进小数〉

〈余留十进小数〉→e〈指数部分〉|d〈余留十进小数〉|ε

〈指数部分〉→d〈余留整指数〉|+〈整指数〉|−〈整指数〉|

〈整指数〉→d〈余留整指数〉

〈余留整指数〉→d〈余留整指数〉|ε

3.3.2 正规式

正规式也称正则表达式，也是表示正规集的工具。它是用以描述单词符号的又一方便工具。

下面是正规式和它所表示的正规集的递归定义。设字母表为 Σ，辅助字母表 $\Sigma'=\{\varnothing,\varepsilon,|,.,*,(,)\}$。

(1) ε 和 $\varnothing$ 都是 Σ 上的正规式，它们所表示的正规集分别为 $\{\varepsilon\}$ 和 $\varnothing$。

(2) 任何 $a\in\Sigma$，a 是 Σ 上的一个正规式，它所表示的正规集为 $\{a\}$。

(3) 假定 e_1 和 e_2 都是 Σ 上的正规式，它们所表示的正规集分别为 $L(e_1)$ 和 $L(e_2)$，那么，(e_1)、$e_1|e_2$、$e_1\cdot e_2$ 和 e_1^* 也都是正规式，它们所表示的正规集分别为 $L(e_1)$、$L(e_1)\cup L(e_1)$、$L(e_1)L(e_2)$ 和 $(L(e_1))^*$。

(4) 仅由有限次使用上述 3 个步骤而定义的表达式才是 Σ 上的正规式，仅由这些正规式所表示的符号串的集合才是 Σ 上的正规集。

其中的“|”读为“或”(也有使用“+”代替“|”的)；“.”读为“连接”；“*”读为“闭包”(即任意有限次的自重复连接)。在不致混淆时，括号可省去，但规定算符的优先顺序为先“*”，再“.”，最后“|”。连接符“.”一般可省略不写。“*”、“.”和“|”都是左结合的。

例 3.2 令 $\Sigma=\{a,b\}$，Σ 上的正规式和相应的正规集的例子如下：

正规式	正规集
a	$\{a\}$
$a\|b$	$\{a,b\}$
ab	$\{ab\}$
$(a\|b)(a\|b)$	$\{aa,ab,ba,bb\}$
a^*	$\{\varepsilon,a,aa,\cdots\}$，即任意个 a 的串
$(a\|b)^*$	$\{\varepsilon,a,b,aa,ab\cdots\}$，即所有 a，b 组成的串
$(a\|b)^*(aa\|bb)(a\|b)^*$	Σ^* 上所有含有两个相继的 a 或两个相继的 b 组成的串

例 3.3 令 $\Sigma=\{d,.,e,+,-\}$，则 Σ 上的正规式 $d^*(.dd^*|\varepsilon)(e(+|-|\varepsilon)dd^*|\varepsilon)$ 表示的是无符号数。其中 d 为 0～9 中的数字。例如，2、12.59、3.6e2 和 471.88e－1 等都是该正规式所表示的集合中的元素。

若两个正规式 e_1 和 e_2 所表示的正规集相同，则说 e_1 和 e_2 等价，写作 $e_1=e_2$。例如，若 $e_1=a|b$，$e_2=b|a$，则有 $e_1=e_2$，即 $a|b=b|a$。又如，$b(ab)^*=(ba)^*b$，$(a|b)^*=(a^*b^*)^*$。

设 r、s、t 为正规式，正规式服从的代数规律如下：

(1) $r|s=s|r$　　“或”的交换律

(2) $r|(s|t)=(r|s)|t$　　“或”的可结合律

(3) $(rs)t=r(st)$　　“连接”的可结合律

(4) $r(s|t)=rs|rt$，$(s|t)r=sr|tr$　　分配律

(5) $\varepsilon r=r$，$r\varepsilon=r$　　ε 是“连接”的恒等元素

(6) $r|r=r$　　“或”的抽取律

程序设计语言中的单词都能用正规式来定义。在例 3.3 中给出了定义无符号数的正规

式。又如，Σ={字母，数字}上的正规式 e_1=字母(字母|数字)* 表示的是所有标识符的集合，或者用 l 代表字母，d 代表数字，$\Sigma=\{l,d\}$，即 $e_1=l(l|d)^*$。正规式 $e_2=dd^*$ 定义了无符号整数。

3.3.3 正规文法和正规式的等价性

一个正规语言可以由正规文法定义，也可以由正规式定义，对任意一个正规文法，存在一个定义同一个语言的正规式；反之，对每个正规式，存在一个生成同一个语言的正规文法，有些正规语言很容易用文法定义，有些正规语言更容易用正规式定义，本节介绍两者间的转换，从结构上建立它们的等价性。

1. 将正规式转换成正规文法

将 Σ 上的一个正规式 r 转换成文法 $G=(V_N,V_T,S,P)$。令 $V_T=\Sigma$，确定产生式和 V_N 的元素用如下办法：

选择一个非终结符 S 生成类似产生式的形式：$S\to r$，并将 S 定为 G 的识别符号。为表述方便，将 $S\to r$ 称作正规式产生式，因为在→的右部中含有“.”、“*”或“|”等正规式符号，不是 V 中的符号。

若 x 和 y 都是正规式，对形如 $A\to xy$ 的正规式产生式，重写成 $A\to xB$，$B\to y$ 两个产生式，其中 B 是新选择的非终结符，即 $B\in V_N$。

对形如 $A\to x^* y$ 的正规式产生式，重写为

$A\to xB$

$A\to y$

$B\to xB$

$B\to y$

其中 B 为一个新的非终结符。

对形如 $A\to x|y$ 的正规式产生式，重写为

$A\to x$

$A\to y$

不断利用上述规则做变换，直到每个产生式都符合正规文法的形式。

例 3.4 将 $r=a(a|d)^*$ 转换成相应的正规文法。

令 S 是文法的开始符号，首先形成 $S\to a(a|d)^*$，然后形成 $S\to aA$ 和 $A\to(a|d)^*$，再变换形成

$S\to aA \quad A\to(a|d)B$

$A\to\varepsilon \quad B\to(a|d)B$

$B\to\varepsilon$

进而变换为全部符合正规文法产生式的形式：

$S\to aA \quad B\to aB$

$A\to aB \quad B\to dB$

$A\to dB \quad B\to\varepsilon$

$A\to\varepsilon$

2. 将正规文法转换成正规式

这一转换过程基本上是上述过程的逆过程，最后只剩下一个开始符号定义的正规式。其转换规则列于表 3.1。

表 3.1　正规文法到正规式的转换规则

	文法产生式	正　规　式
规则 1	$A\to xB \quad B\to y$	$A=xy$
规则 2	$A\to xA \mid y$	$A=x^* y$
规则 3	$A\to x \quad A\to y$	$A=x \mid y$

例 3.5　文法 $G[S]$如下：

$S\to aA$

$S\to a$

$A\to aA$

$A\to dA$

$A\to a$

$A\to d$

首先有

$S=aA \mid a$

$A=(aA \mid dA) \mid (a \mid d)$

再将 A 的正规式变换为 $A=(a \mid d)A \mid (a \mid d)$，又变换为 $A=(a \mid d)^* (a \mid d)$，再将 A 右端代入 S 的正规式得

$S=a(a \mid d)^* (a \mid d) \mid a$

再利用正规式的代数变换可依次得到

$S=a(a \mid d)^* (a \mid d) \mid \varepsilon$

$S=a(a \mid d)^*$

即 $a(a \mid d)^*$ 为所求。

3.4　有穷自动机

有穷自动机(也称有限自动机)作为一种识别装置，能准确地识别正规集，即识别正规文法所定义的语言和正规式所表示的集合。引入有穷自动机理论，正是为词法分析程序的自动构造寻找特殊的方法和工具。

有穷自动机分为两类：确定的有穷自动机(Deterministic Finite Automata，DFA)和不确定的有穷自动机(Nondeterministic Finite Automata，NFA)。下面分别给出确定的有穷自动机和不确定的有穷自动机的定义、与其有关的概念、不确定的有穷自动机的确定化以及确定的有穷自动机的化简等算法。

3.4.1　确定的有穷自动机(DFA)

一个确定的有穷自动机 M 是一个五元组：

$$M=(K,\Sigma,f,S,Z)$$

其中：

(1) K 是一个有穷集，它的每个元素称为一个状态。

(2) Σ 是一个有穷字母表，它的每个元素称为一个输入符号，所以也称 Σ 为输入符号表。

(3) f 是转换函数，是 $K\times\Sigma\to K$ 上的映像。例如，$f(k_i,a)=k_j(k_i\in K,k_j\in K)$，就意味着，当前状态为 k_i、输入字符为 a 时，将转换到下一状态 k_j，把 k_j 称作 k_i 的一个后继状态。

(4) $S\in K$，是唯一的一个初态。

(5) $Z\subseteq K$，是一个终态集，终态也称可接受状态或结束状态。

例 3.6 DFA $M=(\{S,U,V,Q\},\{a,b\},f,S,\{Q\})$，其中 f 定义为

$$f(S,a)=U \quad f(U,a)=Q \quad f(V,a)=U \quad f(Q,a)=Q$$
$$f(S,b)=V \quad f(U,b)=V \quad f(V,b)=Q \quad f(Q,b)=Q$$

一个 DFA 可以表示成一个状态图(或称状态转换图)。假定 DFA M 含有 m 个状态，n 个输入符号，那么这个状态图含有 m 个结点，每个结点最多有 n 个弧射出，整个图含有唯一一个初态结点和若干个终态结点，初态结点冠以“⇒”或标以“－”，终态结点用双圈表示或标以“＋”，若 $f(k_i,a)=k_j$，则从状态结点 k_i 到状态结点 k_j 画标记为 a 的弧。

例 3.6 中的 DFA 的状态图表示如图 3.3 所示。

一个 DFA 还可以用一个矩阵表示，该矩阵的行表示状态，列表示输入符号，矩阵元素表示相应状态和输入符号将转换成的新状态，即 k 行 a 列为 $f(k,a)$的值。可以用“⇒”标明初态；否则第一行即是初态，相应终态行在表的右端标以 1，非终态标以 0。例 3.5 中的 DFA 的矩阵表示如图 3.4 所示。

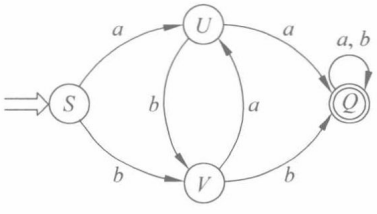


图 3.3　状态图表示

状态＼符号	a	b	
S	U	V	0
U	Q	V	0
V	U	Q	0
Q	Q	Q	1

图 3.4　矩阵表示

对于 Σ^* 中的任何符号串 t，若存在一条从初态结点到某一终态结点的道路，且这条路上所有弧的标记符连接成的符号串等于 t，则称 t 可为 DFA M 所接受，若 M 的初态结点同时又是终态结点，则空字可为 M 所识别(接受)。

可换一种方式叙述如下：

若 $t\in\Sigma^*$，$f(S,t)=P$，其中 S 为 DFA M 的开始状态，$P\in Z$，Z 为终态集。则称 t 可为 DFA M **所接受(识别)**。

为了描述一个符号串 t 可为 DFA M 所接受，需要将转换函数扩充；设 $Q\in K$，函数 $f(Q,\varepsilon)=Q$，即，如果输入符号是空串，则仍停留在原来的状态上；还需要借助下述定义：一个输入符号串 t(将它表示成 t_1t_x 的形式，其中 $t_1\in\Sigma,t_x\in\Sigma^*$)，在 DFA M 上**运行**的定

义为

$$f(Q,t_1t_x) = f(f(Q,t_1),t_x)$$

例如，试证 $baab$ 可为例 3.6 的 DFA 所接受。

因为 $f(S,baab)=f(f(S,b),aab)=f(V,aab)=f(f(V,a),ab)=f(U,ab)=f(f(U,a),b)=f(Q,b)=Q$，$Q$ 属于终态，得证。

DFA M 所能接受的符号串的全体(字的全体)记为 $L(M)$。

结论 Σ 上的一个符号串集 $V\subseteq\Sigma^*$ 是正规的，当且仅当存在一个 Σ 上的确定有穷自动机 M，使得 $V=L(M)$。

DFA 的确定性表现在转换函数 f：$K\times\Sigma\to K$ 是一个单值函数，也就是说，对任何状态 $k\in K$ 和输入符号 $a\in\Sigma$，$f(k,a)$ 唯一地确定了下一个状态。从状态转换图来看，若字母表 Σ 含有 n 个输入符号，那么任何一个状态结点最多有 n 条弧射出，而且每条弧以一个不同的输入符号标记。

3.4.2 不确定的有穷自动机(NFA)

一个不确定的有穷自动机 M 是一个五元组：

$$M = (K,\Sigma,f,S,Z)$$

其中：

(1) K 是一个有穷集，它的每个元素称为一个状态。

(2) Σ 是一个有穷字母表，它的每个元素称为一个输入符号。

(3) f 是一个从 $K\times\Sigma^*$ 到 K 的全体子集的映像，即 $K\times\Sigma^*\to 2^K$，其中 2^K 表示 K 的幂集。

(4) $S\subseteq K$，是一个非空初态集。

(5) $Z\subseteq K$，是一个终态集。

一个含有 m 个状态和 n 个输入符号的 NFA 可表示成一张状态转换图，这张图含有 m 个状态结点，每个结点可射出若干条箭弧与别的结点相连接，每条弧用 Σ^* 中的一个串作标记，整个图至少含有一个初态结点以及若干个终态结点。

例 3.7 一个 NFA $M=(\{0,1,2,3,4\},\{a,b\},f,\{0\},\{2,4\})$，其中：

$f(0,a)=\{0,3\}$

$f(0,b)=\{0,1\}$

$f(1,b)=\{2\}$

$f(2,a)=\{2\}$

$f(2,b)=\{2\}$

$f(3,a)=\{4\}$

$f(4,a)=\{4\}$

$f(4,b)=\{4\}$

它的状态图表示如图 3.5 所示。

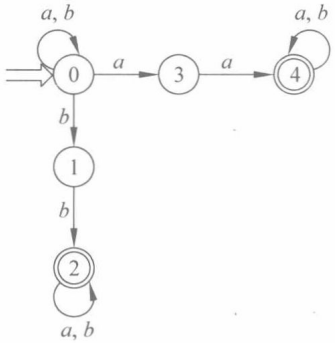


图 3.5 NFA M 的状态图

一个 NFA 也可以用一个矩阵表示。另外一个输入符号串在 NFA 上“运行”的定义也类似于对 DFA 给出的形式。

留给读者自己练习。

对于 Σ^* 中的任何一个串 t，若存在一条从某一初态结点到某一终态结点的道路，且这条道路上所有弧的标记字依序连接成的串（不理睬那些标记为 ε 的弧）等于 t，则称 t 可为 NFA M 所**识别**（**读出**或**接受**）。若 M 的某些结点既是初态结点又是终态结点，或者存在一条从某个初态结点到某个终态结点的 ε 道路，那么空字可为 M 所接受。

例 3.7 中的 NFA M 所能识别的是那些含有相继两个 a 或相继两个 b 的串。

显然 DFA 是 NFA 的特例。**对于每个 NFA M，存在一个 DFA M'，使得 $L(M)=L(M')$**。对于任何两个有穷自动机 M 和 M'，如果 $L(M)=L(M')$，则称 M 与 M' 是**等价**的。

3.4.3 节介绍一种算法，对于给定的 NFA M，构造其等价的 DFA M'。

3.4.3 NFA 转换为等价的 DFA

在有穷自动机的理论里，有这样的定理：**设 L 为一个由不确定的有穷自动机接受的集合，则存在一个接受 L 的确定的有穷自动机**。这里不对定理进行证明，只介绍一种算法，将 NFA 转换成接受同样语言的 *DFA*。这种算法称为**子集法**。

为一个 NFA 构造相应的 DFA 的基本想法是让 DFA 的每一个状态对应 NFA 的一组状态。也就是让 DFA 使用它的状态去记录在 NFA 读入一个输入符号后可能达到的所有状态，在读入输入符号串 $a_1a_2\cdots a_n$ 之后，DFA 处在那样一个状态，该状态表示这个 NFA 的状态的一个子集 T，T 是从 NFA 的开始状态沿着某个标记为 $a_1a_2\cdots a_n$ 的路径可以到达的那些状态构成的。

介绍子集法之前先定义状态集合 I 的两个运算：

(1) 状态集合 I 的 ε-闭包，表示为 ε-closure(I)，定义为一个状态集，是状态集 I 中的任何状态 S 经任意条 ε 弧而能到达的状态的集合。

回顾在前面章节对转换函数的扩充：如输入符号是空串，则自动机仍停留在原来的状态上，显然，状态集合 I 的任何状态 S 都属于 ε-closure(I)。

(2) 状态集合 I 的 a 弧转换，表示为 move(I,a)，定义为状态集合 J，其中 J 是所有那些可从 I 中的某一状态经过一条 a 弧而到达的状态的全体。

下面用图 3.6 的 NFA N 的状态集合来理解上述两个运算。

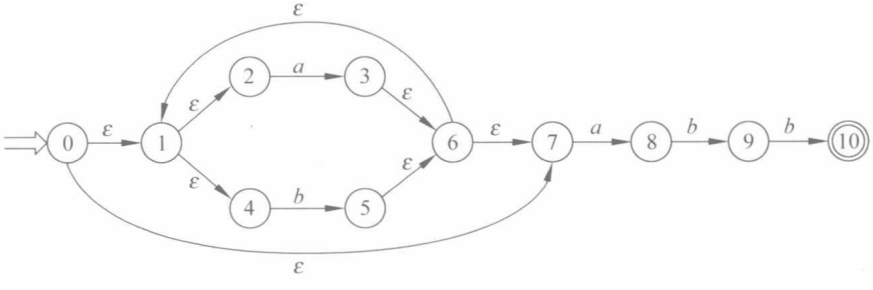


图 3.6　NFA N 的状态图

ε-closure(0)={0,1,2,4,7}

即{0,1,2,4,7}中的任一状态都是从状态 0 经任意条 ε 弧可到达的状态，令{0,1,2,4,7}=A，则 move(A,a)={3,8}，因为在状态 0,1,2,4 和 7 中，只有状态 2 和 7 有 a 弧射出，分别

到达状态 3 和 8。

而 ε-closure({3,8})={1,2,3,4,6,7,8}。

对于一个 NFA $N=(K,\Sigma,f,K_0,K_t)$来说，若 I 是 K 的一个子集，不妨设 $I=\{S_1,S_2,\cdots,S_j\}$，a 是 Σ 中的一个元素，则 $\text{move}(I,a)=f(S_1,a)\cup f(S_2,a)\cup\cdots\cup f(S_j,a)$。

假设 NFA $N=(K,\Sigma,f,K_0,K_t)$按如下办法构造一个 DFA $M=(S,\Sigma,D,S_0,S_t)$使得 $L(M)=L(N)$：

(1) M 的状态集 S 由 K 的一些子集组成(构造 K 的子集的算法见图 3.7)。用$[S_1,S_2,\cdots,S_j]$表示 S 的元素，其中 $S_1,S_2,\cdots,S_j$ 是 K 的状态。并且约定，状态 $S_1,S_2,\cdots,S_j$ 是按某种规则排列的，即对于子集 $S=\{S_2,S_1\}$来说，S 的状态就是$[S_1S_2]$。

```
① 开始，令 ε-closure(K0)为 C 中唯一成员，并且它是未被标
   记的。
② While(C 中存在尚未被标记的子集 T) do
{标记 T;
    for 每个输入字母 a do
        {U :=ε-closure (Move(T,a));
            if U 不在 C 中 then
              将 U 作为未被标记的子集加在 C 中
        }
}
```

图 3.7　子集构造算法

(2) M 和 N 的输入字母表是相同的，即是 Σ。

(3) 转换函数 D 是这样定义的。

$$D([S_1,S_2,\cdots,S_j],a)=[R_1,R_2,\cdots,R_i]$$

其中 ε-closure(move($[S_1,S_2,\cdots,S_j]$,a))=$[R_1,R_2,\cdots,R_i]$。

(4) $S_0=\varepsilon\text{-closure}(K_0)$为 M 的开始状态。

(5) $S_t=\{[S_j,S_k,\cdots,S_e]$，其中$[S_j,S_k,\cdots,S_e]\in S$ 且$\{S_j,S_k,\cdots,S_e\}\cap K_t\neq\varnothing\}$。

图 3.7 是构造 NFA N 的状态 K 的子集的算法。假定所构造的子集族为 C，即 $C=(T_1,T_2,\cdots,T_i)$，其中 $T_1,T_2,\cdots,T_i$ 为状态 K 的子集。

例 3.8　应用图 3.7 的算法对图 3.6 的 NFA N 构造子集，步骤如下：

(1) 首先计算 ε-closure(0)，令 $T_0=\varepsilon\text{-closure}(0)=\{0,1,2,4,7\}$，$T_0$ 未被标记，它现在是子集族 C 的唯一成员。

(2) 标记 T_0；令 $T_1=\varepsilon\text{-closure}(\text{move}(T_0,a))=\{1,2,3,4,6,7,8\}$，将 T_1 加入 C 中，T_1 未被标记。

令 $T_2=\varepsilon\text{-closure}(\text{move}(T_0,b))=\{1,2,4,5,6,7\}$，将 T_2 加入 C 中，它未被标记。

(3) 标记 T_1；计算 ε-closure(move(T_1,a))，结果为{1,2,3,4,6,7,8}，即 T_1，T_1 已在 C 中。

计算 ε-closure(move(T_1,b))，结果为{1,2,4,5,6,7,9}，令其为 T_3，加至 C 中，它未被标记。

(4) 标记 T_2，计算 ε-closure(move(T_2,a))，结果为{1,2,3,4,6,7,8}，即 T_1，T_1 已在 C 中。

计算 ε-closure(move(T_2,b))，结果为{1,2,4,5,6,7}，即 T_2，T_2 已在 C 中。

(5) 标记 T_3，计算 ε-closure(move(T_3,a))，结果为{1,2,3,4,6,7,8}，即 T_1。

计算 ε-closure(move(T_3,b))，结果为{1,2,4,5,6,7,10}，令其为 T_4，加入 C 中，T_4 未被标记。

(6) 标记 T_4，计算 ε-closure(move(T_4,a))，结果为{1,2,3,4,6,7,8}，即 T_1。

计算 ε-closure(move(T_4,b))，结果为{1,2,4,5,6,7}，即 T_2。

至此，算法终止，共构造了 5 个子集：

$T_0=\{0,1,2,4,7\}$

$T_1=\{1,2,3,4,6,7,8\}$

$T_2=\{1,2,4,5,6,7\}$

$T_3=\{1,2,4,5,6,7,9\}$

$T_4=\{1,2,4,5,6,7,10\}$

那么图 3.6 的 NFA N 构造的 DFA M 如下：

(1) $S=\{[T_0],[T_1],[T_2],[T_3],[T_4]\}$

(2) $\Sigma=\{a,b\}$

(3) $D([T_0],a)=[T_1]$

$D([T_0],b)=[T_2]$

$D([T_1],a)=[T_1]$

$D([T_1],b)=[T_3]$

$D([T_2],a)=[T_1]$

$D([T_2],b)=[T_2]$

$D([T_3],a)=[T_1]$

$D([T_3],b)=[T_4]$

$D([T_4],a)=[T_1]$

$D([T_4],b)=[T_2]$

(4) $S_0=[T_0]$

(5) $S_t=[T_4]$

为便于书写，不妨将$[T_0]$、$[T_1]$、$[T_2]$、$[T_3]$、$[T_4]$重新命名，用 0、1、2、3、4 分别表示，该 DFA M 的状态转换图如图 3.8 所示。

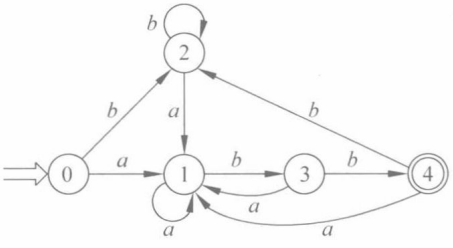


图 3.8　DFA M 的状态图

3.4.4　确定有穷自动机的化简

所谓一个有穷自动机是化简了的，就是说它没有多余状态并且它的状态中没有两个是互相等价的。一个有穷自动机可以通过消除无用状态和合并等价状态而转换成一个与之等价的最小状态的有穷自动机。

所谓有穷自动机的无用状态，是指这样的状态：从该自动机的开始状态出发，任何输入

串也不能到达的那个状态，或者从这个状态没有通路到达终态。例如，图 3.9(a)的有穷自动机 M 中的状态 s_4 便是无用状态。

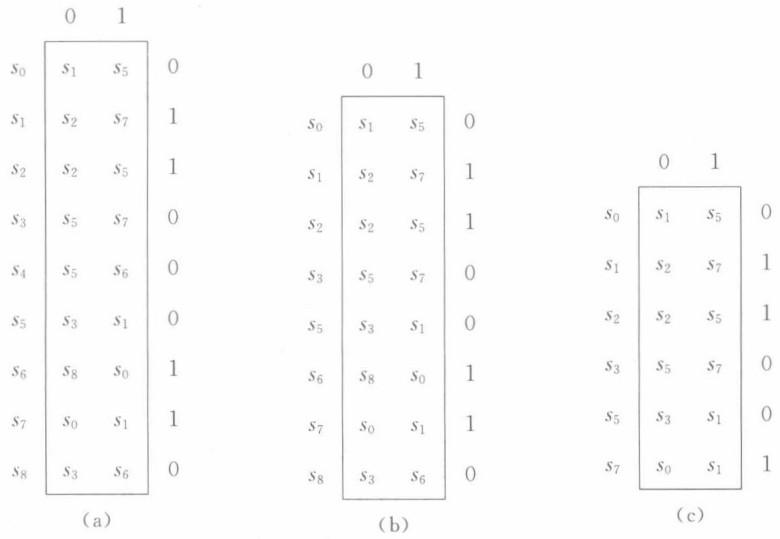


图 3.9　消除多余状态

对于给定的有穷自动机，如果它含有无用状态，可以非常简单地将无用状态消除，而得到与它等价的有穷自动机。例如，图 3.9(a)的状态 s_4 连同状态 s_4 射出的两个弧消掉，得到如图 3.9(b)的有穷自动机。而在图 3.9(b)中，状态 s_6 和 s_8 也是不能从开始状态经由任何输入串而到达的，也将它们连同由它们射出的弧消除而得到如图 3.9(c)的有穷自动机。

在有穷自动机中，两个状态 s 和 t 等价的条件是以下两个：

(1) 一致性条件——状态 s 和 t 必须同时为可接受状态或不可接受状态。

(2) 蔓延性条件——对于所有输入符号，状态 s 和状态 t 必须转换到等价的状态里。

如果有穷自动机的状态 s 和 t 不等价，则称这两个状态是可区别的。显然在图 3.8 的 DFA M 中，状态 0 和 4 是可区别的，因为状态 4 是可接受态(终态)，而 0 是不可接受态。又如状态 2 和 3 是可区别的，因为状态 2 读出 b 后到达 2，状态 3 读出 b 后到达 4，而 2 和 4 是不等价的。

下面介绍一个方法，叫做“分割法”，来把一个 DFA(不含多余状态)的状态分成一些不相交的子集，使得任何不同的两个子集的状态都是可区别的，而同一子集中的任何两个状态都是等价的。通过将此方法施于图 3.10 的 DFA M 上来做一介绍。

例 3.9　将图 3.10 中的 DFA M 最小化。

首先将 M 的状态分成两个子集：一个由终态(可接受态)组成，另一个由非终态组成，这个初始划分为 $P_0=(\{1,2,3,4\},\{5,6,7\})$，显然第 1 个子集中的任何状态都不与第 2 个子集中的状态等价。

现在观察第一个子集$\{1,2,3,4\}$，在读入输入符号 a 后，状态 3 和 4 分别转换为第 1 个子集中所含的状态 1 和 4，而 1 和 2 分别转换为第 2 个子集中所含的状态 6 和 7，这就意味着$\{1,2\}$中的任何状态和$\{3,4\}$中的任何状态在读入 a 后到达了不等价的状态，因此$\{1,2\}$中的任何状态与$\{3,4\}$中的任何状态都是可区别的，因此得到了新的划分如下：

$$P_1=(\{1,2\}\{3,4\}\{5,6,7\})$$

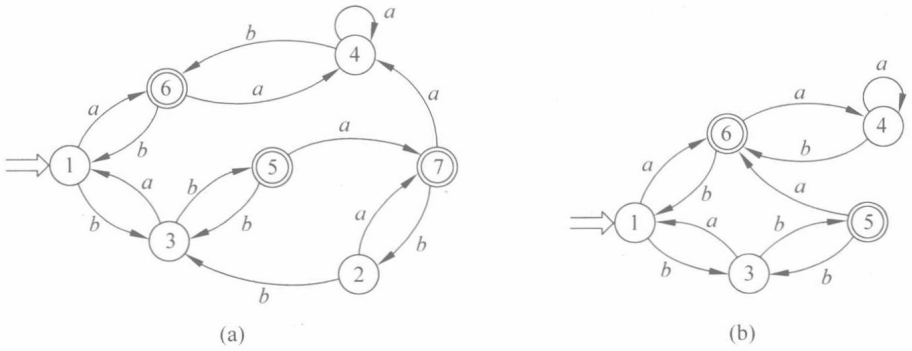


图 3.10　DFA M 和 DFA M'

接着试图在 P_1 中寻找一个子集和一个输入符号使得这个子集中的状态可区别，P_1 中的子集{3,4}对应输入符号 a 将再分割，而得到划分 $P_2=(\{1,2\},\{3\},\{4\},\{5,6,7\})$。

P_2 中的{5,6,7}可由输入符号 a 或 b 而分割，得到划分 $P_3=(\{1,2\},\{3\},\{4\},\{5\},\{6,7\})$。

经过考察，P_3 不能再划分了。令 1 代表{1,2}，消去 2，令 6 代表{6,7}，消去 7，便得到了图 3.10(b)的 DFA M'，它是图 3.8(a)的 DFA M 的最小化。

比起原来的有穷自动机，化简了的有穷自动机具有较少的状态，因而在计算机上实现起来要简洁些。

3.5　正规式和有穷自动机的等价性

正规式和有穷自动机的等价性由以下两点说明：

(1) 对于 Σ 上的 NFA M，可以构造一个 Σ 上的正规式 r，使得 $L(r)=L(M)$。

(2) 对于 Σ 上的每个正规式 r，可以构造一个 Σ 上的 NFA M，使得 $L(M)=L(r)$。

首先介绍如何为 Σ 上的 NFA M 构造相应的正规式 r。

把状态转换图的概念拓广，令每条弧可用一个正规式作标记。

第 1 步，在 M 的状态转换图上加进两个结点，一个为 x 结点，一个为 y 结点。从 x 结点用 ε 弧连接到 M 的所有初态结点，从 M 的所有终态结点用 ε 弧连接到 y 结点。形成一个与 M 等价的 M'，M' 只有一个初态 x 和一个终态 y。

第 2 步，逐步消去 M' 中的所有结点，直至只剩下 x 和 y 结点。在消去过程中，逐步用正规式来标记弧。其消去的规则如下：

(1) 对于 $1 \xrightarrow{r_1} 2 \xrightarrow{r_2} 3$ 代之以 $1 \xrightarrow{r_1 r_2} 3$。

(2) 对于 1 与 2 之间有标记 r_1 和 r_2 两条弧 代之以 $1 \xrightarrow{r_1|r_2} 2$。

(3) 对于 $1 \xrightarrow{r_1} 2$（2 有自环 r_2）$\xrightarrow{r_3} 3$ 代之以 $1 \xrightarrow{r_1 r_2^* r_3} 3$。

最后 x 和 y 结点间的弧上的标记则为所求的正规式 r。

例 3.10　以例 3.7 的 NFA M 为例，M 的状态图见图 3.5，求正规式 r，使 $L(r)=L(M)$。

第 1 步，加 x 和 y 结点，形成如图 3.11(a)所示的 M'。

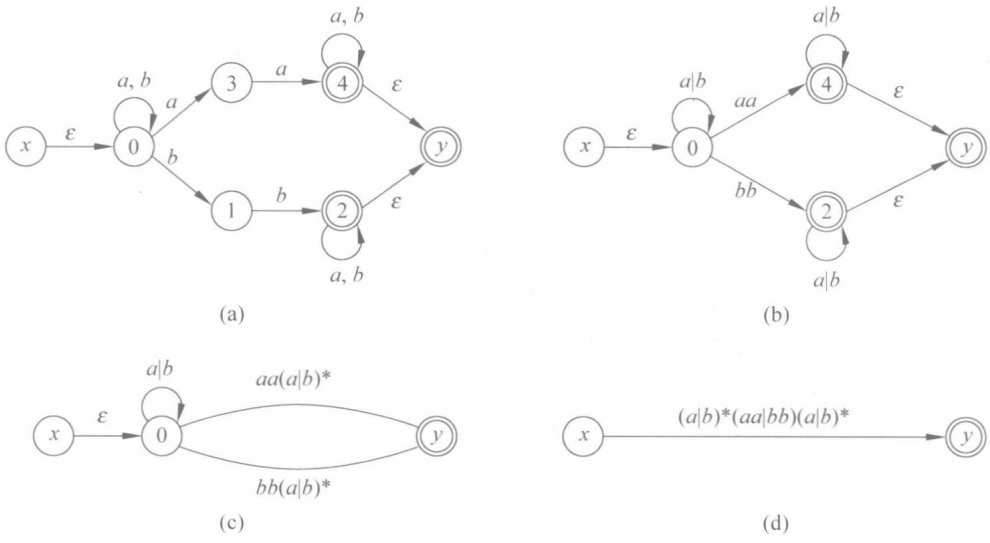


图 3.11　从 NFA M 构造正规式 r

第 2 步，逐步消去 M' 的结点，消去 1 和 3 之后如图 3.11(b)所示；再消去 2 和 4 后如图 3.11(c)所示；再消去 0 结点，最后只剩下 x 和 y 结点，如图 3.11(d)所示。

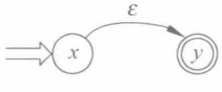
$r=(a|b)^*(aa|bb)(a|b)^*$ 即为所求。

下面介绍从 Σ 上的一个正规式 r 构造 Σ 上的一个 NFA M，使得 $L(M)=L(r)$ 的方法。这个方法称为“语法制导”的方法，即按正规式的语法结构指引构造过程，首先将正规式分解成一系列子表达式，然后使用如下规则为 r 构造 NFA，对 r 的各种语法结构的构造规则具体描述如下：

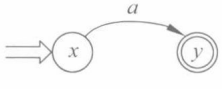
(1) 为 $\varnothing$、ε 和 a 构造 NFA。

① 对于正规式 $\varnothing$，所构造的 NFA 为

② 对于正规式 ε，所构造的 NFA 为

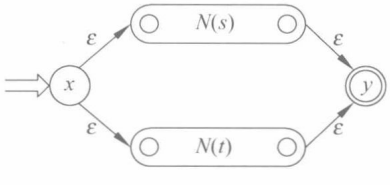


③ 对于正规式 a，$a\in\Sigma$，所构造的 NFA 为



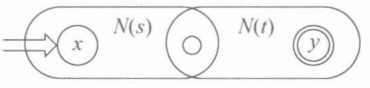
(2) 若 s，t 为 Σ 上的正规式，相应的 NFA 分别为 $N(s)$ 和 $N(t)$，分别为以下正规式构造 NFA。

① 对正规式 $r=s|t$，所构造的 NFA(r)为



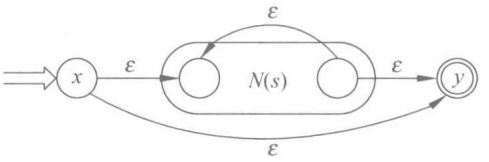
其中 x 是 NFA(r)的初态，y 是 NFA(r)的终态，x 到 $N(s)$和 $N(t)$的初态各有一个 ε 弧，从 $N(s)$和 $N(t)$的终态各有一个 ε 弧到 y，现在 $N(s)$和 $N(t)$的初态或终态已不作为 $N(r)$的初态和终态了。

② 对正规式 $r=st$，所构造的 NFA(r)为



其中 $N(s)$的初态成了 $N(r)$的初态，$N(t)$的终态成了 $N(r)$的终态。$N(s)$的终态与 $N(t)$的初态合并为 $N(r)$的一个既不是初态也不是终态的状态。

③ 对于正规式 $r=s^*$，NFA(r)为

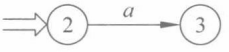


这里 x 和 y 分别是 NFA(r)的初态和终态，从 x 引 ε 弧到 $N(s)$的初态，从 $N(s)$的终态引 ε 弧到 y，从 x 到 y 引 ε 弧，同样 $N(s)$的终态可沿 ε 弧的边直接回到 $N(s)$的初态。$N(s)$的初态或终态不再是 $N(r)$的初态和终态。

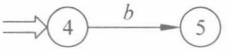
④ 正规式(s)的 NFA 同 s 的 NFA 一样。

例 3.11 为 $r=(a|b)^*abb$ 构造 NFA N，使得 $L(N)=L(r)$。

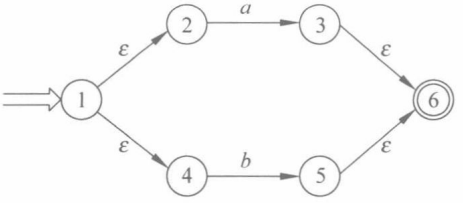
从左到右分解 r，令 $r_1=a$，第 1 个 a，则有



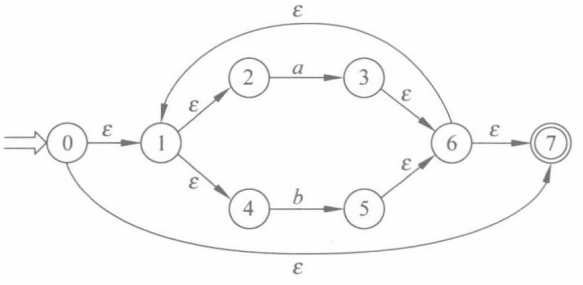
令 $r_2=b$，则有



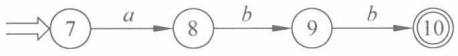
令 $r_3=r_1|r_2$，则有



令 $r_4=r_3^*$，则有



令 $r_5=a, r_6=b, r_7=b, r_8=r_5r_6, r_9=r_8r_7$，则有



令 $r_{10}=r_4r_9$，则最终得到图 3.6 的 NFA N 即为所求。

其实，分解 r 的方式很多，用图 3.12(a)～图 3.12(d)分别表明另一种分解方式和所构造的 NFA。

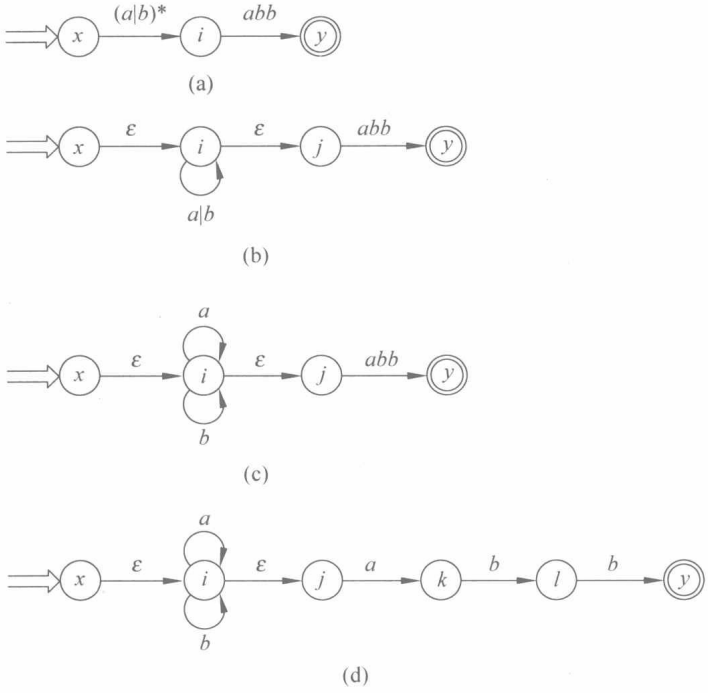


图 3.12　从正规式 r 构造 NFA

3.6　正规文法和有穷自动机的等价性

前面提到，正规集也常常使用正规文法描述，正规文法与有穷自动机有特殊关系，采用下面的规则可从正规文法 G 直接构造一个有穷自动机 NFA M；使得 $L(M)=L(G)$，说明如下：

(1) M 的字母表与 G 的终结符集相同。

(2) 为 G 中的每个非终结符生成 M 的一个状态(不妨取成相同的名字)，G 的开始符 S 是 M 的开始状态 S。

(3) 增加一个新状态 Z，作为 M 的终态。

(4) 对 G 中的形如 $A\to tB$ 的规则(其中 t 为终结符或 ε，A 和 B 为非终结符的产生式)，构造 M 的一个转换函数 $f(A,t)=B$。

(5) 对 G 中形如 $A\to t$ 的产生式，构造 M 的一个转换函数 $f(A,t)=Z$。

例 3.12　与文法 $G[S]$ 等价的 NFA M 如图 3.13 所示。

$G[S]$：$S \to aA$

$S \to bB$

$S \to \varepsilon$

$A \to aB$

$A \to bA$

$B \to aS$

$B \to bA$

$B \to \varepsilon$

尽管在编译程序的设计和构造中很少需要将有穷自动机转换成等价的正规文法，但本节仍对这个算法进行介绍，可以看到，转换规则非常简单。

对转换函数 $f(A,t)=B$，可写一个产生式：

$$A \to tB$$

对可接受状态 Z，增加一个产生式：

$$Z \to \varepsilon$$

此外，有穷自动机的初态对应文法开始符，有穷自动机的字母表为文法的终结符集。

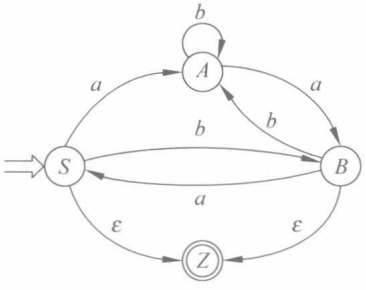


图 3.13　与 $G[S]$等价的 NFA M

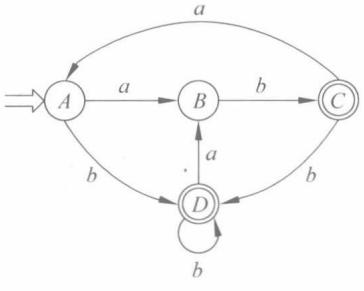


图 3.14　NFA

例 3.13　给出与图 3.14 的 NFA 等价的正规文法 G。

$G=(\{A,B,C,D\},\{a,b\},P,A)$，其中 P 为

$$A \to aB \quad C \to \varepsilon$$
$$A \to bD \quad D \to aB$$
$$B \to bC \quad D \to bD$$
$$C \to aA \quad D \to \varepsilon$$
$$C \to bD$$

3.7　词法分析程序的自动构造工具

在本章所介绍的形式模型——有限自动机、正规表达式以及正规文法基础上容易实现词法分析程序的自动构造。通常是用正规表达式或正规文法作为词法规则的形式描述，然后通过转化为等价的有限自动机来设计相应的单词识别过程。以正规表达式为例，典型的过程可能是：

(1) 每一种别的单词均对应一个正规表达式，所有正规表达式以文本方式作为自动构造工具的输入。

(2) 自动构造工具将每一个正规表达式转换成有限自动机的形式，比如使用 3.5 节中的方法将正规表达式转换成 NFA。

(3) 必要时，自动构造工具会将有限自动机确定化，比如使用 3.4.3 节中的方法得到等价的 DFA。

(4) 必要时，自动构造工具会将有限自动机最小化，比如使用 3.4.4 节中的方法得到等价的拥有状态数目最少的 DFA。

(5) 自动构造工具按照一定的控制策略生成词法分析程序中扫描程序的代码，该扫描程序可以选择对每一单词种别所对应的有限自动机进行模拟运行，并从当前输入符号序列中识别下一个单词，然后返回相应的单词符号。

通常，单词符号所采用的数据结构也需要由使用者来给定，连同每一单词种别对应的正规表达式一同作为自动构造工具的输入；单词符号中的单词种别一般会由使用者预先设定。另外，一些工具会按照描述的先后次序以及可识别单词的最大长度等来确定内部控制策略，这些约定通常也要明确告知使用者。

基于这种方法来构造词法分析程序的工具很多，本节主要介绍自动构造工具 lex[38]。一方面，使读者初步了解 lex 工具的使用；另一方面，有助于加深对词法分析程序自动构造的原理和过程的理解和认识。应用广泛的一个 lex 版本是 flex[39]。

lex 工具的功能是读入用户编写的一个 lex 描述文件，生成一个名为 lex.yy.c 的 C 源程序文件。lex.yy.c 中包含一个核心函数 yylex()，它是一个扫描子程序，读入源程序的字符流，识别并返回下一个单词符号，如图 3.15 所示。

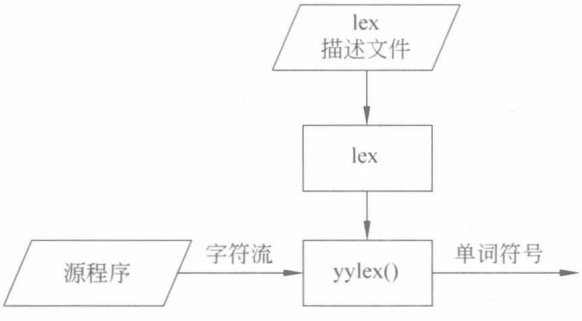


图 3.15　lex 简介

lex 描述文件中包含针对每一类词法单元的规则。规则由正规表达式和 C 语言代码两部分组成。

下面分几个小节简要介绍 lex 描述文件的格式和内容、lex 的使用、lex 和 yacc 的联用以及若干例子。

3.7.1　lex 描述文件中使用的正规表达式

lex 描述文件中，在书写词法单元的识别规则时，需要用到正规表达式。

在 3.3.2 节，介绍了基本的正规表达式。为方便使用，lex 中的正规表达式所允许的表达方式要丰富许多。以下列举了 lex 中主要的正规表达式表示形式：

- x，可以匹配字符 x。
- .，可以匹配除换行符"\n"之外的任何字符。
- 用方括号括起来的字符列表，可以匹配该字符列表中的所有字符。字符列表中除字符外还可以出现由间隔符'-'表示的字符范围。例如，[xyz]匹配字符 x，y 或 z。又如，[x-zA4-6O]匹配字符 x，y，z，A，4，5，6 或 O。除了\之外，其他元字符在方括号中没有特殊含义。若第一个字符是'-'，则不被当作元字符。
- 用[^和]括起来的字符列表，可以匹配该字符列表之外的所有字符。例如，[^A-Z]匹配所有除大写字母之外的字符。特别地，[^]可匹配任何字符。
- 用双引号"括起来一个串，可以匹配这个串本身。这个串里面的所有元字符，除了\和"之外，都会失去元字符的作用。例如，可用"if"匹配序列 if，可用"["匹配单个左方括号，可用"/*"匹配序列/*。
- 用大括号括起来的正规表达式宏名字，相当于将这个宏名字展开为相应的正规表达式。正规表达式宏名字的定义见 3.7.2 节。
- 除了加双引号"外，匹配单个元字符的另一种方法是利用转义字符 \ 。例如，*，可匹配一个字符 *；如果需要匹配序列*，就必须写作*。若\后面的字符是小写字母，则可能表示 C 语言转义字符，如\t 表示制表符。
- 反斜杠\后面跟八进制数值，而\x 后面跟十六进制数值，则匹配这个数值对应的 ASCII 字符。例如，\0 匹配 NUL 字符(ASCII 码为 0)，\123 匹配八进制数 123 对应的 ASCII 字符，\x2a 匹配八进制数 2a 对应的 ASCII 字符。
- $r*$，匹配正规表达式 r 的星闭包。
- $r+$，匹配正规表达式 r 的正闭包。
- $r?$，匹配正规表达式 r 的任选。
- $r\{n\}$，匹配正规表达式 r 的 n 次幂。
- $r\{m,n\}$，匹配正规表达式 r 的 m 到 n 次幂。
- $r\{m,\}$，匹配正规表达式 r 的大于等于 m 次幂。
- (r)，匹配正规表达式 r，括号用于重新规定优先级。
- rs，匹配正规表达式 r 与正规表达式 s 的连接。
- $r|s$，匹配正规表达式 r 与正规表达式 s 的并。
- r/s，匹配正规表达式 r，但仅限于随后的输入符号可以匹配正规表达式 s。要注意，在确定是否可以匹配 s 期间不管读入过多少个输入符号都将被退回。
- ^r，匹配正规表达式 r，但仅限于在一行的开始处。
- $r\$$，匹配正规表达式 r，但仅限于在一行的结尾处。
- $<c>r$，匹配正规表达式 r，但仅限于开始条件为 c。开始条件用来区分不同上下文，其定义见 3.7.2 节。c 也可以是一个开始条件的列表，或者是 *，后者用来表示任意的开始条件。

关于 lex 中正规表达式的正确使用还有不少技术细节的问题，本书限于篇幅不可能涉及所有细节，所以在实际应用中手头最好准备一份较详细的技术手册。

3.7.2 lex 描述文件的格式

lex 描述文件由 3 个部分组成，各部分之间被只含%%的行分隔开：

```
辅助定义部分
%%
规则部分
%%
用户子程序部分
```

其中，辅助定义部分、规则部分和用户子程序部分都是可选的，可以不出现。在没有用户子程序部分时，第二个%%也可省略。

辅助定义部分包含正规表达式宏名字的声明以及开始条件的声明。它们可能出现在规则部分的正规表达式中，用法见 3.7.1 节。

声明正规表达式宏名字的格式为

```
宏名字　正规表达式
```

例如：

```
DIGIT          [0-9]
NUMBER         {DIGIT}+"."{DIGIT}*
```

这样，正规表达式中若出现{NUMBER}，就相当于([0-9])+"."([0-9])* 。

开始条件的声明始于%Start(可缩写为%s 或%S)的行，后跟一个名字列表，每个名字代表一个开始条件。开始条件可以在规则的活动部分使用 BEGIN 来激活。直到下一个 BEGIN 执行时，拥有给定开始条件的规则被激活，而不拥有开始条件的规则变为不被激活。

开始条件主要用来区分不同上下文。限于篇幅，这里不打算给出有关开始条件的声明和使用的例子。

规则部分是描述文件的核心，一条规则由两部分组成：

```
正规表达式　动作
```

正规表达式的形式参见 3.7.1 节。正规表达式必须从第一列写起，而结束于第一个非转义的空白字符。这一行的剩余部分即为动作。动作必须从正规式所在行写起。当某条规则的动作超过一条语句时，必须用花括号括起来。如果动作部分为空，则匹配该正规表达式的输入字符流就会被直接丢弃。

输入字符流中不与任何规则中的正规表达式匹配的串默认为将被照抄到输出文件。如果不希望照抄输出，就要为每一个可能出现的词法单元提供规则。

例如，以下描述对应的程序将从输入流中删掉 "remove these characters"：

```
%%
"remove these characters"
```

又如，以下描述对应的程序将多个空白或 Tab 字符缩减为一个空白字符，同时滤掉每行行尾的所有空白或 Tab 字符：

```
%%
[\t]+          putchar(' ');
[ \t]+$        /* ignore this token */
```

动作可以是任意 C 语言代码，包括 return 语句，它在 yylex()被调时返回某个值。每一次调用 yylex()之后，将会从上一次离开的位置继续处理输入字符流，直到文件结束或执行

了一个 return 语句。

动作中可以用到 yytext、yyleng 等变量。其中，yytext 指向当前正被某规则匹配的字符串；yyleng 存储 yytext 中字符串的长度，被匹配的串在 yytext[0]～yytext[yyleng－1]中。

此外，动作中还允许包含特定的指导语句或函数，如 ECHO、BEGIN、REJECT、yymore()、yyless(n)、unput(c)、input()等。技术细节可参考有关 lex 的技术文档。

在辅助定义部分和规则部分，任何未从第一列开始的文本内容，以及被%{和%}括起来的部分，将被复制到 lex. yy. c 文件中(不包括%{})。注意，这里的%{必须从所在行的第一列开始。

在规则部分，出现在第一条规则之前的从第一列开始的或被%{和%}括起来的部分里可以声明扫描子程序 yylex()的局部变量，以及每次进入 yylex()时执行的代码。

在辅助定义部分中，第一列开始的注释(即始于/ * 的行)也将被复制，直到遇到下一个 * /。但规则部分中不可以这样。

最后，用户子程序部分中的调用扫描子程序或被扫描子程序调用的所有 C 语言函数将被原样照抄到 lex. yy. c 文件中。

值得提到的是，当遇到文件结尾时，词法分析程序将自动调用 yywrap()来确定下一步做什么。如果 yywrap()返回 0，那么就继续扫描；如果 yywrap()返回 1，那么就认为对输入串的处理已结束。lex 库中的 yywrap() 标准版本总是返回 1。用户可以根据需要在用户子程序部分写一个自己的 yywrap()，它将取代 lex 库中的版本。

例 3.14 分析由下列 lex 描述文件，说明由它产生的扫描子程序的功能。

```
%{
        int num_lines=0,num_chars=0;
%}
%%
\n      {++num_lines; ++num_chars;}
.       {++num_chars;}
%%
main(){
    yylex();
    printf("# of lines=%d, # of chars=%d\n",num_lines,num_chars );
}
```

解：首先，第 1 行到第 3 行都位于分隔符%{和%}之间，这些行将被直接插入到由 lex 产生的 C 语言代码中，它将位于任何过程的外部。第二行中定义了两个全局变量：行计数器 num_lines 和字符计数器 num_chars。

在第 4 行的%%之后，第 5、6 行描述了两个规则。在第一个规则中，正规表达式只包含一个换行符\n，对应的动作是行计数器 num_lines 加 1，以及字符计数器 num_chars 加 1。在第二个规则中，正规表达式是'.'，可以匹配除换行符\n 之外的任何字符，对应的动作是字符计数器 num_chars 加 1。

最后，在用户子程序部分中包括了一个 main 函数，它调用函数 yylex()，且输出行计数器 num_lines 和字符计数器 num_chars 的值。

由上述描述文件产生的扫描子程序的功能是统计并输出给定输入文本中的行数和字

符数。

3.7.3 lex 的使用

设例 3.14 中的 lex 描述文件的名字为 count.l。在 Linux 环境(假设安装了相应的开发包,并且设置了正确的环境变量)中,可以通过以下步骤编译和执行:

```
$ lex count.l
$ cc - o count lex.yy.c - ll
$ ./count < count.l
    ⋮
$
```

其中,$ 为系统提示符。

第一行命令执行后,将会产生文件 lex.yy.c。

第二行命令是用编译器 cc 对 lex.yy.c 进行编译。选项-o count 指定了可执行文件名为 count,不指定时默认为 a.out。-ll 是 lex 库文件的选项。

第三行是执行 count。输入参数是文件 count.l 中的文本。执行结果是输出文件 count.l 中文本的行数和字符数。

例 3.15 给定 lex 描述文件 toupper.l 如下:

```
%{
      #include <stdio.h>
}%
%%
[a-z]    Printf("%c",yytext[0]+'A'-'a')
%%
```

试指出正确执行如下命令序列后的输出结果:

```
$ lex toupper.l
$ cc -o toupper lex.yy.c -ll
$ ./toupper <toupper.l
```

解:输出结果为

```
%{
      #INCLUDE <STDIO.H>
}%
%%
[A-Z]    PRINTF("%C",YYTEXT[0]+'A'-'A')
%%
```

3.7.4 与 yacc 的接口约定

lex 的一个主要应用是与 yacc[34](一个语法分析程序的生成器,参见 7.3 节)的联用。yacc 产生的分析子程序在申请读入下一个单词时会调用 yylex()。yylex()返回一个单词符

号，并将相关的属性值存入全局量 yylval。

为了联用 lex 和 yacc，需要在运行 yacc 程序时加选项-d，以产生文件 y. tab. h，其中会包含在 yacc 描述文件中（由%tokens 定义）的所有单词种别。文件 y. tab. h 将被包含在 lex 描述文件中。

例如，如果有一个单词种别是 INTEGER，那么 lex 描述文件的一部分可能是

```
%{
#include "y.tab.h"
extern int yylval;
}%
%%
0|[1-9][0-9]*          { yylval=atoi(yytext); return INTEGER; }
[+*()\n]               { return yytext[0];}
.                      {  /* do nothing */ }
%%
```

lex 与 yacc 联用的具体例子可参见 7.3.2 节，届时会用到这个 lex 描述文件。

注意：在这个 lex 描述文件中，每个正规表达式之后的语义动作中均含有返回相应单词种别的 return 语句，而单词自身的值则通过全局量 yylval 或 yylex 进行传递。这是生成可供语法分析程序调用的词法分析子程序所需要的，即每调用一次返回下一单词符号，如图 3.1 所示。

练　习

1. 构造下列正规式相应的 DFA。

(1) $1(0|1)^*101$

(2) $1(1010^*|1(010)^*1)^*0$

(3) $a((a|b)^*|ab^*a)^*b$

(4) $b((ab)^*|bb)^*ab$

2. 已知 NFA=$(\{x,y,z\},\{0,1\},M,\{x\},\{z\})$ 其中：

$M(x,0)=\{z\}$，$M(y,0)=\{x,y\}$，$M(z,0)=\{x,z\}$，$M(x,1)=\{x\}$，$M(y,1)=\varnothing$，$M(z,1)=\{y\}$，构造相应的 DFA。

3. 将图 3.16 中的 NFA 确定化。

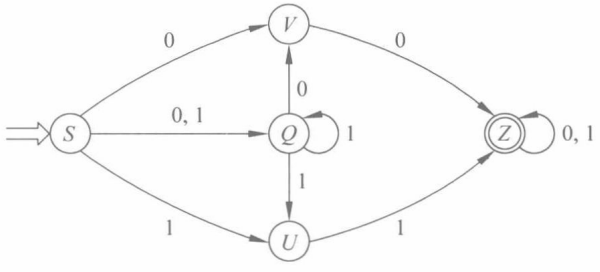
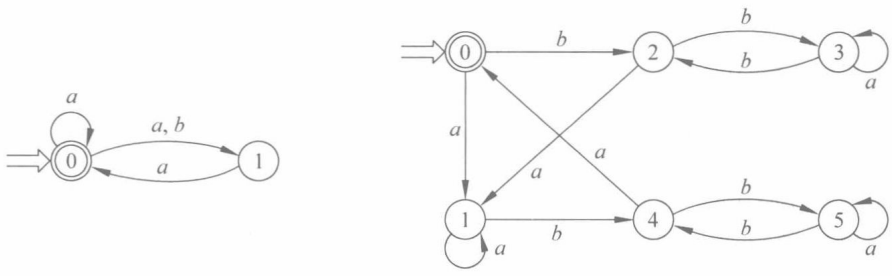


图 3.16　NFA

4. 把图 3.17(a)和(b)中的 NFA 分别确定化和最小化。



(a) (b)

图 3.17 NFA

5. 构造一个 DFA,它接受 $\Sigma=\{0,1\}$上所有满足如下条件的字符串:每个 1 都有 0 直接跟在右边。然后构造该语言的正规文法。

6. 设无符号数的正规式为 θ:

$$\theta = dd^* \mid dd^*.dd^* \mid .dd^* \mid dd^*10(s \mid \varepsilon)dd^*$$
$$\mid 10(s \mid \varepsilon)dd^* \mid .dd^*10(s \mid \varepsilon)dd^*$$
$$\mid dd^*.dd^*10(s \mid \varepsilon)dd^*$$

化简 θ,画出 θ 的 DFA,其中 $d=\{0,1,2,\cdots,9\}$,$s=\{+,-\}$

7. 为正规文法 $G[S]$

$S \to aA \mid bQ$

$A \to aA \mid bB \mid b$

$B \to bD \mid aQ$

$Q \to aQ \mid bD \mid b$

$D \to bB \mid aA$

$E \to aB \mid bF$

$F \to bD \mid aE \mid b$

构造相应的最小的 DFA。

8. 给出下述正规文法所对应的正规式:

$$S \to 0A \mid 1B$$
$$A \to 1S \mid 1$$
$$B \to 0S \mid 0$$

9. 将图 3.18 的 DFA 最小化,并用正规式描述它所识别的语言。

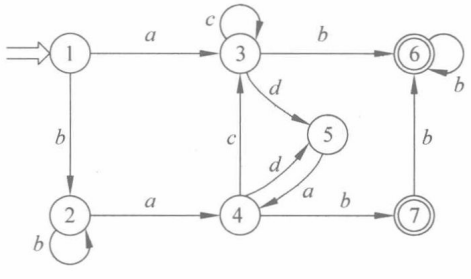


图 3.18 DFA

10. 构造下述文法 $G[S]$的自动机:

$S \to A0$

$A \to A0 \mid S1 \mid 0$

该自动机是确定的吗?若不确定,则对它确定化。该自动机相应的语言是什么?

说明:产生式形式为 $A \to a$ 或 $A \to Ba$,$B,A \in V_N$,$a \in V_T^*$ 的文法也是正规文法,并称为

左线性文法。为左线性文法 $G[S]$构造 NFA M 的规则如下：

(1) 字母表与 G 的终结符集相同。

(2) G 中的每个非终结符生成 M 的一个状态。

(3) G 的开始符对应 M 的终态。

(4) 增加一个新的状态 F，作为 M 的初态。

(5) 对 G 中的形如 $A \rightarrow Ba$ 的产生式，构造 M 的转换函数 $f(B,a)=A$；对 $A \rightarrow a$，构造 $f(F,a)=A$。

11. 有一种用以证明两个正规表达式等价的方法，那就是构造它们的最小 DFA，表明这两个 DFA 是一样的(除了状态名不同外)。使用此方法。证明下面的正规表达式是等价的。

(1) $(a|b)^*$

(2) $(a^*|b^*)^*$

(3) $((\varepsilon|a)b^*)^*$

12. 文法 $G[\langle$单词$\rangle]$为

〈单词〉→〈标识符〉|〈整数〉

〈标识符〉→〈标识符〉〈字母〉|〈标识符〉〈数字〉|〈字母〉

〈整数〉→〈整数〉〈数字〉|〈数字〉

〈字母〉→$A|B|\cdots|Y|Z$

〈数字〉→0|1|2|…|8|9

(1) 改写 G 为 G'，使 G'为与 G 等价的正规文法。

(2) 给出相应的有穷自动机。

13. 有如下 lex 描述文件的识别规则部分，请指出输入特定串后输出是什么。

```
%%
[0-9A-Fa-f]+H { printf ("Number "); }
[A-Za-z][A-Za-z0-9] * { printf ("Identifier "); }
"LET" { printf ("Keyword "); }
"=" { printf ("Operator "); }
. {}
%%
```

其中输入的串是“LET Something01=DeadBeefH”。

14. 考虑如下 lex 描述文件的识别规则部分：

```
%%
ab printf("1 %s\n",yytext);
b * a * c? printf("2 %s\n",yytext);
ba. a * printf("3 %s\n",yytext);
aa * c printf("4 %s\n",yytext);
ab * c? printf("5 %s\n",yytext);
%%
```

提示：yytext 是当前匹配的内容。

(1) 下面的输入串对应的输出是什么?

abbacbababcab

(2) 上面的规则有可以被删除且不改变词法分析器行为的吗? 如果有,请找出可以被删除的规则;如果没有,请解释。

15. 参考 3.2 节,对附录 A 中 PL/0 编译器源码进行裁减,或者对 getsym 及其相关代码进行重新包装,实现一个 PL/0 语言的独立词法分析程序。该词法分析程序读入 PL/0 语言源程序,输出一个单词符号的序列。对于标识符和无符号整数,显示单词种别和单词自身的值两项内容;对于其他单词符号,仅显示其单词种别。

第 4 章　自顶向下语法分析方法

语法分析是编译程序的核心部分。语法分析的作用是识别由词法分析给出的单词符号串是否是给定文法的正确句子(程序)。语法分析常用的方法可分为自顶向下分析和自底向上分析两大类。虽然语法分析可以通过确定分析或者不确定分析来实现,但在实际的编译器构造中,几乎都是采用确定分析方式,不确定分析仅具有理论价值。本章主要介绍自顶向下的确定分析。在第 5、6 章中,分别介绍两种确定的自底向上分析方法:算符优先分析[①]和 LR 分析。这些分析方法各有优缺点,但都是迄今编译程序构造的实用方法。

自顶向下分析方法也称面向目标的分析方法,也就是从文法的开始符号出发企图推导出与输入的单词符号串完全相匹配的句子,若输入串是给定文法的句子,则必能推出,反之必然出错。自顶向下的确定分析方法需对文法有一定的限制,但由于实现方法简单、直观,便于手工构造或自动生成语法分析器,因而仍是目前常用的方法之一。而自顶向下的不确定分析方法是带回溯的分析方法,这种方法实际上是一种穷举的试探方法,效率低,代价高,因而极少使用,仅在 4.4 节中粗略介绍。

4.1　确定的自顶向下分析思想

确定的自顶向下分析方法,是从文法的开始符号出发,考虑如何根据当前的输入符号(单词符号)唯一地确定选用哪个产生式替换相应非终结符以往下推导,或如何构造一棵相应的语法树。现举例说明。

例 4.1　有文法 $G1[S]$:

$S \rightarrow pA \mid qB$

$A \rightarrow cAd \mid a$

$B \rightarrow dB \mid b$

若输入串 $W = pccadd$。自顶向下的推导过程为 $S \Rightarrow pA \Rightarrow pcAd \Rightarrow pccAdd \Rightarrow pccadd$,相应的语法树为图 4.1。

这个文法有以下两个特点:

(1) 每个产生式的右部都由终结符号开始。

(2) 如果两个产生式有相同的左部,那么它们的右部由不同的终结符开始。

对于这样的文法,显然在推导过程中完全可以根据当前的输入符号决定选择哪个产生式往下推导,因此分析过程是唯一确定的。再看一个例子。

例 4.2　若有文法 $G2[S]$:

$S \rightarrow Ap$

$S \rightarrow Bq$

---

① 算符优先分析方法适用范围较小,可以根据课时等实际情况进行取舍。

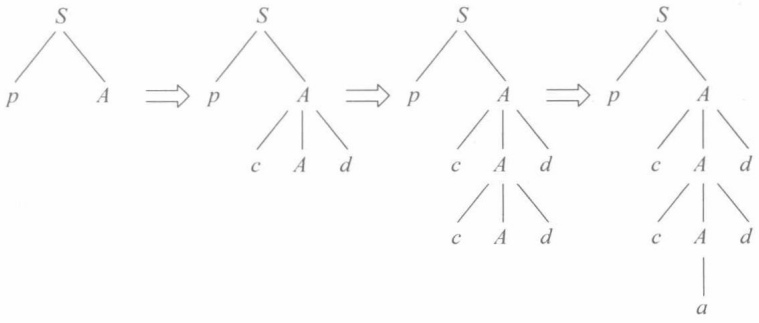


图 4.1　确定的自顶向下语法分析树(一)

$A \to a$

$A \to cA$

$B \to b$

$B \to dB$

当输入串 $W=ccap$，则推导过程为 $S \Rightarrow Ap \Rightarrow cAp \Rightarrow ccAp \Rightarrow ccap$，构造相应语法树如图 4.2 所示。

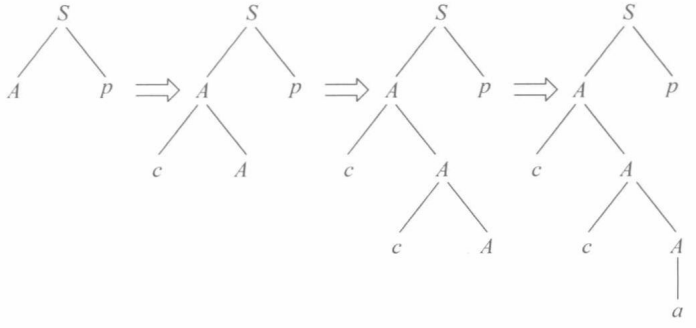


图 4.2　确定的自顶向下语法分析树(二)

这说明 $ccap$ 是例 4.2 文法的句子。

例 4.2 文法的特点如下：

(1) 产生式的右部不全是由终结符开始。

(2) 如果两个产生式有相同的左部，它们的右部是由不同的终结符或非终结符开始。

(3) 文法中无空产生式。

对于产生式中相同左部含有非终结符开始的产生式时，在推导过程中选用哪个产生式不像例 4.1 文法那样直观，对于 $W=ccap$ 为输入串时，其第一个符号是 c，这时从 S 出发选择 $S \to Ap$ 还是选择 $S \to Bq$，就需要知道是从 Ap 还是从 Bq 能推出 $c\alpha(\alpha \in V)$ 形式，若当且仅当从 Ap 能推出 $c\alpha$，则选 $S \to Ap$ 进行推导；若当且仅当 Bq 能推出 $c\alpha$，则选 $S \to Bq$ 进行推导。为方便考察，作如下定义。

定义 4.1　设 $G=(V_T, V_N, P, S)$ 是上下文无关文法。

$$\mathrm{FIRST}(\alpha)=\{a \mid \alpha \overset{*}{\Rightarrow} a\beta, \quad a \in V_T, \alpha, \beta \in V^*\}$$

若 $\alpha \overset{*}{\Rightarrow} \varepsilon$，则规定 $\varepsilon \in \mathrm{FIRST}(\alpha)$。称 $\mathrm{FIRST}(\alpha)$ 为 α 的**开始符号集**或**首符号集**。

不难求出在文法 $G2$ 中：

$\mathrm{FIRST}(Ap)=\{a,c\}$

$\mathrm{FIRST}(Bq)=\{b,d\}$

这样，在文法 $G2$ 中，关于 S 的两个产生式的右部虽然都以非终结符开始，但它们右部的符号串可以推导出的开始符号集不相交，因而可以根据当前的输入符号是属于哪个产生式右部的开始符号集而决定选择相应产生式进行推导。这样仍能构造确定的自顶向下分析。

在文法 $G1$、$G2$ 中都不包含空产生式，处理比较直观简单。下面考虑当文法中有空产生式时的情况，先看例子。

例 4.3 有文法 $G[S]$：

$S\rightarrow aA$

$S\rightarrow d$

$A\rightarrow bAS$

$A\rightarrow\varepsilon$

若输入串 $W=abd$，则试图推导出 abd 串的推导过程为 $S\Rightarrow aA\Rightarrow abAS\Rightarrow abS\Rightarrow abd$，相应语法树为图 4.3。

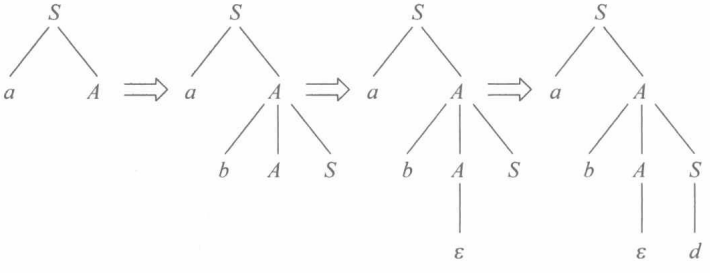


图 4.3 确定的自顶向下语法分析树(三)

从以上推导过程中可以看到，在第 2 步到第 3 步的推导中，即 $abAS\Rightarrow abS$ 时，因当前面临的输入符号为 d，而最左非终结符 A 的产生式右部的开始符号集都不包含 d，但有 ε，因此对于 d 的匹配自然认为只能依赖于在可能的推导过程中 A 的后面的符号，所以这时选用产生式 $A\rightarrow\varepsilon$ 往下推导，而当前 A 后面的符号为 S，S 产生式右部的开始符号集包含了 d，所以例中可用 $S\rightarrow d$ 推导得到匹配。

由此可以看出，当某一非终结符的产生式中含有空产生式时，它的非空产生式右部的开始符号集两两不相交，并与在推导过程中紧跟该非终结符右边可能出现的终结符集也不相交，则仍可构造确定的自顶向下分析，为此，定义一个文法符号的**后跟符号**的集合如下。

定义 4.2 设 $G=(V_T,V_N,P,S)$ 是上下文无关文法，$A\in V_N$，S 是开始符号

$$\mathrm{FOLLOW}(A)=\{a\mid S\overset{*}{\Rightarrow}\mu A\beta \text{ 且 } a\in V_T,a\in\mathrm{FIRST}(\beta),\mu\in V_T^*,\beta\in V^+\}$$

若 $S\overset{*}{\Rightarrow}\mu A\beta$，且 $\beta\overset{*}{\Rightarrow}\varepsilon$，则 $\#\in\mathrm{FOLLOW}(A)$。

也可定义为

$$\mathrm{FOLLOW}(A)=\{a\mid S\overset{*}{\Rightarrow}\cdots Aa\cdots,a\in V_T\}$$

若有 $S \overset{*}{\Rightarrow} \cdots A$，则规定 $\# \in \mathrm{FOLLOW}(A)$。

这里用 # 作为输入串的结束符，也称输入串括号。

因此当文法中含有形如

$$A \rightarrow \alpha$$
$$A \rightarrow \beta$$

的产生式时，其中 $A \in V_N$，α、$\beta \in V^*$，若 α 和 β 不能同时推导出空，假定 $\alpha \overset{*}{\Rightarrow} \varepsilon$，$\beta \overset{*}{\Rightarrow} \varepsilon$，则当 $\mathrm{FIRST}(\alpha) \cap (\mathrm{FIRST}(\beta) \cup \mathrm{FOLLOW}(A)) = \varnothing$ 时，对于非终结符 A 的替换仍可唯一地确定候选。

定义 4.3 一个产生式的选择符号集 SELECT。给定上下文无关文法的产生式 $A \rightarrow \alpha$，$A \in V_N$，$\alpha \in V^*$，若 $\alpha \overset{*}{\Rightarrow} \varepsilon$，则 $\mathrm{SELECT}(A \rightarrow \alpha) = \mathrm{FIRST}(\alpha)$。

如果 $\alpha \overset{*}{\Rightarrow} \varepsilon$，则 $\mathrm{SELECT}(A \rightarrow \alpha) = (\mathrm{FIRST}(\alpha) - \{\varepsilon\}) \cup \mathrm{FOLLOW}(A)$。

定义 4.4 一个上下文无关文法是 LL(1) 文法的充分必要条件是，对每个非终结符 A 的两个不同产生式，$A \rightarrow \alpha$，$A \rightarrow \beta$，满足

$$\mathrm{SELECT}(A \rightarrow \alpha) \cap \mathrm{SELECT}(A \rightarrow \beta) = \varnothing$$

其中 α、β 不同时能 $\overset{*}{\Rightarrow} \varepsilon$。

根据前面的讨论容易看出，LL(1) 文法是能够使用确定的自顶向下分析技术的。

LL(1) 的含义是：第 1 个 L 表明自顶向下分析是从左向右扫描输入串，第 2 个 L 表明分析过程中将用最左推导，1 表明只需向右看一个符号便可决定如何推导，即选择哪个产生式（规则）进行推导。类似地，也可以有 LL(k) 文法，也就是需向前查看 k 个符号才可确定选用哪个产生式。通常采用 $k=1$，个别情况采用 $k=2$。

回顾例 4.3 的文法：

$S \rightarrow aA$

$S \rightarrow d$

$A \rightarrow bAS$

$A \rightarrow \varepsilon$

不难看出：

$\mathrm{SELECT}(S \rightarrow aA) = \{a\}$

$\mathrm{SELECT}(S \rightarrow d) = \{d\}$

$\mathrm{SELECT}(A \rightarrow bAS) = \{b\}$

$\mathrm{SELECT}(A \rightarrow \varepsilon) = \{a, d, \#\}$

所以

$\mathrm{SELECT}(S \rightarrow aA) \cap \mathrm{SELECT}(S \rightarrow d) = \{a\} \cap \{d\} = \varnothing$

$\mathrm{SELECT}(A \rightarrow bAS) \cap \mathrm{SELECT}(A \rightarrow \varepsilon) = \{b\} \cap \{a, d, \#\} = \varnothing$

由定义 4.4 知例 4.3 文法是 LL(1) 文法，所以可用确定的自顶向下分析。

例 4.4 文法 $G[S]$ 为

$S \rightarrow aAS$

$S \rightarrow b$

$A \rightarrow bA$

$A \rightarrow \varepsilon$

则

$\mathrm{SELECT}(S \rightarrow aAS) = \{a\}$

$\mathrm{SELECT}(S \rightarrow b) = \{b\}$

$\mathrm{SELECT}(A \rightarrow bA) = \{b\}$

$\mathrm{SELECT}(A \rightarrow \varepsilon) = \{a, b\}$

所以

$\mathrm{SELECT}(S \rightarrow aAS) \cap \mathrm{SELECT}(S \rightarrow b) = \{a\} \cap \{b\} = \varnothing$

$\mathrm{SELECT}(A \rightarrow bA) \cap \mathrm{SELECT}(A \rightarrow \varepsilon) = \{b\} \cap \{a, b\} \neq \varnothing$

因此，例 4.4 的文法不是 LL(1)文法，因而也就不可能用确定的自顶向下分析。下面分析输入串 $W = ab$ 的两种不同推导过程。第一种推导过程为 $S \Rightarrow aAS \Rightarrow abAS \Rightarrow abS$，在句型 abS 中由于 S 不能推出 ε，所以这种推导过程推不出 ab；第二种推导过程为 $S \Rightarrow aAS \Rightarrow aS \Rightarrow ab$，这种推导过程推出了 ab。在上述两种推导过程中，第 1 个输入符 a 的匹配都用了产生式 $S \rightarrow aAS$，得到句型 aAS。这时按最左推导需用 A 的产生式右部替换 A，而关于 A 的产生式有两个不同的右部，即有两个候选。当前的输入符号为 b，第一种推导过程认为产生式 $A \rightarrow bA$ 的右部开始符号为 b，所以可用 bA 替换 A，使 b 得到匹配；第二种推导过程认为 A 的后跟符集合中含有 b，所以用产生式 $A \rightarrow \varepsilon$ 进行推导，用 ε 替换了 A，得到句型 aS，符号 b 由 S 往下推导去匹配，而关于 S 的产生式恰有 $S \rightarrow b$，所以用它推导 b 得到匹配。

以上两种推导过程中，当第 2 步推导时当前输入符为 b，对句型 aAS 中的 A 用哪个产生式推导不能唯一确定，因此导致了这个文法不能构造确定的自顶向下分析。

4.2 LL(1)文法的判别

当需要选用自顶向下分析技术时，必须判别所给文法是否是 LL(1)文法。因而对任给文法需首先计算 FIRST、FOLLOW、SELECT 集合，进而判别文法是否为 LL(1)文法。

在下面的讨论中假定所给文法是经过压缩的，即文法中不包含多余规则和有害规则。现举例说明判断 LL(1)文法的步骤。

例 4.5 文法 $G[S]$为

$S \rightarrow AB$

$S \rightarrow bC$

$A \rightarrow \varepsilon$

$A \rightarrow b$

$B \rightarrow \varepsilon$

$B \rightarrow aD$

$C \rightarrow AD$

$C \rightarrow b$

$D \rightarrow aS$

$D \rightarrow c$

判别步骤如下。

第一步，求出能推出 ε 的非终结符。

首先建立一个以文法的非终结符个数为上界的一维数组，其数组元素为非终结符，对应每一非终结符有一个标志位，用以记录能否推出 ε。其值有 3 种情况："未定"、"是"、"否"。

例 4.5 所对应数组 $X[]$的内容如表 4.1 所示。

表 4.1　非终结符能否推出空串的数组

非终结符	S	A	B	C	D
初值	未定	未定	未定	未定	未定
第 1 次扫描		是	是		否
第 2 次扫描	是			否	

计算能推出 ε 的非终结符步骤如下：

(1) 将数组 $X[]$中对应每一非终结符的标记置初值为"未定"。

(2) 扫描文法中的产生式。

① 删除所有右部含有终结符的产生式，若这使得以某一非终结符为左部的所有产生式都被删除，则将数组中对应该非终结符的标记值改为"否"，说明该非终结符不能推出 ε。

② 若某一非终结符的某一产生式右部为 ε，则将数组中对应该非终结符的标志置为"是"，并从文法中删除该非终结符的所有产生式。本例中对应非终结符 A、B 的标志改为"是"。

(3) 扫描产生式右部的每一符号。

① 若所扫描到的非终结符在数组中对应的标志是"是"，则删去该非终结符；若这使产生式右部为空，则将产生式左部的非终结符在数组中对应的标志改为"是"，并删除以该非终结符为左部的所有产生式。

② 若所扫描到的非终结符号在数组中对应的标志是"否"，则删去该产生式；若这使产生式左部非终结符的有关产生式都被删去，则把在数组中该非终结符对应的标志改成"否"。

(4) 重复(3)，直到扫描完一遍文法的产生式，数组中非终结符对应的特征再没有改变为止。

由(2)中①得知例中对应非终结符 D 的标志改为"否"。

经过上述(2)中①、②两步后，文法中的产生式只剩下 $S\to AB$ 和 $C\to AD$，也就是只剩下右部全是非终结符串的产生式。

再由(3)中的①步扫描到产生式 $S\to AB$ 时，在数组中 A、B 对应的标志都为"是"，删去后 S 的右部变为空，所以 S 对应标志置为"是"。

最后由(3)中的②扫描到产生式 $C\to AD$ 时，其中，A 对应的标志为"是"，D 对应的标志为"否"，删去该产生式后，再无左部为 C 的产生式，所以 C 的对应标志改成"否"。

第二步，计算 FIRST 集。

(1) 根据 FIRST 集定义对每一文法符号 $X\in V$ 计算 FIRST(X)。

① 若 $X\in V_T$，则 FIRST(X)=$\{X\}$。

② 若 $X\in V_N$，且有产生式 $X\to a\cdots$，$a\in V_T$，则 $a\in$FIRST(X)。

③ 若 $X\in V_N$，$X\to\varepsilon$，则 $\varepsilon\in$FIRST(X)。

④ 若 $X,Y_1,Y_2,\cdots,Y_n\in V_N$，而有产生式 $X\to Y_1Y_2\cdots Y_n$。当 $Y_1,Y_2,\cdots,Y_{i-1}\overset{*}{\Rightarrow}\varepsilon$ 时(其中 $1\leqslant i\leqslant n$)，则 $\mathrm{FIRST}(Y_1)-\{\varepsilon\},\mathrm{FIRST}(Y_2)-\{\varepsilon\},\cdots,\mathrm{FIRST}(Y_{i-1})-\{\varepsilon\},\mathrm{FIRST}(Y_i)$都包含在 $\mathrm{FIRST}(X)$中。

⑤ 当④中所有 $Y_i\overset{*}{\Rightarrow}\varepsilon,(i=1,2,\cdots,n)$，则 $\mathrm{FIRST}(X)=\mathrm{FIRST}(Y_1)\cup\mathrm{FIRST}(Y_2)\cup\cdots\cup\mathrm{FIRST}(Y_n)\cup\{\varepsilon\}$。

反复使用上述②～⑤步，直到每个符号的 FIRST 集合不再增大为止。

求出每个文法符号的 FIRST 集合后，也就不难求出一个符号串的 FIRST 集合。

若符号串 $\alpha\in V^*,\alpha=X_1X_2\cdots X_n$，当 X_1 不能$\overset{*}{\Rightarrow}\varepsilon$，则置 $\mathrm{FIRST}(\alpha)=\mathrm{FIRST}(X_1)$。

若对任何 $j(1\leqslant j\leqslant i-1,2\leqslant i\leqslant n),\varepsilon\in\mathrm{FIRST}(X_j),\varepsilon\notin\mathrm{FIRST}(X_i)$，则

$$\mathrm{FIRST}(\alpha)=\bigcup_{j=1}^{i-1}(\mathrm{FIRST}(X_j)-\{\varepsilon\})\cup\mathrm{FIRST}(X_i)$$

当对任何 $j(1\leqslant j\leqslant n)$，$\mathrm{FIRST}(X_j)$都含有 ε 时，则

$$\mathrm{FIRST}(\alpha)=\bigcup_{j=1}^{i-1}(\mathrm{FIRST}(X_j))\cup\{\varepsilon\}$$

由此算法可计算出例 4.5 文法各非终结符的 FIRST 集。

$$\mathrm{FIRST}(S)=(\mathrm{FIRST}(A)-\{\varepsilon\})\cup(\mathrm{FIRST}(B)-\{\varepsilon\})\cup\{\varepsilon\}\cup\{b\}=\{b,a,\varepsilon\}$$

$$\mathrm{FIRST}(A)=\{b\}\cup\{\varepsilon\}=\{b,\varepsilon\}$$

$$\mathrm{FIRST}(B)=\{\varepsilon\}\cup\{a\}=\{a,\varepsilon\}$$

$$\mathrm{FIRST}(C)=(\mathrm{FIRST}(A)-\{\varepsilon\})\cup\mathrm{FIRST}(D)\cup\mathrm{FIRST}(b)=\{b,a,c\}$$

$$\mathrm{FIRST}(D)=\{a\}\cup\{c\}=\{a,c\}$$

所以最终求得：

$$\mathrm{FIRST}(S)=\{a,b,\varepsilon\}$$
$$\mathrm{FIRST}(A)=\{b,\varepsilon\}$$
$$\mathrm{FIRST}(B)=\{a,\varepsilon\}$$
$$\mathrm{FIRST}(C)=\{a,b,c\}$$
$$\mathrm{FIRST}(D)=\{a,c\}$$

每个产生式的右部符号串的开始符号集合为

$$\mathrm{FIRST}(AB)=\{a,b,\varepsilon\}$$
$$\mathrm{FIRST}(bC)=\{b\}$$
$$\mathrm{FIRST}(\varepsilon)=\{\varepsilon\}$$
$$\mathrm{FIRST}(b)=\{b\}$$
$$\mathrm{FIRST}(aD)=\{a\}$$
$$\mathrm{FIRST}(AD)=\{a,b,c\}$$
$$\mathrm{FIRST}(b)=\{b\}$$
$$\mathrm{FIRST}(aS)=\{a\}$$
$$\mathrm{FIRST}(c)=\{c\}$$

(2) 由关系图法求文法符号的 FIRST 集。

① 每个文法符号对应图中一个结点，对应终结符的结点时用符号本身标记，对应非终

结符的结点时用 FIRST(A)标记。这里 A 表示非终结符。

② 如果文法中有产生式 $A\to\alpha X\beta$,且 $\alpha\overset{*}{\Rightarrow}\varepsilon$,则从对应 A 的结点到对应 X 的结点连一条箭弧。

③ 凡是从 FIRST(A)结点有路径可到达的终结符结点所标记的终结符都为 FIRST(A)的成员。

④ 根据判别步骤的第一步确定 ε 是否为某非终结符 FIRST 集的成员,若是,则将 ε 加入该非终结符的 FIRST 集中。

以例 4.5 文法为例计算 FIRST 集的关系图,如图 4.4 所示。

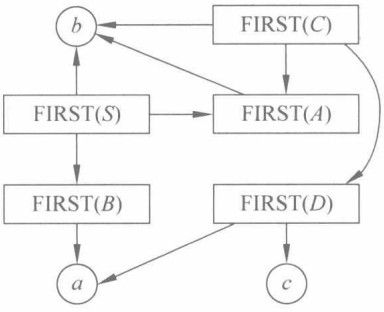


图 4.4 计算 FIRST 集的关系图

由关系图法求得例 5.5 文法非终结符的 FIRST 集结果如下:

$$\text{FIRST}(S)=\{b,a,\varepsilon\}$$
$$\text{FIRST}(A)=\{b,\varepsilon\}$$
$$\text{FIRST}(B)=\{a,\varepsilon\}$$
$$\text{FIRST}(C)=\{a,b,c\}$$
$$\text{FIRST}(D)=\{a,c\}$$

与根据定义求得的结果相同。注意,请读者自己考虑,为什么不能把 ε 结点画在关系图中?

第三步,计算 FOLLOW 集。

(1) 对文法中每一个 $A\in V_N$,根据定义计算 FOLLOW(A)。

① 设 S 为文法的开始符号,把{#}加入 FOLLOW(S)中(这里 # 为句子括号)。

② 若 $A\to\alpha B\beta$ 是一个产生式,则把 FIRST(β)的非空元素加入 FOLLOW(B)中。

如果 $\beta\overset{*}{\Rightarrow}\varepsilon$,则把 FOLLOW($A$)也加入 FOLLOW($B$)中,因为当有形如

$$D\to\alpha_1 A\beta_1$$
$$A\to\alpha B\beta$$

的产生式时,$A,B,D\in V_N,\alpha,\alpha_1,\beta,\beta_1\in V^*$,在推导过程中可能出现如下的句型序列:

$$S\overset{*}{\Rightarrow}\cdots\alpha_1 A\beta_1\cdots\Rightarrow\cdots\alpha_1\alpha B\beta\beta_1\cdots\Rightarrow\cdots\alpha_1\alpha B\beta_1\cdots$$

由定义 4.2 可知,FIRST(β_1)$\in$FOLLOW(A)必有 FIRST(β_1)$\in$FOLLOW(B)。

故 FOLLOW(A)$\subseteq$FOLLOW(B)。

③ 反复使用②直到每个非终结符的 FOLLOW 集不再增大为止。

现在计算例 4.5 文法各非终结符的 FOLLOW 集。

$$\text{FOLLOW}(S)=\{\#\}\cup\text{FOLLOW}(D)$$
$$\text{FOLLOW}(A)=(\text{FIRST}(B)-\{\varepsilon\})\cup\text{FOLLOW}(S)\cup\text{FIRST}(D)$$
$$\text{FOLLOW}(B)=\text{FOLLOW}(S)$$
$$\text{FOLLOW}(C)=\text{FOLLOW}(S)$$
$$\text{FOLLOW}(D)=\text{FOLLOW}(B)\cup\text{FOLLOW}(C)$$

由以上最终计算结果得:

$$\text{FOLLOW}(S)=\{\#\}$$

$$\text{FOLLOW}(A) = \{a, \#, c\}$$
$$\text{FOLLOW}(B) = \{\#\}$$
$$\text{FOLLOW}(C) = \{\#\}$$
$$\text{FOLLOW}(D) = \{\#\}$$

(2) 用关系图法求非终结符的 FOLLOW 集。

① 文法 G 中的每个符号和 # 对应图中的一个结点，对应终结符和 # 的结点用符号本身标记。对应非终结符的结点(如 $A \in V_N$)则用 FOLLOW(A)或 FIRST(A)标记。

② 从开始符号 S 的 FOLLOW(S)结点到 # 号的结点连一条箭弧。

③ 如果文法中有产生式 $A \to \alpha B \beta X$，且 $\beta \overset{*}{\Rightarrow} \varepsilon$，则从 FOLLOW($B$)结点到 FIRST($X$)结点连一条箭弧，当 $X \in V_T$ 时，则与 X 相连。

④ 如果文法中有产生式 $A \to \alpha B \beta$ 且 $\beta \overset{*}{\Rightarrow} \varepsilon$，则从 FOLLOW($B$)结点到 FOLLOW($A$)结点连一条箭弧。

⑤ 对每一个 FIRST(A)结点，如果有产生式 $A \to \alpha X \beta$，且 $\alpha \overset{*}{\Rightarrow} \varepsilon$，则从 FIRST($A$)到 FIRST($X$)连一条箭弧。

⑥ 凡是从 FOLLOW(A)结点有路径可以到达的终结符或 # 号的结点，其所标记的终结符或 # 号即为 FOLLOW(A)的成员。

现在对例 4.5 文法用关系图法计算 FOLLOW 集，如图 4.5 所示。

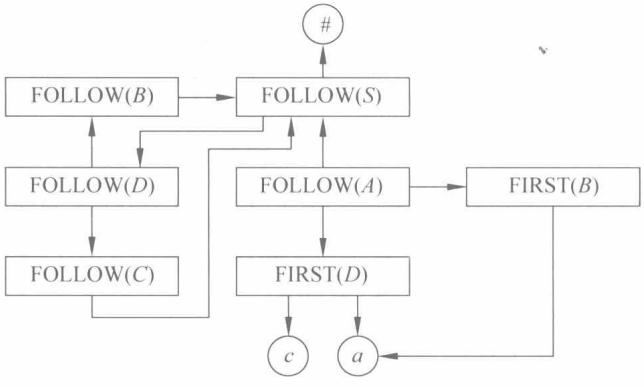


图 4.5　计算 FOLLOW 集的关系图

则得

$$\text{FOLLOW}(S) = \{\#\}$$
$$\text{FOLLOW}(A) = \{a, c, \#\}$$
$$\text{FOLLOW}(B) = \{\#\}$$
$$\text{FOLLOW}(C) = \{\#\}$$
$$\text{FOLLOW}(D) = \{\#\}$$

与根据定义计算的结果相同。

此外，对文法符号 FIRST 集和 FOLLOW 集的计算还有关系矩阵法等，有兴趣的读者可参考有关书籍。

第四步，计算 SELECT 集。

对例 4.5 文法的 FIRST 集和 FOLLOW 集的计算结果如表 4.2。

表 4.2　文法的 FIRST 集和 FOLLOW 集

非终结符名	是否 $\overset{*}{\Rightarrow}\varepsilon$	FIRST 集	FOLLOW 集
S	是	$\{b,a,\varepsilon\}$	$\{\#\}$
A	是	$\{b,\varepsilon\}$	$\{a,c,\#\}$
B	是	$\{a,\varepsilon\}$	$\{\#\}$
C	否	$\{a,b,c\}$	$\{\#\}$
D	否	$\{a,c\}$	$\{\#\}$

每个产生式的 SELECT 集合计算如下：

$\text{SELECT}(S \to AB) = (\text{FIRST}(AB) - \{\varepsilon\}) \cup \text{FOLLOW}(S) = \{b,a,\#\}$

$\text{SELECT}(S \to bC) = \text{FIRST}(bC) = \{b\}$

$\text{SELECT}(A \to \varepsilon) = (\text{FIRST}(\varepsilon) - \{\varepsilon\}) \cup \text{FOLLOW}(A) = \{a,c,\#\}$

$\text{SELECT}(A \to b) = \text{FIRST}(b) = \{b\}$

$\text{SELECT}(B \to \varepsilon) = (\text{FIRST}(\varepsilon) - \{\varepsilon\}) \cup \text{FOLLOW}(B) = \{\#\}$

$\text{SELECT}(B \to aD) = \text{FIRST}(aD) = \{a\}$

$\text{SELECT}(C \to AD) = \text{FIRST}(AD) = \{a,b,c\}$

$\text{SELECT}(C \to b) = \text{FIRST}(b) = \{b\}$

$\text{SELECT}(D \to aS) = \text{FIRST}(aS) = \{a\}$

$\text{SELECT}(D \to c) = \text{FIRST}(c) = \{c\}$

由以上计算结果可得相同左部产生式的 SELECT 交集为

$\text{SELECT}(S \to AB) \cap \text{SELECT}(S \to bC) = \{b,a,\#\} \cap \{b\} = \{b\} \neq \varnothing$

$\text{SELECT}(A \to \varepsilon) \cap \text{SELECT}(A \to b) = \{a,c,\#\} \cap \{b\} = \varnothing$

$\text{SELECT}(B \to \varepsilon) \cap \text{SELECT}(B \to aD) = \{\#\} \cap \{a\} = \varnothing$

$\text{SELECT}(C \to AD) \cap \text{SELECT}(C \to b) = \{b,a,c\} \cap \{b\} = \{b\} \neq \varnothing$

$\text{SELECT}(D \to aS) \cap \text{SELECT}(D \to c) = \{a\} \cap \{c\} = \varnothing$

由 LL(1)文法定义得知该文法不是 LL(1)文法，因为关于 S 和 C 的相同左部，其产生式的 SELECT 集的交集不为$\varnothing$。

4.3　某些非 LL(1)文法到 LL(1)文法的等价变换

在 4.1 节和 4.2 节中指出，确定的自顶向下分析要求给定语言的文法必须是 LL(1)形式。然而，不一定每个语言都有 LL(1)文法，对一个语言的非 LL(1)文法是否能变换为等价的 LL(1)形式以及如何变换是本节讨论的主要问题。由 LL(1)文法的定义可知，若文法中含有直接或间接左递归，或含有左公共因子，则该文法肯定不是 LL(1)文法，因而，要设法消除文法中的左递归，提取左公共因子对文法进行等价变换。在某些特殊情况下可能使其变为 LL(1)文法。

4.3.1　提取左公共因子

若文法中含有形如 $A \to \alpha\beta \mid \alpha\gamma$ 的产生式，就会导致对相同左部的产生式其右部的

FIRST 集相交，也就是 SELECT($A\to\alpha\beta$)$\cap$SELECT($A\to\alpha\gamma$)$\neq\varnothing$，不满足 LL(1)文法的充分必要条件。

可将产生式 $A\to\alpha\beta|\alpha\gamma$ 等价变换为

$$A\to\alpha(\beta\mid\gamma)$$

其中括号为元符号，再引进新非终结符 A'，去掉括号使产生式变换为

$$A\to\alpha A'$$
$$A'\to\beta\mid\gamma$$

写成一般形式为

$$A\to\alpha\beta_1\mid\alpha\beta_2\mid\cdots\mid\alpha\beta_n$$

提取左公共因子后变为

$$A\to\alpha(\beta_1\mid\beta_2\mid\cdots\mid\beta_n)$$

引进非终结符 A'后变为

$$A\to\alpha A'$$
$$A'\to\beta_1\mid\beta_2\mid\cdots\mid\beta_n$$

若在 $\beta_i,\beta_j,\beta_k,\cdots$(其中 $1\leqslant i,j,k\leqslant n$)中仍含有左公共因子，这时可再次提取，这样反复进行提取，直到引进新非终结符的有关产生式再无左公共因子为止。

例 4.6 文法 $G1$ 的产生式为

(1) $S\to aSb$

(2) $S\to aS$

(3) $S\to\varepsilon$

对产生式(1)、(2)提取左公共因子后得：

$$S\to aS(b\mid\varepsilon)$$
$$S\to\varepsilon$$

进一步变换为文法 $G'1$：

$$S\to aSA$$
$$A\to b$$
$$A\to\varepsilon$$
$$S\to\varepsilon$$

例 4.7 文法 $G2$ 的产生式为

(1) $A\to ad$

(2) $A\to Bc$

(3) $B\to aA$

(4) $B\to bB$

产生式(2)的右部以非终结符开始，因此左公共因子可能是隐式的，所以这种情况下对右部以非终结符开始的产生式用左部相同而右部以终结符开始的产生式进行相应替换，对文法 $G2$ 分别用产生式(3)、(4)的右部替换产生式(2)中的 B，可得

(1) $A\to ad$

(2) $A\to aAc$

(3) $A\to bBc$

(4) $B \rightarrow aA$

(5) $B \rightarrow bB$

提取产生式(1)、(2)的左公共因子得

$$A \rightarrow a(d \mid Ac)$$

$$A \rightarrow bBc$$

$$B \rightarrow aA$$

$B \rightarrow bB$ 引进新非终结符 A'，去掉括号后得 $G'2$ 为

(1) $A \rightarrow aA'$

(2) $A \rightarrow bBc$

(3) $A' \rightarrow d$

(4) $A' \rightarrow Ac$

(5) $B \rightarrow aA$

(6) $B \rightarrow bB$

不难验证经提取左公共因子后文法 $G'1$ 仍不是 LL(1)文法，而文法 $G'2$ 变成了 LL(1)文法，因此文法中不含左公共因子只是 LL(1)文法的必要条件，而不是充分条件。

值得注意的是，对文法进行提取左公共因子变换后，有时会使某些产生式变成无用产生式，在这种情况下必须对文法重新压缩(或化简)。

例 4.8 有文法 $G3$ 的产生式为

(1) $S \rightarrow aSd$

(2) $S \rightarrow Ac$

(3) $A \rightarrow aS$

(4) $A \rightarrow b$

用产生式(3)、(4)中右部替换产生式(2)中右部的 A，文法变为

(1) $S \rightarrow aSd$

(2) $S \rightarrow aSc$

(3) $S \rightarrow bc$

(4) $A \rightarrow aS$

(5) $A \rightarrow b$

对产生式(1)、(2)提取左公共因子得

$$S \rightarrow aS(d \mid c)$$

引入新非终结符 A' 后变为

(1) $S \rightarrow aSA'$

(2) $S \rightarrow bc$

(3) $A' \rightarrow d \mid c$

(4) $A \rightarrow aS$

(5) $A \rightarrow b$

显然，原文法中非终结符 A 变成不可到达的符号，产生式(4)、(5)也就变为无用产生式，所以应删除。

此外也存在某些文法不能在有限步骤内提取完左公共因子的情况。

例 4.9 文法 $G4$ 的产生式为

(1) $S \rightarrow Ap \mid Bq$

(2) $A \rightarrow aAp \mid d$

(3) $B \rightarrow aBq \mid e$

用产生式(2)、(3)的右部替换产生式(1)中的 A、B，使文法变为

(1) $S \rightarrow aApp \mid aBqq$

(2) $S \rightarrow dp \mid eq$

(3) $A \rightarrow aAp \mid d$

(4) $B \rightarrow aBq \mid e$

对产生式(1)提取左公共因子则得

$$S \rightarrow a(App \mid Bqq)$$

再引入新非终符 S'，结果得等价文法为

(1) $S \rightarrow aS'$

(2) $S \rightarrow dp \mid eq$

(3) $S' \rightarrow App \mid Bqq$

(4) $A \rightarrow aAp \mid d$

(5) $B \rightarrow aBq \mid e$

同样分别用产生式(4)、(5)的右部替换产生式(3)中右部的 A、B，再提取左公共因子，最后结果为

(1) $S \rightarrow aS'$

(2) $S \rightarrow dp \mid eq$

(3) $S' \rightarrow aS''$

(4) $S' \rightarrow dpp \mid eqq$

(5) $S'' \rightarrow Appp \mid Bqqq$

(6) $A \rightarrow aAp \mid d$

(7) $B \rightarrow aBq \mid e$

可以看出，若对产生式(5)中 A、B 继续用产生式(6)、(7)的右部替换，只能使文法的产生式愈来愈多，无限增加下去，而不能得到提取左公共因子的预期结果。

由上面所举例子可以说明以下问题：

(1) 不一定每个文法的左公共因子都能在有限的步骤内替换成无左公共因子的文法，上面的文法 $G4$ 就是如此。

(2) 一个文法提取了左公共因子后，只解决了相同左部产生式右部的 FIRST 集不相交的问题。当改写后的文法不含空产生式，且无左递归时，则改写后的文法是 LL(1)文法，若还有空产生式时，则还需用 LL(1)文法的判别方式进行判断才能确定是否为 LL(1)文法。

4.3.2 消除左递归

观察在文法中含有如下形式的两种产生式的情形。

(1) $A \rightarrow A\beta, A \in V_N, \beta \in V^*$

(2) $A \rightarrow B\beta$

$B \rightarrow A\alpha \quad A, B \in V_N, \quad \alpha, \beta \in V^*$

含(1)中情况的产生式，则称文法含有左递归的规则或称直接左递归。含(2)中情况的产生式可以形成推导 $A \overset{+}{\Rightarrow} A\cdots$，则称文法中含有左递归或间接左递归。文法中只要含有(1)或含有(2)或二者皆有，均认为文法是左递归的。然而，一个文法是左递归时不能采用自顶向下分析法，下面用例 4.10 和例 4.11 加以说明。

例 4.10 文法 $G5$ 含有直接左递归：

$$S \rightarrow Sa$$

$$S \rightarrow b$$

所能产生的语言 $L=\{ba^n \mid n \geqslant 0\}$，输入串 $baaaa\#$ 应是该语言的句子，但用自顶向下分析时可看出，当输入符为 b 时，为与 b 匹配则应选用 $S \rightarrow b$ 来推导，但这样就推不出后边部分；而若用 $S \rightarrow Sa$ 推导则出现图 4.6 的情况，无法确定到什么时候才用 $S \rightarrow b$ 替换。

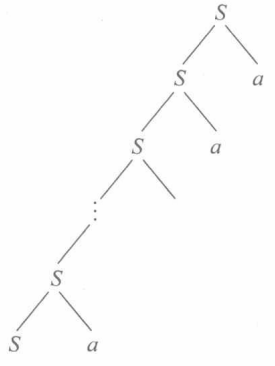


图 4.6 含直接左递归文法的语法分析树结构

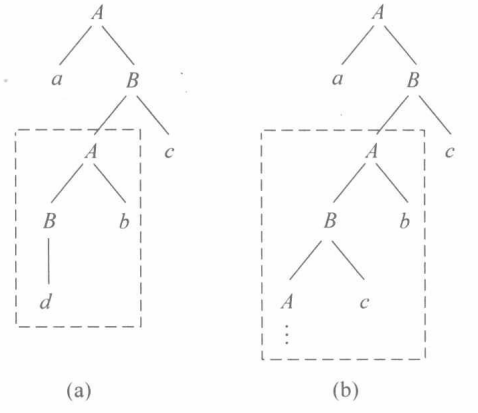


图 4.7 含间接左递归文法的语法分析树

例 4.11 文法 $G6$ 含有间接左递归：

(1) $A \rightarrow aB$

(2) $A \rightarrow Bb$

(3) $B \rightarrow Ac$

(4) $B \rightarrow d$

若有输入串为 $adbcbcbc\#$，当分析过程至 $A \Rightarrow aB \Rightarrow aAc \Rightarrow aBbc$ 时，B 若用产生式(4)替换，则分析过程的语法树为图 4.7(a)。推导到此终止，不能推出 $adbcbcbc\#$；而若选用产生式(3)，则会出现图 4.7(b)所示的情况。

自左向右分析法在没有与当前输入符号匹配而进入产生式选择循环时只能用带回溯的不确定分析方法。此方法将在 4.4 节介绍。

由上述例子不难看出含有左递归的文法绝对不是 LL(1)文法(对此结论读者可以自己证明)，所以也就不可能用确定的自顶向下分析法。然而，为了使某些含有左递归的文法经等价变换消除左递归后可能变为 LL(1)文法，可采取下列方法。

1. 消除直接左递归

把直接左递归改写为右递归。例如，对文法 $G5$：

$$S \to Sa$$
$$S \to b$$

可改写为

$$S \to bS'$$
$$S' \to aS' \mid \varepsilon$$

改写后的文法和原文法产生的语言句子集都为$\{ba^n \mid n \geqslant 0\}$，不难验证改写后的文法为 LL(1) 文法。

一般情况下，假定关于 A 的全部产生式是

$$A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

其中，$\alpha_i (1 \leqslant i \leqslant m)$不等于 ε，$\beta_j (1 \leqslant j \leqslant n)$不以 A 开头，消除直接左递归后改写为

$$A \to \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'$$
$$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon$$

2. 消除间接左递归

要消除间接左递归，需先通过产生式非终结符置换，将间接左递归变为直接左递归，然后再按(1)消除直接左递归。

以文法 $G6$ 为例：

(1) $A \to aB$

(2) $A \to Bb$

(3) $B \to Ac$

(4) $B \to d$

用产生式(1)、(2)的右部置换产生式(3)中的非终结符 A，得到左部为 B 的产生式：

(1) $B \to aBc$

(2) $B \to Bbc$

(3) $B \to d$

消除左递归后得

$$B \to aBcB' \mid dB'$$
$$B' \to bcB' \mid \varepsilon$$

再把原来其余的产生式 $A \to aB$ 和 $A \to Bb$ 加入，最终文法为

(1) $A \to aB$

(2) $A \to Bb$

(3) $B \to aBcB' \mid dB'$

(4) $B' \to bcB' \mid \varepsilon$

该文法与 $G6$ 等价，即它们产生相同的句子集。

读者可以检验改写后的文法是否为 LL(1) 文法。

3. 消除文法中一切左递归的算法

对文法中一切左递归的消除要求文法中不含回路，即无 $A \overset{+}{\Rightarrow} A$ 的推导。

满足这个要求的充分条件是，文法中不包含形如 $A \rightarrow A$ 的有害规则和 $A \rightarrow \varepsilon$ 的空产生式。

算法步骤如下：

(1) 把文法的所有非终结符按某一顺序排序，例如：

$$A_1, A_2, \cdots, A_n$$

(2) FOR $i := 1$ TO N DO

BEGIN

FOR $j := 1$ TO $i-1$ DO

BEGIN

若 A_j 的所有产生式为

$A_j \rightarrow \delta_1 | \delta_2 | \cdots | \delta_k$

将其替换形如 $A_i \rightarrow A_j r$ 的产生式得到

$A_i \rightarrow \delta_1 r | \delta_2 r | \cdots | \delta_k r$

END

消除 A_i 中的一切直接左递归。

END

(3) 去掉无用产生式。

例如，按上述方法消除如下文法的一切左递归：

(1) $S \rightarrow Qc | c$

(2) $Q \rightarrow Rb | b$

(3) $R \rightarrow Sa | a$

若非终结符排序为 S、Q、R，左部为 S 的产生式(1)无直接左递归，左部为 Q 的产生式(2)中右部不含 S，所以把产生式(1)的右部代入产生式(3)得

(4) $R \rightarrow Qca | ca | a$

再将产生式(2)的右部代入产生式(4)得

(5) $R \rightarrow Rbca | bca | ca | a$

对产生式(5)消除直接左递归得

$$R \rightarrow bcaR' \mid caR' \mid aR'$$

$$R' \rightarrow bcaR' \mid \varepsilon$$

最终文法变为

$$S \rightarrow Qc \mid c$$

$$Q \rightarrow Rb \mid b$$

$$R \rightarrow bcaR' \mid caR' \mid aR'$$

$$R' \rightarrow bcaR' \mid \varepsilon$$

若非终结符的排序为 R、Q、S，则把产生式(3)代入产生式(2)得

$$Q \rightarrow Sab \mid ab \mid b$$

再将此代入产生式(1)得

$$S \rightarrow Sabc \mid abc \mid bc \mid c$$

消除该产生式的左递归后，文法变为

$$S \rightarrow abcS' \mid bcS' \mid cS'$$

$$S' \rightarrow abcS' \mid \varepsilon$$

$$Q \rightarrow Rb \mid b$$
$$R \rightarrow Sa \mid a$$

由于 Q、R 为不可到达的非终结符，所以以 Q、R 为左部及包含 Q、R 的产生式应删除。最终文法变为

$$S \rightarrow abcS' \mid bcS' \mid cS'$$
$$S' \rightarrow abc\ S' \mid \varepsilon$$

当非终结符的排序不同时，最后结果的产生式形式不同，但它们是等价的。

4.4 不确定的自顶向下分析思想

在 4.1 节至 4.3 节中可以很清楚地看到，当文法不满足 LL(1)时，则不能用确定的自顶向下分析，但在这种情况下可用不确定的自顶向下分析，也就是带回溯的自顶向下分析。引起回溯的原因是：在文法中当关于某个非终结符的产生式有多个候选时，而面临当前的输入符无法确定选用唯一的产生式，从而引起回溯，现以下面 3 个简单例子来说明。

1. 由于相同左部的产生式的右部 FIRST 集交集不为空而引起回溯

例如，有文法：

$S \rightarrow xAy$

$A \rightarrow ab \mid a$

若当前输入串为 xay，则可能的第一步推导树为图 4.8(a)。进一步推导对 A 可选择 $A \rightarrow ab$ 替换，得语法树为图 4.8(b)，其中 xa 都已匹配，当前面临的输入符为 y 与 b 不能匹配，所以将输入串指针退回到 a，对 A 的替换重新选用下一个产生式 $A \rightarrow a$ 进行试探，如图 4.8(c)所示。输入串中当前符 a 得到匹配，指针向前移动到 y，与语法树中 y 匹配，匹配成功。

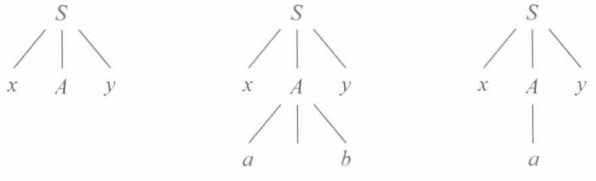


图 4.8 不确定的自顶向下语法分析树(一)

2. 由于相同左部非终结符的右部存在能 $\overset{*}{\Rightarrow}\varepsilon$ 的产生式，且该非终结符的 FOLLOW 集中含有其他产生式右部 FIRST 集的元素

例如例 4.4 的文法：

(1) $S \rightarrow aAS$

(2) $S \rightarrow b$

(3) $A \rightarrow bAS$

(4) $A \rightarrow \varepsilon$

对输入串 $ab\#$ 的试探推导过程在图 4.9 中给出。当面临 a 时，用产生式(1)推导，a 得到匹配，输入串指针移到 b，语法树中可用 A 向下推导，而对 A 的产生式，可选产生式(3)或产生式(4)，先试选用产生式(3)推导。

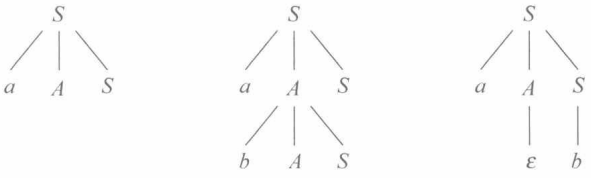


图 4.9　不确定的自顶向下语法分析树(二)

这时 b 得到匹配,输入串指针向右移动,输入符已结束,但从语法树可以看出,末端结点并非全是终结符,而且 ab 右部的非终结符 ASS 不能推出 ε,所以可知推导是失败的,需回溯使输入指针退回到 b,对 A 的推导改为选用下一个产生式 $A\rightarrow\varepsilon$,对 b 用 A 的后跟符匹配。

继续用 S 的产生式和 b 匹配,S 的两个产生式只能选用 $S\rightarrow b$,则最终得到匹配,试推成功。

3. 由于文法含有左递归而引起回溯

例如,有文法:

(1) $S\rightarrow Sa$

(2) $S\rightarrow b$

若推导 $baa\#$,开始由于当前输入符是 b,所以试图用产生式(2)推导,对应语法树为图 4.10(a)。

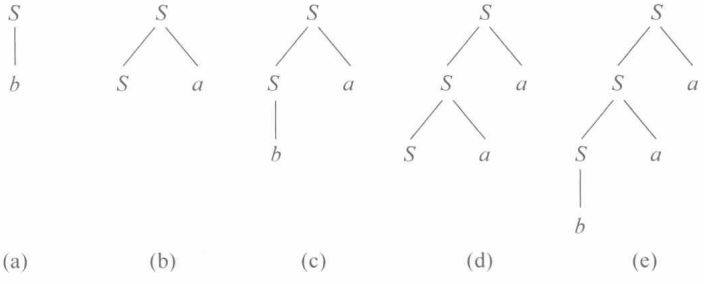


图 4.10　不确定的自顶向下语法分析树(三)

这时推导树的末端结点都是终结符,输入串未分析完,所以应重新选用产生式(1)来推导,对应语法树为图 4.10(b)。

语法树末端结点最左符号为非终结符,当前输入符为 b,所以选用产生式(2)继续推导,得语法树为图 4.10(c)。

此时语法树最左边的符号为终结符 b 与当前符号匹配,语法树下一符号与输入串的下一符号都为 a,所以匹配,但语法树的末端结点只有两个终结符,而输入串还有剩余部分 $a\#$,所以要把第 2 步的推导回溯,输入串指针也恢复到第一个符号,对第 2 步推导的重新选择只有选择产生式(1)。相应语法树为图 4.10(d)。

这时语法树的最左末端结点为非终结符 S,当前输入符为 b,所以试用产生式(2)推导,得语法树为图 4.10(e)。

当前输入符与语法树最左末端结点匹配,剩余的输入符 aa 与语法树其余的末端结点也逐个相匹配,最后遇到输入串结束符,所以分析成功。

由以上讨论可以看出，带回溯的自顶向下分析是一个试探过程，当分析不成功时则推翻分析，退回到适当位置，再重新试探其余候选可能的推导，这样需要记录已选过的产生式，直到把所有可能的推导序列都试完仍不成功，才能确认输入串不是该文法的句子而报错，由于在编译程序真正实现时往往是边分析边插入语义动作，因而带回溯分析代价很高，效率很低，在实用编译程序中几乎不用，因此对它实现的详细算法不做介绍。

4.5 LL(1)分析的实现

本节介绍两种常用的 LL(1)分析的实现方法。

4.5.1 递归下降 LL(1)分析程序

在递归下降 LL(1)分析程序的设计中，每个非终结符都对应一个分析子程序，分析程序从调用文法开始符号所对应的分析子程序开始执行。非终结符对应的分析子程序根据下一个单词符号可确定自顶向下分析过程中应该使用的产生式，根据所选定的产生式，分析子程序的行为依据产生式右端依次出现的符号来设计：

- 每遇到一个终结符，则判断当前读入的单词符号是否与该终结符相匹配(只要求单词符号与该终结符对应，不必考虑单词自身的值)，若匹配，则继续读取下一个单词符号；若不匹配，则进行错误处理。
- 每遇到一个非终结符，则调用相应的分析子程序。

例如，设有如下产生式：

<function>→FUNC ID(<parameter_list>) <statement>

其中，<function>，<parameter_list>和<statement>是非终结符，而 FUNC 和 ID 是终结符。若它是左部为<function>的唯一产生式，那么非终结符<function>对应的分析子程序 ParseFunction()的设计可描述为

```
void ParseFunction()
{
    MatchToken(T_FUNC);          //T_FUNC 为终结符 FUNC 对应的单词种别
    MatchToken(T_ID);            //T_ID 为终结符 ID 对应的单词种别
    MatchToken(T_LPAREN);        //T_LPAREN 为终结符'('对应的单词种别
    ParseParameterList();
    MatchToken(T_RPAREN);        //T_RPAREN 为终结符')'对应的单词种别
    ParseStatement();
}
```

其中，ParseParameterList()和 ParseStatement()分别为非终结符<parameter_list>和<statement>对应的分析子程序，而函数 MatchToken()则是用于匹配当前终结符和正在扫描的单词符号(若匹配，则调用词法分析程序取下一个单词符号；否则报告词法错误信息)。函数 MatchToken()的一种简单的设计为

```
void MatchToken(int expected)
{
```

```
    if (lookahead ! =expected)                    //判别当前扫描的单词符号是否与期望的终结符匹配
    {
        printf("syntax error \n");                //若不匹配,则报告出错信息,跳出
        exit(0);
    }
  else                                  //若匹配,消掉当前单词符号,从词法分析程序读入下一个单词符号
      lookahead=getToken();                       //并将该单词符号的单词种别赋值给 lookahead
}
```

其中,lookahead 为全局量,存放当前所扫描单词符号的单词种别。

在随后的讨论以及例子中,将继续使用全局量 lookahead 和 MatchToken()函数。为叙述简洁,如不特别指明,后面将文法中的终结符直接用来代表当前所扫描单词符号的单词种别。

一般情况下,设 LL(1)文法中某一非终结符 A 对应的所有产生式的集合为

$$A \rightarrow u_1 \mid u_2 \mid \cdots \mid u_n$$

那么相对于非终结符 A 的分析子程序 ParseA()可以具有如下形式的一般结构:

```
void ParseA()
{
    switch (lookahead)
    {
        case SELECT (A →u1):
            ……                                  /*根据 u1 设计的分析过程*/
            break;
        case SELECT (A →u2):
            ……                                  /*根据 u2 设计的分析过程*/
            break;
        ⋮
        case SELECT (A →un):
            ……                                  /*根据 un设计的分析过程*/
            break;
        default:
            printf("syntax error \n");
            exit(0);
    }
}
```

值得注意的是,由于是 LL(1)文法,所以产生式 $A \rightarrow u_1, A \rightarrow u_2, \cdots, A \rightarrow u_n$ 的 SELECT 集合是两两互不相交的,故上述选择语句中的各个选择之间是互斥的。

例 4.12 设文法 $G[S]$为

$S \rightarrow AaS \mid BbS \mid d$

$A \rightarrow a$

$B \rightarrow \varepsilon \mid c$

容易计算出各产生式的 SELECT 集合:

$$\text{SELECT}(S \rightarrow AaS) = \{a\}$$
$$\text{SELECT}(S \rightarrow BbS) = \{c, b\}$$
$$\text{SELECT}(S \rightarrow d) = \{d\}$$
$$\text{SELECT}(A \rightarrow a) = \{a\}$$
$$\text{SELECT}(B \rightarrow \varepsilon) = \{b\}$$
$$\text{SELECT}(B \rightarrow c) = \{c\}$$

因为 SELECT($S \rightarrow AaS$)，SELECT($S \rightarrow BbS$) 以及 SELECT($S \rightarrow d$)互不相交，SELECT ($B \rightarrow \varepsilon$)和 SELECT($S \rightarrow d$)不相交，所以，$G[S]$是 LL(1)文法。

这样，开始符号 S 对应的分析子程序可以设计为

```
void ParseS( )
{
    switch (lookahead)
    {
        case a:
            ParseA( );
            MatchToken(a);
            ParseS( );
            break;
        case b,c:
            ParseB( );
            MatchToken(b);
            ParseS( );
            break;
        case d:
            MatchToken(d);
            break;
        default:
            printf("syntax error \n")
            exit(0);
    }
}
```

终结符 A 对应的分析子程序可以设计为

```
void ParseA( )
{
    if (lookahead==a)
    {
        MatchToken(a);
    }
    else
    {
        printf("syntax error \n");
```

```
        exit(0);
    }
}
```

终结符 B 对应的分析子程序可以设计为：

```
void ParseB( )
{
    if (lookahead==c)
    {
        MatchToken(c);
    }
    else if (lookahead==b) {
        }
        else {
            printf("syntax error \n");
            exit(0);
        }
}
```

在实践中，这种递归下降分析程序的设计思想也可用于其他的语法描述形式。例如，在EBNF形式的语法描述中，每一条规则右部的语法成分之间除了可以有连接算符之外，还包含其他一些算符，主要有选择、重复、任选以及优先括号等。这种更加丰富的表达方式有利于精简分析子程序的设计，从而提高递归下降分析程序的效率。在递归下降分析子程序的设计中，针对不同算符，可选择不同的处理语句：

- $X_1|X_2|\cdots|X_m$：表示多个成分之间的选择，可对应到选择语句。
- $\{X\}$：表示成分 X 的重复(0到多次)，可对应到循环语句。
- $[X]$：表示成分 X 的任选(0或1次)，可对应到if-then语句。
- (X)：表示成分 X 的优先处理，可对应到复合语句。

在处理多个成分之间的选择时，也需要保证各成分的SELECT集合之间互不相交。计算SELECT集合，需要先计算出必要的FIRST集合和FOLLOW集合，它们的含义与本章中上下文无关文法下相应的概念类似。由于本书不涵盖有关EBNF形式定义的内容，所以这里不给出这些集合的严格定义，而是在随后介绍的实例中进行必要的讨论。为帮助理解这些实例，下面列出关于FIRST集合的一些性质：

- $\mathrm{FIRST}(X_1|X_2|\cdots|X_m)=\mathrm{FIRST}(X_1)\cup\cdots\cup\mathrm{FIRST}(X_m)$
- $\mathrm{FIRST}(\{X\})=\mathrm{FIRST}(X)\cup\{\varepsilon\}$
- $\mathrm{FIRST}([X])=\mathrm{FIRST}(X)\cup\{\varepsilon\}$
- $\mathrm{FIRST}((X))=\mathrm{FIRST}(X)$

附录A中的PL/0语法分析程序是基于表4.3的EBNF形式的语法描述设计的递归下降分析程序。相比第1章表1.1中PL/0语言语法的EBNF描述，表4.3的EBNF规则较少，只有9条，因此在PL/0语法分析程序中对应设计了9个分析子程序。

表 4.3　改进后的 PL/0 语言语法的 EBNF 描述

说法单位	EBNF 描述
<程序>	::=<分程序>.
<分程序>	::=[**const** <常量定义>{,<常量定义>};] [**var** <变量定义>{,<变量定义>};] [{**procedure** <id>;<分程序>;}] <语句>
<常量定义>	::=<标识符>=<integer>
<变量定义>	::=<id>
<语句>	::= <id> :=<表达式> |**if** <条件> **then** <语句> |**while** <条件> **do** <语句> |**call** <id> |**read** '('<id>{,<id>}')' |**write** '('<表达式>{,<表达式>}')' |**begin** <语句>{;<语句>} **end** |ε
<条件>	::= <表达式> (=|#|<|<=|>|>=) <表达式> |**odd** <表达式>
<表达式>	::= [+|-]<项>{(+|-)<项>}
<项>	::= <因子>{(* |/)<因子>}
<因子>	::= <id> |<integer> |'('<表达式>')'

需要验证一下表 4.3 的 EBNF 描述中任何一个含有选择算符的子表达式的所有下一级子表达式的 SELECT 集合是互不相交的。对于下列分析过程，必要时可参考 4.6.3 节的表 4.6，其中列举了表 4.3 中描述的 PL/0 部分语法单位的 First 集合和 Follow 集合。

对于<因子>规则中的选择表达式<id>|<integer>|'('<表达式>')'，下一级的子表达式<id>，<integer>和'('<表达式>')'的 FIRST 集合分别为{<id>}，{<integer>}和{'('}，所以这 3 个子表达式的 SELECT 集合是互不相交的。同样，对于<项>规则中的 * |/，<表达式> 规则中的两处 + |−，以及<条件>规则中的=|#|<|<=|>|>=等含有选择算符的子表达式，都容易计算出它们下一级子表达式的 FIRST 集合，实际上也是 SELECT 集合，并可以验证这些 SELECT 集合之间是互不相交的。

对于<条件> 规则中的选择表达式，可以计算出

- FIRST(<表达式> (=|#|<|<=|>|>=) <表达式>)={+,−,<id>,<integer>,'('}
- FIRST(**odd** <表达式>)={**odd**}

可见，这两个子表达式的 SELECT 集合不相交。

对于<语句>规则右边的选择表达式，子表达式 <id> :=<表达式>，**if** <条件> **then** <语句>，**while** <条件> **do** <语句>，**call** <id>，**read** '('<id> {,<id>}')'，**write** '('<表达式> {,<表达式>}')'，**begin** <语句> {;<语句>} **end** 和空语句 ε 的 FIRST 集合分别是{<id>}，{**if**}，{**while**}，{**call**}，{**read**}，{**write**}，{**begin**}和{ε}。显然，除空语句外，

其余 7 个下一级子表达式的 SELECT 集合就是它们的 FIRST 集合。另外，借助直观的理解，可以根据表 4.3 的 EBNF 描述得知空语句的 FOLLOW 集合为{**end**,.}，从而可知，其 SELECT 集合为 {**end**,.}。显然，这些 SELECT 集合之间是互不相交的。

通过以上分析，可以得出结论：对于表 4.3 的 EBNF 描述，在任何一个含有选择算符的子表达式中，它的所有下一级子表达式的 SELECT 集合是互不相交的。这样，就可以基于这个 EBNF 描述设计一个通过向前察看一个单词符号的递归下降分析程序实现 PL/0 语言的语法分析。

PL/0 语法分析程序由 8 个分别与表 4.3 中的语法规则直接对应的递归子过程组成：

- block()对应<分程序>。
- constdeclaration()对应<常量定义>。
- vardeclaration()对应<变量定义>。
- statement()对应<语句>。
- condition()对应<条件>。
- expression()对应<表达式>。
- term()对应<项>。
- factor()对应<因子>。

表 4.3 中<程序>对应的分析过程包含于 PL/0 编译程序的主函数 main()中。函数 main()的程序结构为

```
int main()
{
    ......                                  /*初始化*/
    ......                                  /*读写文件*/
    getsym();
    block(......)                           /* 处理<分程序>*/
    ......
    if (sym ! =period)
        error(9);                           /*提示 9 号出错信息:缺少程序结束符'.'*/
    ......
    return 0;
}
```

每个递归子过程的内容都是根据相应规则的右部来设计的。例如，根据表 4.3 中<表达式>、<项>和<因子>等规则，PL/0 语法分析程序中包含 expression()、term()和 factor()等分析子过程(函数)，它们的程序结构为

```
int expression(......)
{
    if (sym==plus||sym==minus)              /* 处理 +| - */
    {
        getsym();
        term (......);                      /* 处理<项> */
    }
```

```
    else
    {
        term (.....);                          /* 处理<项> */
    }
    while (sym==plus||sym==minus)              /* 处理{(+|-)<项>} */
    {
        getsym;
        term (.....);                          /* 处理<项> */
    }
        return 0;
    }

    int term(.....)
    {
        factor(.....);                         /*处理<因子>*/
        while (sym==times||sym==slash)         /* 处理{(*|/)<因子>} */
        {
            getsym();
            factor(.....);                     /*处理<因子>*/
        }
        return 0;
    }

    int factor (.....)
    {
        if (sym==ident) {                      /* <因子>为常量或变量 */
            getsym();
        elseif (sym==number)                   /* <因子>为立即数 */
                getsym();
            else if(sym==lparen);              /* <因子>为'('<表达式>')' */
                {
                    expression(.....);         /* 处理<表达式> */
                    if (sym==rparen)
                        getsym();
                    else
                        error(22);             /*提示22号出错信息：缺少右括号*/
                }
        return 0;
}
```

4.5.2 表驱动 LL(1)分析程序

递归下降分析程序比较直观，容易设计，但不足之处就是递归调用可能带来的效率问题。本节介绍另外一种 LL(1)分析程序的实现方法，称为表驱动的方法。一个表驱动的

LL(1)分析程序由预测分析程序、先进后出栈和预测分析表3个部分组成，其中只有预测分析表与文法有关，而分析表又可用一个矩阵 $\boldsymbol{M}$（或称二维数组）表示。矩阵的元素 $M[A,a]$ 中的下标 A 表示非终结符，a 为终结符或句子括号 #，矩阵元素 $M[A,a]$ 中的内容是一条关于 A 的产生式，表明当用非终结符 A 向下推导时，面临输入符 a 时所应采取的候选产生式。当元素内容无产生式时，则表明用 A 为左部向下推导时遇到了不该出现的符号，因此元素内容为转向出错处理的信息。

预测分析程序的工作过程用图 4.11 表示。

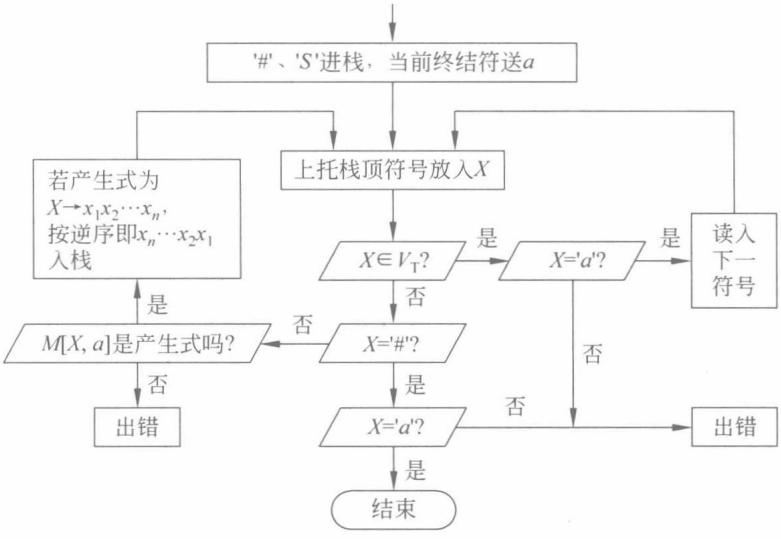


图 4.11　预测分析程序的流程图

图中符号说明如下：

'#'是句子括号，即输入串的括号。

'S'是文法的开始符号。

'X'是存放当前栈顶符号的工作单元。

'a'是存放当前输入符号 a 的工作单元。

现以表达式文法为例构造预测分析表。表达式文法为：

$$E \to E+T \mid T$$
$$T \to T*F \mid F$$
$$F \to i \mid (E)$$

构造步骤如下。

(1) 判断文法是否为 LL(1)文法。

由于文法中含有左递归，所以必须先消除左递归，使文法变为

$$E \to TE'$$
$$E' \to +\ TE' \mid \varepsilon$$
$$T \to FT'$$
$$T' \to *\ F\ T' \mid \varepsilon$$
$$F \to i \mid (E)$$

根据第 4.2 节的内容可得以下结果。

① 可推出 ε 的非终结符表为

E	E'	T	T'	F
否	是	否	是	否

② 各非终结符的 FIRST 集合如下：

$$\text{FIRST}(E) = \{(, i\}$$
$$\text{FIRST}(E') = \{+, \varepsilon\}$$
$$\text{FIRST}(T) = \{(, i\}$$
$$\text{FIRST}(T') = \{*, \varepsilon\}$$
$$\text{FIRST}(F) = \{(, i\}$$

③ 各非终结符的 FOLLOW 集合为

$$\text{FOLLOW}(E) = \{), \#\}$$
$$\text{FOLLOW}(E') = \{), \#\}$$
$$\text{FOLLOW}(T) = \{+,), \#\}$$
$$\text{FOLLOW}(T') = \{+,), \#\}$$
$$\text{FOLLOW}(F) = \{*, +,), \#\}$$

④ 各产生式的 SELECT 集合为

$$\text{SELECT}(E \to TE') = \{(, i\}$$
$$\text{SELECT}(E' \to +TE') = \{+\}$$
$$\text{SELECT}(E' \to \varepsilon) = \{), \#\}$$
$$\text{SELECT}(T \to FT') = \{(, i\}$$
$$\text{SELECT}(T' \to *FT') = \{*\}$$
$$\text{SELECT}(T' \to \varepsilon) = \{+,), \#\}$$
$$\text{SELECT}(F \to (E)) = \{(\}$$
$$\text{SELECT}(F \to i) = \{i\}$$

由上可知，有相同左部产生式的 SELECT 集合的交集为空，所以文法是 LL(1)文法。

(2) 构造预测分析表。

对每个终结符或'#'号用 a 表示。

若 $a \in \text{SELECT}(A \to \alpha)$，则把 $A \to \alpha$ 放入 $M[A, a]$ 中。

把所有无定义的 $M[A, a]$ 标上出错标记。

为了使表简化，其产生式的左部可以不写入表中，表中空白处为出错。

上例的预测分析表为表 4.4。

表 4.4　表达式文法的预测分析表

	i	$+$	$*$	$($	$)$	$\#$
E	$\to TE'$			$\to TE'$		
E'		$\to +TE'$			$\to \varepsilon$	$\to \varepsilon$
T	$\to FT'$			$\to FT'$		
T'		$\to \varepsilon$	$\to *FT'$		$\to \varepsilon$	$\to \varepsilon$
F	$\to i$			$\to (E)$		

下面用预测分析程序、栈和预测分析表对输入串 $i+i*i\#$ 进行分析，栈的变化过程在表 4.5 中给出。

表 4.5　对符号串 $i+i*i\#$ 的分析过程

步　骤	分析栈	剩余输入串	推导所用产生式或匹配
1	$\#E$	$i+i*i\#$	$E\to TE'$
2	$\#E'T$	$i+i*i\#$	$T\to FT'$
3	$\#E'T'F$	$i+i*i\#$	$F\to i$
4	$\#E'T'i$	$i+i*i\#$	“i”匹配
5	$\#E'T'$	$+i*i\#$	$T'\to\varepsilon$
6	$\#E'$	$+i*i\#$	$E'\to +TE'$
7	$\#E'T+$	$+i*i\#$	“+”匹配
8	$\#E'T$	$i*i\#$	$T\to FT'$
9	$\#E'T'F$	$i*i\#$	$F\to i$
10	$\#E'T'i$	$i*i\#$	“i”匹配
11	$\#E'T'$	$*i\#$	$T'\to *FT'$
12	$\#E'T'F*$	$*i\#$	“$*$”匹配
13	$\#E'T'F$	$i\#$	$F\to i$
14	$\#E'T'i$	$i\#$	“i”匹配
15	$\#E'T'$	$\#$	$T'\to\varepsilon$
16	$\#E'$	$\#$	$E'\to\varepsilon$
17	$\#$	$\#$	接受

4.6　LL(1)分析中的出错处理

在编译程序设计中，错误处理主要包含两个方面的任务：一是报错，发现错误时应尽可能准确指出错误位置和错误属性；二是错误恢复，尽可能进行校正，使编译工作可以继续下去，提高程序调试的效率。

关于如何设计错误处理程序，目前并没有特别值得关注的理论成果。本节只简单讨论 LL(1)分析过程中实现错误处理的最基本方法。

4.6.1　应急恢复

对于表驱动 LL(1)分析，在以下两种情况下需要报错：

- 栈顶的终结符与当前输入符号不匹配。
- 非终结符 A 位于栈顶，面临的输入符号为 a，但分析表 M 的表项 $M[A,a]$ 为空。

一种简单的错误恢复措施是**应急恢复**(panic-mode error recovery)。例如，在表驱动 LL(1)分析中，可以专为表项 $M[A,a]$ 为空的情形指定一些所谓的**同步符号**。在分析过程中遇到这种情形时，就跳过输入符号串中的一些符号直至遇到同步符号为止。一种简便的做法是将 FIRST(A)或 FOLLOW(A)中的所有符号当作 A 的同步符号，相应的处理过程可

设计为：

- 跳过输入符号串中的一些符号直至遇到 FOLLOW(*A*)中的符号，然后把 *A* 从栈中弹出，便可以使分析继续下去。
- 跳过输入符号串中的一些符号直至遇到 FIRST(*A*)中的符号时，可根据 *A* 恢复分析。

4.6.2 短语层恢复

另外一种称为**短语层恢复**(phrase-level error recovery)的措施比上述方法精确一些。原因在于，短语是与上下文相关的一个概念，而上述只考虑非终结符的同步符号的方法是与上下文无关的。

图 4.12 描述了短语层错误恢复可采取的一种流程：

- 在进入某个语法单位的分析时，检查当前符号 sym 是否属于进入该语法单位需要的符号集合 BeginSym。若不属于，则报错，并滤去补救的符号集合 S＝BeginSym∪EndSym 外的所有符号。
- 在该语法单位分析结束时，检查当前符号 sym 是否属于离开该语法单位时需要的符号集合 EndSym。若不属于，则报错，并滤去补救的符号集合 S＝BeginSym∪EndSym 外的所有符号。
- 无论是上述哪种情况，若遇 BeginSym 中的符号，则重新分析该语法单位；若遇 EndSyn 中的符号，则退出该语法单位的分析。

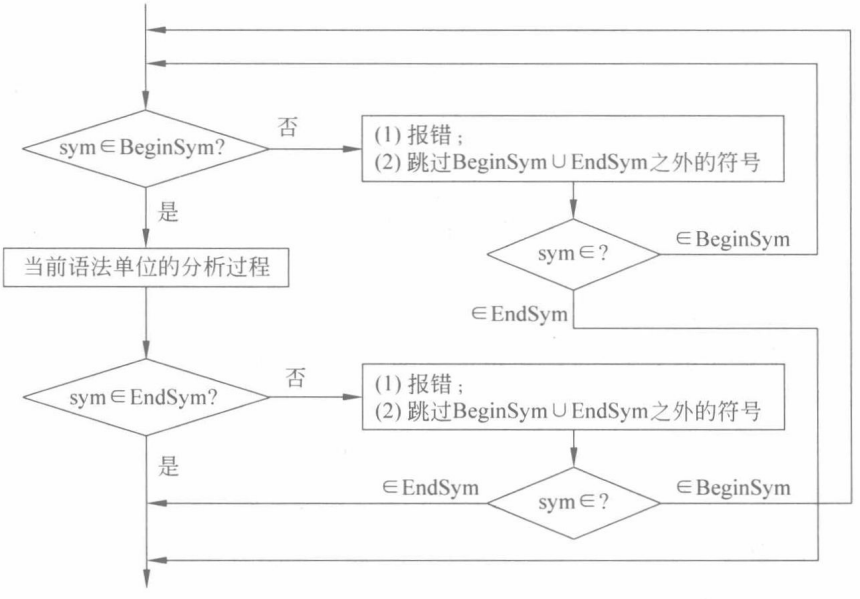


图 4.12 短语层恢复可采取的流程

下面来看在递归下降分析程序中采用短语层错误恢复的一个简单例子。设有如下文法产生式：

$$B \rightarrow [\ A\ ] \mid (\ A\ )$$

$$A \rightarrow a$$

相应于非终结符 *B*，分析子程序可设计为

```
void ParseB ( EndSym )
{
    if ( sym ∉{'[','('} )
    {
        报错;跳过 S 之外的单词符号;              /* S={ '[','(' } ∪ EndSym */
    }
    while ( sym ∈{'[','('} )
    {
        if (sym==='[' )
        {
            MatchToken( '[' );
            ParseA ( EndSym∪{']'} );
        }
        else
        {
            MatchToken( '(' );
            ParseA ( EndSym∪{')'} );
        }
        if (sym ∉EndSym )
        {
            报错;跳过 S 之外的单词符号;          /* S={ '[','(' } ∪EndSym */
        }
    }
}
```

该子程序对应于图 4.12 所描述的流程。BeginSym={'[','('},实际上是取 FIRST(B)。EndSym 作为参数,由上一级子程序传入。值得注意的是,调用下一级分析子程序 ParseA 时,根据 A 不同上下文的后跟符号而使用了不同的参数。在方括号上下文中,使用的参数是 EndSym∪{']'};而在圆括号上下文中,使用的参数是 EndSym∪{')'}。这可以体现"短语层"恢复的含义。试比较,若不区分方括号还是圆括号上下文,那么有可能使用 EndSym∪{']',')'} 作为参数,后果会如何? 读者可以思考一下。

相应于非终结符 A,分析子程序可设计为

```
procedure ParseA ( EndSym )
{
    if ( lookahead ∉{ 'a' } )
    {
        报错;跳过 S 之外的单词符号;              /* S={ 'a' }∪EndSym */
    }
    while ( lookahead ∈{ 'a' } )
    {
        MatchToken ( 'a' );
        if ( lookahead ∉EndSym )
        {
            报错;跳过 S 之外的单词符号;          /* S={ 'a' }∪EndSym */
        }
```

```
    }
}
```

4.6.3 PL/0 语法分析程序的错误处理

PL/0 编译程序中，语法分析阶段的错误处理过程采用了这种短语层错误恢复的思想，其代码结构体现了如图 4.12 所描述的流程。

PL/0 分析程序在进入和退出某个语法单位时，调用一个称为 test 的函数：

```
int test(bool* s1,bool* s2,int n)   /*s1 为需要的集合,s2 为补救的集合,n 为错误编号*/
{
    if (! inset(sym,s1))
    {
        error(n);
        while ((! inset(sym,s1)) && (! inset(sym,s2)))
        {
            getsymdo;
        }
    }
    return 0;
}
```

图 4.12 所描述的流程中，进入和离开某个语法单位时的测试任务可通过调用 test 函数实现。PL/0 分析程序中调用 test 函数时所用的 s1 和 s2 参数对应图 4.12 中的 BeginSym 和 EndSym。在进入语法单位时，s1 为 BeginSym，s2 为 EndSym；在退出语法单位时，刚好相反，s2 为 BeginSym，s1 为 EndSym。

这里的 BeginSym 被置为该语法单位的 FIRST 集合，但不含 ε；EndSym 以 FOLLOW 集合为基础，但为了提高错误恢复的质量，可以让它随不同上下文有所变化。为方便讨论，在表 4.6 中列出了表 4.3 中描述的 PL/0 部分语法单位的 FIRST 集合和 FOLLOW 集合。

表 4.6 PL/0 部分语法单位的 FIRST 集合和 FOLLOW 集合

语法单位	FIRST 集合	FOLLOW 集合
分程序	const var procedure <id> if call begin while read write	. ;
语句	<id> call begin if while read write	. ; end
条件	odd + − (ident number	then do
表达式	+ − <id> <integer> (	. ; ,) = # < <= > >= end then do
项	<id> <integer> (	. ; ,) = # < <= > >= + − end then do
因子	<id> <integer> (	. ; ,) = # < <= > >= + − * / end then do

下面以语法单位“因子”的处理为例加以说明。以下是对应的处理子程序：

```
int factor(bool* fsys,…)                    /* fsys 对应 EndSym */
```

```
{
    int i;
    testdo(facbegsys,fsys,24);              /* 调用 test 函数,facbegsys 对应 BeginSym */
    while(inset(sym,facbegsys))             /* 循环直到 sym 不是 facbegsys 中的单词种别 */
      {
          if(…)
          ……
          testdo(fsys,facbegsys,23);        /* 调用 test 函数 */
      }
    return 0;
}
```

其中,testdo() 是对函数 test()的包装。

这段代码的工作过程如图 4.12 所示,facbegsys 和 fsys 分别对应图 4.12 中的 BeginSym 和 EndSym。在进入一个语法单位“因子”的处理时,调用 test 函数检查当前单词符号的单词种别是否属于该语法单位的 facbegsys。若不属于,则报错(错误号 24),并滤去单词种别在该语法单位的 facbegsys 和 fsys 之外的所有单词符号。这之后,如果首先遇到 facbegsys 中单词种别的单词符号,那么就重新开始“因子”的处理;如果首先遇到 fsys 中单词种别的单词符号,那么就结束“因子”的处理,继续接下来的分析过程。

在语法单位分析结束时,再次调用 test 函数,检查当前当前单词符号的单词种别是否属于调用该语法单位时应有的 fsys。若不属于,则报错(错误号 23),并滤去单词种别在该语法单位的 facbegsys 和 fsys 之外的所有单词符号。同样,在这之后,如果首先遇到 facbegsys 中单词种别的单词符号,那么就重新开始“因子”的处理;如果首先遇到 fsys 中单词种别的单词符号,那么就结束“因子”的处理,继续接下来的分析过程。

fsys 随不同上下文可以有所变化。例如,如果是在处理 write 语句内部的表达式而调用 expression(fsys,…) 时,则可以将表 4.6 中“表达式”的 FOLLOW 集合传给 fsys;然而,如果是在处理“因子”下一层的表达式而调用 expression(fsys,…) 时,则可以将这个 FOLLOW 集合中的符号“,”去掉后传给 fsys。

当然,还可以有更多的考虑,如关系运算符只会出现在“条件”上下文中,所以在其他上下文中调用 expression(fsys,…) 时,fsys 中可以去掉这些关系运算符。

附录 A 的 PL/0 编译程序中,不同语法单位的错误处理过程没有统一标准,实现的精细程度有所不同。值得注意的是,一些错误恢复代码的加入影响到了整个代码的可读性,读者在阅读时应当将它们从主体代码中区分出来。

练　习

1. 对文法 $G[S]$

$S \rightarrow a \mid \wedge \mid (T)$

$T \rightarrow T,S \mid S$

(1) 给出$(a,(a,a))$和$(((a,a),\wedge,(a)),a)$的最左推导。

(2) 对文法 G 进行改写,然后对每个非终结符写出不带回溯的递归子程序。

(3) 经改写后的文法是否是 LL(1)的？给出它的预测分析表。

(4) 给出输入串$(a,a)\#$的分析过程，并说明该串是否为 G 的句子。

2. 对下面的文法 G：

$E\rightarrow TE'$

$E'\rightarrow +E|\varepsilon$

$T\rightarrow FT'$

$T'\rightarrow T|\varepsilon$

$F\rightarrow PF'$

$F'\rightarrow *F'|\varepsilon$

$P\rightarrow (E)|a|b|\wedge$

(1) 计算这个文法的每个非终结符的 FIRST 集和 FOLLOW 集。

(2) 证明这个文法是 LL(1)的。

(3) 构造它的预测分析表。

(4) 构造它的递归下降分析程序。

3. 已知文法 $G[S]$：

$S\rightarrow MH|a$

$H\rightarrow LSo|\varepsilon$

$K\rightarrow dML|\varepsilon$

$L\rightarrow eHf$

$M\rightarrow K|bLM$

判断 G 是否是 LL(1)文法，如果是，构造 LL(1)分析表。

4. 证明下述文法不是 LL(1)的。

$S\rightarrow C\$$

$C\rightarrow bA|aB$

$A\rightarrow a|aC|bAA$

$B\rightarrow b|bC|aBB$

能否构造一等价的文法，使其是 LL(1)的？并给出判断过程。

5. 文法 G 如下：

<程序>→begin<语句表>end

<语句表>→<语句>|<语句表>;<语句>

<语句>→<无条件语句>|<条件语句>

<无条件语句>→a

<条件语句>→<如果语句>|<如果语句>else<语句>

<如果语句>→<如果子句><无条件语句>

<如果子句>→if b then

试将 G 改写为 LL(1)文法，并构造其预测分析表，判断改写后的文法是否为 LL(1)文法。

6. 判断下面哪些文法是 LL(1)的，哪些能改写为 LL(1)文法，并对每个 LL(1)文法设计相应的递归下降识别器。

(1) $S \rightarrow A \mid B$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

(2) $S \rightarrow AB$

$A \rightarrow Ba \mid \varepsilon$

$B \rightarrow Db \mid D$

$D \rightarrow d \mid \varepsilon$

(3) $S \rightarrow aAaB \mid bAbB$

$A \rightarrow S \mid d\ b$

$B \rightarrow b\ B \mid a$

(4) $S \rightarrow i \mid (E)$

$E \rightarrow E+S \mid E-S \mid S$

(5) $S \rightarrow SaA \mid bB$

$A \rightarrow aB \mid c$

$B \rightarrow Bb \mid d$

(6) $M \rightarrow MaH \mid H$

$H \rightarrow b(M) \mid (M) \mid b$

7. 对于一个文法若消除了左递归，提取了左公共因子后是否一定为 LL(1)文法？试对下面的文法进行改写，并对改写后的文法进行判断。

(1) $A \rightarrow baB \mid \varepsilon$

$B \rightarrow Abb \mid a$

(2) $A \rightarrow aABe \mid a$

$B \rightarrow Bb \mid d$

(3) $S \rightarrow Aa \mid b$

$A \rightarrow SB$

$B \rightarrow ab$

(4) $S \rightarrow AS \mid b$

$A \rightarrow SA \mid a$

(5) $S \rightarrow Ab \mid Ba$

$A \rightarrow aA \mid a$

$B \rightarrow a$

(6) $S \rightarrow aSbS \mid bSaS \mid \varepsilon$

8. 按照本章介绍的消除一切左递归算法消除下面文法中的左递归(要求依非终结符的两种排序方式 S、Q、P 和 Q、P、S 分别执行该算法)：

$$S \rightarrow PQ \mid a$$

$$P \rightarrow QS \mid b$$

$$Q \rightarrow SP \mid c$$

9. 对下面的文法：

<bexpr>→<bexpr>or<bterm>|<bterm>
<bterm>→<bterm>and<bfactor>|<bfactor>
<bfactor>→not<bfactor>|(<bexpr>)|ture|false

构造一个预测分析器。

10. 请为语言 L 写一文法使其满足 LL(1)，其中 $L=\{w|w\in(a|b)^*$ 且 w 中 a、b 的个数相等$\}$。

11. 在阅读附录 A 中 PL/0 编译程序的基础上，

(1) 熟悉该编译程序的整体架构，识别出各语法单位对应的子程序；

(2) 对应表 4.3 给出的语法规则，读懂各子程序之间的调用关系；

(3) 读懂词法分析程序是如何接入到编译程序主体的；

(4) 进一步阅读 PL/0 编译程序，读懂出错处理相关的代码。

12. 对附录 A 中 PL/0 编译器源码进行裁减和改造，使其仅包含词法和语法分析过程。该分析程序读入 PL/0 语言源程序，如果没有发现词法或语法错误，再输出其相应的语法分析树。语法分析树的显示格式可自行设计，建议采用缩进的文本表示形式。